

VECTOR

News from the APL world:

- APLX Version3 charting reviewed 34
- Scholes on Programming Style 40
- Stephen Taylor on Industrial FP 52
- Ian Clark helps you rig an election 63
- Lescasse on Web Components 100
- Kai Jäger directs Excel 125



*The Journal of the
British APL Association*

A specialist Group of the British Computer Society

ISSN 0955-1433
www.vector.org.uk
Vol.21 No.3 May 2005

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (b/w or colour prints welcome). Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The Editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file or an HTML document. APL code can be accepted in workspaces from I-APL, APL+Win, IBM APL2 or Dyalog APL/W, or in documents from Windows Write (use the APL2741 TrueType font, available free from the website), and MS Word (any version, ideally using the Vector template which is also available from the website).

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Membership Rates 2005–2006

Category	Fee	Vectors	Passes
UK Private	£20	1	1
Overseas Private	£22	1	1
(Supplement for Airmail, not needed for Europe)	£4		
UK Corporate Membership	£100	5	5
Overseas Corporate	£110	5	
Sustaining	£500	10	5
Non-voting Member (Student, OAP, unemployed)	£10	1	1

The membership year normally runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa, Mastercard or JCB, at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready format (A4 or A5) with a 20mm blank border after reduction. Illustrations should be photographs (b/w or colour prints) or line drawings. Rates (excl VAT) are £250 per full page, £125 for half-page or less (there is a £75 surcharge per page if spot colour is required).

Deadlines for bookings and copy are given under the Quick Reference Diary. Advertisements should be booked with, and sent to Gill Smith, Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Tel: 01439-788385.

Email: apl385@compuserve.com.

Contents

		Page
Editorial	Stephen Taylor	2
APL NEWS		
Quick Reference Diary		4
BAA News: AGM and Accounts		5
News from Sustaining Members		8
APL Product Guide	Gill Smith	12
The Education Vector		
Zark Newsletter Extracts	edited by Jonathan Barman	18
Crossword and Puzzle		27
J-ottings 43: A Rippling Good Yarn!	Norman Thomson	30
Reviews		
Built-in Charting in APLX Version 3	Adrian Smith	34
GENERAL ARTICLES		
How to Write Computer Programs	John Scholes	40
Industrial FP: A case study	Stephen Taylor	52
How To Rig An Election	Ian Clark	63
Queens and Knights	Roger Hui	78
Time(r) for the Game of Life	Cliff Reiter	88
TECHNICAL SECTION		
APL2000 + Web Component (part 1 of 2)	Eric Lescasse	100
Notes on Virtual Memory	Bill Rutiser	115
Master and Slave: Dyalog APL Directs Excel!	Kai Jäger	125
Conformity of APL Implementations to ISO APL Standard Extended	Eke van Batenburg	135
A Skeleton Key to Windows Performance Monitoring	Bill Rutiser	141
Index to Advertisers		143

Please, Mr President, Don't Forget About the Working Man

Some of you have complained we could do more to help the working APL programmer. And you're right.

Unless the Vector Working Group plans and solicits articles, *Vector* relies on what you choose to send us. And commercial programmers rarely see they have the time to write about what they're up to. So hats off to the academics and hobbyists who have found or have just made time over the years to contribute to *Vector*. Please keep it up!

And we're resolved to serve the working programmer better. This is the last issue to feature a Technical Section, and we've packed it with useful stuff. Kai Jäger writes his complete beginner's guide to driving Excel from Dyalog APL. Excel is such an important part of our environment that we make no apology for publishing another article on interfacing it with APL. Even if you've never used OLE before, Kai's step-by-step instructions will get you started with something that works; and through the miracle of Unicode you can copy his code straight off our website into a Dyalog APL session.

Similarly, the Web has become such an important medium for delivering computer services that we're publishing in two parts Eric Lescasse's guide to writing or porting APL2000 applications to Web Services.

When I started writing in APL, I loved how it liberated me from the yammering detail of FORTRAN. No more declaring variable types or array sizes; no more nannyng loop conditions and counters. My new wings soon melted as I soared beneath the low sky of a 256Kb workspace and I had to learn how to live without WS FULL. But Moore's Law worked its magic at last, making my time scarcer than silicon, and my everyday writing is guided by linguistic (or do I mean 'literary'?) not engineering considerations. Mostly. But even everyday programming can still encounter the limits of the machines. Bill Rutiser explains in two articles how virtual memory works beneath the interpreter, and how to evaluate the impact of memory issues on your application.

MicroAPL has long emphasised cross-platform development in APL, and porting tools for migrating between interpreters. Adrian Smith offers his initial impressions of the built-in charting capability in APLX Version 3 (we'll publish a full review in one of the next few issues), and Eke van Batenburg has reviewed the how various interpreters conform to the ISO standard for APL.

John Scholes' modestly titled article argues the merits of the FP (Functional Programming) style of writing. This is illustrated in my case study of a program that interprets text files returned from mainframe queries, which parses about 700 queries a week at Britain's largest insurer. The program has 25 executable lines; 24 of them simply define functions or constants.

Reading is as important to literacy as writing. Cliff Reiter uses the well-known Game of Life to compare the performance of different strategies in J, including Ewart Shaw's 'signature' 1-line Game of Life program. Roger Hui explores in J a harder version of the Eight Queens problem. And Ian Clark considers in APL2000 an alternative to postal voting for electoral fraud.

Stephen Taylor

Letter to the Editor

From: Devon H. McCormick

I was wearing my APL2003 shirt (very limited edition – 3 and counting) in Arizona last month and one of my sister's friends commented on it. He was astonished to see APL mentioned as he had worked with it in the (late 1970s?) early 1980s in New York City for his father, Marvin Markowitz, who was apparently attempting to build an OS for a little computer IBM was developing at that time. Yes, he was one of several who lost out to Bill Gates and company.

I know this story sounds a bit suspicious – an OS in APL? – but I trust the source and he claims first-hand knowledge, not some friend-of-a-friend fable.

The other thing I find hard to believe, though it's certainly possible, was the existence of a fairly big APL group in NYC (during the time I was very active in the community) that I didn't know about. I'll have to ask this person to clarify if he was in the city or somewhere else in New York state.

Does anyone know anything about this? Maybe a Vector reader would have some recollection?

Quick Reference Diary

Date	Venue	Event
August 5-7	Blackwater	APL Moot
Oct 19-21 2005	Denmark	Dyalog 2005 Conference
Nov 6-10 2005	Florida	APL2000 User's Meeting. Naples Beach Hotel, Florida.

Dates for Future Issues of VECTOR

	Vol.21 No.4*	Vol.22 No.1	Vol.22 No.2
Copy date	3rd June	3rd June	23rd Sept
Ad booking	10th June	10th June	30th Sept
Ad Copy	17th June	17th June	7th October
Distribution	July 2005	July 2005	October 2005

* This is the Ken Iverson special issue, and the "normal" 22:1 will be produced at the same time.

Vector Back Numbers

Back numbers of Vector are available from:

**British APL Association,
c/o Gill Smith,
Brook House, Gilling East,
YORK YO62 4JJ**

Price in UK: £10 per complete volume (4 issues);
£12 (overseas); £16 (airmail) including postage.

Please note that Vol.1 No.2 is now out of stock.

Minutes of the AGM of the BAA held at the Royal Statistical Society on 20 May 2005

The Chairman, Adrian Smith, opened the meeting at 2:10 pm.

The minutes of the last AGM (printed in Vector 20.4) were accepted as a true record without demur. He announced the committee for 2005/6.

Chairman	Paul Grosvenor
Secretary	Anthony Camacho
Treasurer	Nicholas Small
Editor	Stephen Taylor
Education	Alan Mayer
Activities	Ray Cannon
Projects	Ian Clark

It was agreed not to appoint a webmaster; instead the editor will coordinate a web team.

Adrian reported a quiet year. The main activity was Ray Cannon's moot which was very successful. Vector is coming back on schedule (he showed a copy of 21.2 which will be posted to members next week). We have most of the material for the next issue and hope to be back on schedule soon. We have two new sustaining members.

Nicholas Small presented the accounts. There were no questions.

Stephen Taylor spoke about plans for Vector and the web site. He repeated that we hope to be back on schedule soon. We have an issue in memory of and to honour Ken Iverson almost ready. He expects the emphasis to move to electronic media and wants the site to become a showcase for all varieties of APL. He hopes that the Vector site will mirror sites such as Kx and J Forum and that eventually it may become the APL preferred source world-wide.

Ray Cannon announced the next APL programming moot, to be held at Blackwater from 5th to 7th August 2005. People are encouraged to bring a computer and a sleeping bag. There are convenient takeaways and a pub opposite the hall. Everyone is invited and there will be NO CHARGE (buy your own food and drink). If you are interested you are encouraged to book early with Ray. Obviously places are limited.

The meeting closed at 2:25 pm
Anthony Camacho 20 May 2005

British APL Association

Summary of Annual Accounts 2004/05

(Unaudited)

Summary of income and expenditure/receipts and payments:

	2004/05		2003/04
	(R&P)	(I&E)	(I&E)
	£	£	£
Income/Receipts			
Subscriptions	11581	12735	12914
Bank interest	1498	1498	1225
Vector advertising (incl. VAT)	544	750	1572
Other	285	285	355
	-----	-----	-----
Total receipts	13908	15268	16066
	-----	-----	-----
Expenditure/Payments			
Meetings	652	467	2338
Administration	528	628	860
Vector production and despatch	11159	14468	14634
Projects	0	0	541
Other	403	104	673
	-----	-----	-----
Total payments	12742	15668	19047
	-----	-----	-----
Assets summary:			
Bank and other balances		37497	36331
Debtors		1777	636
Creditors		(11353)	(8647)
		-----	-----
Net assets		27921	28321
		-----	-----

Notes to the accounts

Pence figures have been omitted, so columns may not add exactly.

The value of stocks of Vector have not been assessed, nor has the value of the Association's computing hardware and software.

For 2004/05, figures are shown both as income and expenditure, i.e. revenues strictly relating to the activities of that year, and as receipts and payments, i.e. what goes in and out of our bank account. The comparative figures for 2003/04 relate to income and expenditure.

Membership at 30.4.05

(previous year's figures in parentheses)

	UK		FOREIGN		TOTAL	
	Number	Vectors	Number	Vectors	Number	Vectors
Sustaining	8 (6)	43 (28)	4 (4)	28 (28)	12 (10)	71 (56)
Corporate*	2 (4)	10 (16)	1 (1)	10 (10)	3 (5)	20 (26)
Corp. Ind*	5 (9)	5 (9)	15 (15)	16 (16)	20 (24)	21 (25)
Individual	91 (106)	91 (106)	207 (209)	207 (209)	298 (315)	298 (315)
Non-voting	15 (18)	15 (18)	0 (0)	0 (0)	15 (18)	15 (18)
Life	1 (1)	1 (1)	1 (1)	1 (1)	2 (2)	2 (2)
Library	1 (1)	1 (1)	4 (6)	4 (6)	5 (7)	5 (7)
Russians			11 (11)	11 (11)	11 (11)	11 (11)
APL Groups			13 (13)	36 (39)	13 (13)	37 (39)

480 (499)

- * Add the Vector numbers in these rows to get the total subscribed for by corporate members.

News from Sustaining Members

Kx Systems

Kx Announces Executive Changes

Palo Alto (June 1, 2005) - Kx Systems, leader in high-performance databases and timeseries analysis for finance, today announced several executive changes and promotions. Effective July 1, Janet Lustgarten, a co-founder who served as Kx CEO since 1993, assumes the role of Chairman of the Board. Lustgarten is spending more time with her family and will continue to work with Kx as a strategic consultant. She is succeeded as CEO by Arthur Whitney, who becomes chief executive after 12 years as Kx CTO.

Janet has done an outstanding job of making Kx a top provider of software for financial services," said Whitney. "We are fortunate she will continue to guide Kx as the company strengthens its leadership position and accelerates its growth."

Said Lustgarten, "I am extremely proud of what the Kx team has accomplished to date and am excited about participating in its continued success. Our products have been forward-thinking for more than a decade. We anticipate that Kx will continue to solve leading-edge problems in financial services."

Simon Garland has been tapped as CTO; he has been with Kx for three years in the role of SVP Technology and brings nearly 30 years of experience in software architecture and development to his new role. Sharon Knestrick has been promoted to Director of Finance reporting to Robert Fourn, COO. Knestrick has been with Kx for two years in the role of controller.

"We have a senior management team and board of directors with many years of experience at the company," said Whitney. "And we continue to have a strong relationship with First Derivatives plc, who have provided sales management and customer services to Kx since 2001."

About Kx

Kx Systems enables leading companies to capture, analyze, compare and store vast amounts of streaming and historical data at real-time speeds. Kx products close the gap between what ordinary databases deliver and what today's real-time enterprises need. For more information and Kx and its products, visit www.kx.com.

Dyalog Ltd

Welcome to the Dyalog User Conference 2005

We are pleased to invite you to join us for the third annual Dyalog APL Conference, to be held at LO Skolen, Elsinore, Denmark, on October 19th-21st, 2005.

The annual Dyalog APL conference provides a unique opportunity to learn about the ways in which Dyalog APL is being used to deliver solutions to problems in commercial and academic environments across the globe. As usual, we have invited speakers who we believe have a story to tell which will inspire and educate - and possibly also entertain - most of us. If you think we forgot you, and you have a story you would like to tell, please contact us as soon as possible!

All participants will receive a Beta release of Version 11.0 of Dyalog APL, and Dyalog will be hosting a number of workshops to explain how our users can benefit from the new functionality in this version, which we plan to release shortly after the conference. For price and booking information, and the programme when it becomes available, see our website, <http://www.dyalog.com>.

A Few Words on Version 11.0

Work on the next version of Dyalog APL is now so far advanced that we are able to discuss some of the most significant features which we intend to include in what will be known as Version 11.0, which we expect to make generally available in the 4th Quarter of 2005:

Object Oriented Extensions

Perhaps the most significant feature of Version 11.0 will be the new system function `NEW`, which will make it possible to instantiate instances of *Classes* which are entirely defined using APL. Classes will allow APL users to take advantage of Object Oriented design techniques, which complement the traditional Array or Vector notation which "traditional" APL provides. We believe that classes will add significant new tools of thought to our arsenals, and make it easier to share and reuse code.

Core Language Enhancements

In addition to the significant new language features described above, version 11.0 will contain a handful of language enhancements, including features which have been pioneered in other APL systems. New language features will include a Function Power operator, Squish Quad Indexing, Lowest Common Multiplier (LCM) and Greatest Common Divisor (GCD).

64 Bit Version

In addition to the current 32-bit version, which will be available on the same platforms as version 10, version 11.0 will be available in a 64-bit version – initially under Windows XP/64 and AIX. The major benefit of the 64-bit version is the increase in maximum workspace size. The 64-bit version will allow you to use all the memory available in current hardware systems – up to 10^{19} bytes. Version 11.0 will have a limit of 4 billion elements on each array, which we expect to remove in 2006. The 64-bit version will be licensed separately from the 32-bit version, at a higher price – and will not be available as a free runtime.

Come to Dyalog 2005 to learn more about Version 11.0 and our plans for the years to come!

Optima Systems Ltd

They keep on telling us that there are 24 hours in a day. All I can say then is that the hours must be getting shorter. Half way through the year and we still haven't started some of the projects scheduled for the beginning of January!

We continue to be very active with our financial customers here in the UK and are making a lot of new friends. The APL systems under development have proven themselves to be fast, efficient and massive time savers. Using APL combined with some of the newer XP programming techniques and close working relationships with the business users has given us a major advantage over traditional IT department methods. Key features of the approach include;

- On-cost delivery
- On-time delivery
- Regular, incremental production releases
- Simple project management
- Little or no up-front project specification required
- Happy users
- Very happy managers
- Ecstatic directors

Work continues with our Clinical Research colleagues in America and has allowed us to re-launch the "LV" product. LV allows us to simulate the use and effectiveness of drugs around the world and match the generated profit from sales to the cost of making these sales. Once again APL gives us a flexible approach to the system design and implementation AND allows us to calculate across many large arrays of numbers.

Finally the product which actually should have started in earnest last year and is designed using Dyalog's Pocket APL should soon start to take proper shape. If we can, finally, find the time we hope to be in a position to demo it next year.

Causeway Graphical Systems Ltd

There was a song once "You never know what you've got 'til it's gone" (Joni Mitchell, I think) – for Causeway the opposite is true. More a case of "If you want to know what you've got, run a seminar on it". The first public run-out of our APL to C# conversion kit went much better than we could have hoped. It was very hard work (for us) but very rewarding for the delegates, who went away with fully working examples that they can copy and extend. All we needed to add to the converter was support for `DDL` (so an ancient LIFE program could pause for a few 10ths of a second between generations) and a couple of new overloads for strings.

So, what do we have, how do we make it more widely available, and how much should it cost?

- an engine to allow array-thinkers to code productively in VB.Net or C#. If you are a 'closet APLer' hidden in a big corporation coding new systems in a conventional language, then this could really make your day. What we can offer you is a library of array functions which run on all the native data-types and are fast and reliable. If you knew VS APL, you know these.
- an escape hatch for 'legacy' APL systems which are too damned expensive (or complex) to rewrite. Our generated C# code is well-structured and easy to read and modify. It is quite possible to make a one-off conversion to C# or VB.Net and hand over the generated code to a programming team for integration and maintenance.
- a chance to write that all-new killer-app in your favourite modern APL and ship it as a single-file EXE or DLL without giving away your APL source or bolting on any kind of runtime engine.

The array library is easy to document, and is unlikely to suffer many big changes. I think we can just sell this off the web at a very reasonable price. Look out for it on www.sharparrays.com by the time you read this. The conversion tools will be available on a 'seminar only' basis for quite a while yet, and we can take a maximum of 6 people through this at a time. This limits the availability, and suggests a 4-digit price. If you are desperate to get your hands on this, call us sooner rather than later.

The Vector Product Guide

compiled by Gill Smith

VECTOR's exclusive Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage. The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages.

Your listing here is absolutely free, will be updated on request, and is also carried on the Vector web site, with a hotlink to your own site. It is the most complete and most used APL address book in the world.

We are in the process of carrying out a full audit of the existing entries. Thanks to all those who have already replied. Note that if you do not confirm your details by the end of June, your entry will not appear in the next Vector!

You can check your address details on the following pages, and the full company information is always available on our website.

All contributions and updates to the Vector Product Guide should be sent to:
Gill Smith, Brook House, Gilling East, York, YO62 4JJ. Tel: 01439-788385,
Email: apl385@compuserve.com

ADDRESSES

ORGANISATION	CONTACT	ADDRESS, TELEPHONE, FAX, EMAIL etc.
ACM SigAPL	David Siegel	ACM, 1515 Broadway, 17th Floor, New York, NY 10036, USA (Subs only)
Adfee	Bernard Smoor	Dorpsstraat 50, 4128 BZ Leidschendam, Netherlands. Tel: +31 347 342 337 Fax: +31 347 342 342 Email: adfee@concepts.nl
Andrews	Dr Anne D Wilson	12 Thorny Hills, Kendal, Cumbria LA9 7AL, UK. Tel: 01539-731205 Email: ADWilson@kencomp.net
APL-385	Adrian Smith	Brook House, Gilling East, York YO62 4JJ, UK. Tel: 01439-788385 Email: apl385@compuserve.com
APL2000 (Europe)	Fred Honea	see APL Systems IDC SL
APL Bay Area APLBUG	Curtis Jones (Sec)	228 South 15th Street, San Jose, CA 95112-2150, USA Tel: +1 (408) 292-4060 Email: jonesca@vnet.ibm.com
APL Borealis Inc.	Richard Procter	381 Manor Road East, Toronto, Ontario M4S 1S7, Canada. Tel: (416) 457-7828. Fax: (416) 482-6582 Email: info@aplborealis.com
APL Club Austria	Harald F. Nelson	c/o N-TECH, Siebenbrunnfeldg. 4-6, A-1050 Wien, Austria. Tel: +43 1 5458063 Fax: +43 1 5458063-17
APL Germany e.V.	Dr. Reiner Nussbaum	Buchenerstrasse 78, D-68259 Mannheim, Germany. Email: reiner@nussbaum-gift.de
APL Software Services	Dick Holt	1570 Heather Field Lane, Earlsville VA 22936 USA. Tel: +1 434-973-4255 email: dick_holt@email.com
APL Solutions Inc	Eric Landau	1107 Dale Drive, Silver Spring, MD 20910-1607 USA Tel: +1 (301) 589-4621 Fax: +1 (301) 589-4618 Email: aplsi@starpower.net
APL Systems IDC SL	Fred Honea	Alfredo Marquerie, 12 - 2 F, 28034 Madrid, Spain. Tel: +34 91 730 7008 (Office) +34 60 680 5949 (Mobile). Fax: +1 775 743 6131. Email: uksales@apl2000.net
Assured Systems	Lois & Richard Hill	49 First Street, Black Rock 3193, Australia. Tel: +61 3 9589 5578 Fax: +61 3 9589 3220 Email: hillrj@melbpc.org.au
AtomicVector Productions Inc.	John Manges	413 Comanche Trail, Lawrenceville, GA 30044, USA. Tel: +1 (770) 972-3755 Email: E-mail: mailto:seapldoc@aol.com
AUSCAN Software Ltd	Richard Procter	PO Box 39, Mansfield, Ontario L0N 1M0 Canada Tel: +1-705-434-1239 Email: rjp@ca.inter.net
BACUS	Joseph De Kerf	Rooienberg 72, B-2570 Duffel, Belgium. Tel: +32 15 31 47 24
Beautiful Systems, Inc.	Jim Goff	PO Box 2235 Jenkintown, PA 19046, USA Tel: +1 (215) 635-0375; Fax: +1 (215) 635-9212 Email: info@beautifulsystems.com
Ray Cannon		21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS, UK. Tel: 01252-874697 Email: ray_cannon@compuserve.com
The Carlisle Group	Paul Mansour	544 Jefferson Avenue, Scranton PA, 18510 USA. Tel: +1 570-963-2036
Causeway Graphical Systems Ltd	Adrian Smith	The Maltings, Castlegate, MALTON, North Yorks YO17 7DP, UK. Tel: 01653-696760 Fax: 01653-697719 Email: adrian@causeway.co.uk
Paul Chapman		51B Lambs Conduit Street, London WC1N 3NB, UK. Tel: 020 7404 5401. Compuserve: 100343.3210
Cinerea AB	Rolf Kornemark	Box 61, S-193 00 Sigtuna, Sweden. Tel/Fax: +46 859 255 421 Email: rolf@cinerea.se
CODEWORK	Mauro Guazzo	Corso Lanza 105, 10133 Torino, Italy. Tel: +39 11 6601560 Fax: +39 11 6601560 Email: codework@codework- il.com
ComLog Software	Jeff Pedneau	18728 Bloomfield Road, Olney, MD 20832 USA Tel: +1 (301) 260-1435 Email: jeff@softmed.com
CoSy.com	Bob Armstrong	42 Peck Slip #4B, New York, NY 10036-1725, USA. Tel: +1 212-285-1864 Fax: +1 212-285-1864. E-mail: bob@CoSy.com.
David Crossley	David Crossley	187 Le Tour du Pont, 84210 ST DIDIER, France. Tel: +33.4.90.66.08.87 Email: crossley@au-village.com
Danish User Group	Helene Boesen	c/o Insight Systems ApS, Nordre Strandvej 119G, Hellebæk, Denmark
Dinosoft Oy	Pertti Kalliojärvi	Lönnrotinkatu 21C, 00120 Helsinki, FINLAND. Tel: +358 9 70028820 Fax: +358 9 70028824 Email: dinosoft@dinosoft.fi
Dittrich & Partner Consulting GmbH	Axel Holzmüller	Kieler Strasse 17, D-42697 Solingen, Germany. Tel: +49 212-260 660 Fax: +49 212-260 6666; Email: info@dpc.de

Dutch APL Association	Bernard Smoor (Sec)	Postbus 1341, 3430BH Nieuwegein, Netherlands. Tel: +31 347 342 337 Fax: +31 347 342 342
Dyalog Ltd.	Peter Donnelly	Grove House, Lulyens Close, Chineham Court, Basingstoke, Hampshire RG24 8AG, UK. Tel: 01256-338461 Fax: 01256-316559 Email: sales@dyalog.com; support@dyalog.com
Dynamic Logistics Systems GmbH	Michael Baas	Wilhelm-Schöffer-Str. 29, 63571 Gelnhausen Germany. Tel: +49 6051 13067 Fax: +49 6051 16142. Email: info@dls-planning.com
DynArray Corporation	Dr James Brown	16360 Monterey Rd, Suite 260, Morgan Hill, CA 95037, USA Tel: +1 (408)-782-6648 Fax: +1 (408)-782-6627 Email:info@DynArray.com
Entropy Software Limited	George MacLeod	Calle Viena 13, Residencia Larrosa, Guardamar del Segura, 03140 Alicante, Spain. Tel: +34 61 00 41 940 Email: george.macleod2003@yahoo.co.uk
Evestic AB	Olle Evero	Berteliusvagen 12A, S-146 38 Tullinge, Sweden Tel & Fax: +46 778 4410 Email: olle.evero@mail.com
FinnAPL	Olli Paavola	Suomen APL-Yhdistys RY, FinnAPL RF, PL 1005, 00101 Helsinki 10, Finland Email: olli.paavola@pyr.fi
First Derivatives plc	Michael O'Neill	First Derivatives House, Kilmorey Business Park, Kilmorey Street, Newry, Co. Down, N. Ireland BT34 2DH. Tel: 028 3025 2242 Fax 028 3025 2060 Email:enquiries@firstderivatives.com
First Derivative Analytics Ltd.	Ken Chakahwata	114 Lemsford Lane, Welwyn Garden City, Herts AL8 6YP, UK Tel/Fax: 01707-339620. Email: KenChakahwata@compuserve.com
General Software Ltd	M.E. Martin	Little Wester House, Westerhill Road, LINTON, Kent ME17 4BS Tel: 01622 832463 E-mail: martin@gsoft.plus.com
Godin London Incorporated	Gaëtan Godin	12 Gerrard St., London, Ontario, Canada N5C 4C5 Tel: +1 (519) 679-8290 Fax: +1 (519) 438-6381 Email: info@godin.on.ca
HMW Computing	Chris Hogan	Hamilton House, 1 Temple Avenue, London EC4Y 0HA, UK. Tel: 0870-1010-469; Email:HMW@4xtra.com
Hoekstra Systems Ltd	Bob Hoekstra	Dominique, Salisbury Road, Woking, Surrey, GU22 7UR, UK. Tel: 01483-771028. Fax: 01483-837324 Email: Bob.Hoekstra@HoekstraSystems.ltd.uk
Michael Hughes		28 Rushton Road, Wilbarston, Market Harborough, Leics. LE16 8QL, UK. Tel: 01536-770998 Email: Michael@Hughes.uk.com
I-APL Ltd	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS, UK. Tel: 0117-973 0036. Email: acam@blueyonder.co.uk
IBM APL Products	Nancy Wheeler	APL Products, IBM Silicon Valley Lab, Dept H36/F40, 555 Bailey Avenue, San Jose CA 95141, USA. Tel: +1 (408) 463-APL2 [+1 (408) 463-2752] Fax: +1 (408) 463-4488 Email: APL2@vnet.ibm.com
INFOTROY	Alexey Miroshnikov	11 Bolshaya Monetnaya Str., suite 6, St. Petersburg 197101 Russia. Tel: +7 812 325-9797 Fax: +7 812 233-9685 Email:aim@infotroy.ru
Insight Systems ApS	Helene Boesen	Nordre Strandvej 119G, DK-3150 Hellebæk, Denmark Tel: +45 70 26 13 26 Fax: +45 70 26 13 25 Email: info@insight.dk
JAD Software	David Crossley	175 East 96th St., Apt. 17G, New York, NY 10128 Country: USA Tel: +1 (212) 369-6713 Fax: +1 (212) 761-0124 Email: jadsms@usa.net
Japan APL Assoc	Juichiro Takeuchi	Keio Univ. Faculty of Sci.&Tech. 3-14-1 Hiyoshi, Kohoku-ku Yokohama Kanagawa, Japan 223-8522. Tel: +81(045)563-1411 Fax: +81(045)566-1617 E-mail:takeuchi@ae.keio.ac.jp
J Austria	Joachim Hoffmann	Hohenitz Nord 104, 8182 Puch bei Weiz, Austria. Tel: +43 (0)699 191 42 512 Mobile: 0043 (0)699 1 91 42 51 2. Email: joachim.hoffmann@gmx.at
JS Software Inc.	Eric Iverson	33 Major Street, Toronto, Ontario, Canada M5S 2K9. Tel: +1 (952) 470-7345 Fax: +1 (952) 470-9202 Email: info@jssoftware.com
KJK-tieto Oy	Kimmo Kekäläinen	Merikasarminkatu 10 B 56,00160 Helsinki, Finland. Tel: +358 50 55 27 207; Email: kimmo.kekalainen@kjk-tieto.com
Kx Systems	Simon Garland [Europe]	555 Bryant St., No. 375, Palo Alto CA 94301 USA. Tel: +1 866 kdb fast email: info@kx.com (in Europe simon@kx.com)
Lambert Technology Ltd	Stephen Taylor	81 South Hill Park, London NW3 2SS, UK. Tel: +44(0)20 7813 3786. Email: stj@lamberttechnology.com
Phil Last Ltd	Phil Last	146 Crossbrook Street, Cheshunt, Herts, EN8 8JY, UK. Tel: 01992-633807 Fax: 0121-359 0375 Email: phil.last@net.ntl.com

Lescasse Consulting	Eric Lescasse	18 rue de la Belle Feuille, 92100 Boulogne, France. Tel: +33.1.46.05.10.76 Fax: +33.1.46.04.60.23 Email: eric@lescassee.com
Lingo Allegro USA, Inc	Walter G. Fil	203 N. LaSalle Street, Suite 2100, Chicago, Illinois 60601, USA. Tel: +1 (800) 546 4621 E-mail: lingo-allegro@visto.com
Lucas Solutions	Jim Lucas	Stubbedamsvej 9C, 3. tv., 3000 Helsingør, Denmark Tel: +45 49 26 52 42. Email: jel@danbbs.dk
Alastair Kinloch Ltd	Alastair Kinloch	519 Webster's Land, Edinburgh EH1 2RX, Scotland, UK. Tel: +44 (0)7802 430 202 Email: alastair.kinloch@btinternet.com
MasterWork Software Ltd	Fraser Jackson	PO Box 56-036, Tawa, Wellington, New Zealand. Tel: +64 (4) 232-4440 Fax: +64 (4) 232-4452 Email: fraser.jackson@xtra.co.nz
Mercia Software Ltd.	Gareth Brentnall	Mercia House, Ashted Lock, Aston Science Park, Birmingham, B7 4AZ, UK. Tel: 0121-359 6096. Fax: 0121-359 0375
MicroAPL Ltd.	Richard Nabavi	The Roller Mill, Mill Lane, Uckfield, E.Sussex TN22 5AA Tel: 01825 768050. Fax: 01825 749472 Email: MicroAPL@microapl.demon.co.uk
Milinta Inc.	Dan Baronet	Contact Dan Baronet at dbaronet@milinta.com
Ellis Morgan	Ellis Morgan	Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants GU32 3PE, UK. Tel: 01730-263843 Email: ap@ellismorgan.co.uk
Dr. Nussbaum gift mbH	Dr Reiner Nussbaum	Buchenerstrasse 78, D-68259 Mannheim, Germany Tel: +49 621 7152190, Email: reiner@nussbaum-gift.de
NY Sig	David Siegel	PO Box 2697, New York, NY 10163-2697, USA. Email: NYSIGAPL@ACM.ORG
Oasis b.v.	Louis Rijkse	Lekstraat 4, 3433 ZB Nieuwegein, Holland. Tel: +31 30 60 66 336 Fax: +31 30 60 65 844 Email: rijks@oasis.nl
Omega Computing Inc	Alan Graham, Andrew Chou	3 Columbus Avenue, Edison, NJ 08817, USA. Tel: +1 (732) 985 9519 Email: alangraham@mindspring.com
Optima Systems Ltd	Paul Grosvenor	Optima House, Mill Court, Spindle Way, Crawley, West Sussex, RH10 1TT, UK. Tel: 01293 562700 Fax: 01293 562699 Email: paul@optima-systems.co.uk
Qualed Inc.	Nicole Schless Georges Brigham	121 West Main Street, Milford, CT 06460, USA. Tel: +1 (203) 874-4334 Fax: +1 (203) 876-9083. Email: sales@qualed.com; info@qualed.com
RadSys Technologies AB	Randolph Schrab	Lovsångarv. 18, S-756 52 Uppsala, Sweden. Tel: +46 18 32 41 53 Fax: +46 708 1996 11 Email: randolph.schrab@radsys.se
Renaissance Data Systems	Ed Shaw	P.O. Box 511, Botsford, CT 06404, USA. Tel: +1 (203) 270-9729 Email: aplbooks@earthlink.com
Resources & Results	Frank Rhodes	342 Glen Oaks Blvd. Dallas, TX 75232, USA. Tel: +1 (817) 501-9825 Email: frank21@decision-support.net
RE Time Tracker Oy	Richard Eller	Mikonkatu 8 A, 2.krs, PL 363, 00101 Helsinki, Finland. Tel: +358 9-621 3300 Fax: +358 9-621 3378 Email: re@rett.fi
Right Seat Software, Inc.	Torn Atkins	1110 12th Street, Golden, CO 80401, USA. Tel: +1 303 278 2244. Fax: +1 303 278 6967. Email: info@voxyproxy.com
Graeme Robertson	Dr. Graeme D. Robertson	15 Little Basing, Basingstoke, Hants RG24 8AX, UK. Tel: +44 (0) 1256-364071 Email: GraemeDR@nildram.co.uk
The Rochester Group Inc.	Robert Miller	600 Park Avenue, Rochester, NY 14607-2926, USA. Tel: +1 (716) 271-1110. Fax: +1 (716) 271-1230
Rome/Italy SIG	Mario Sacco	Casella Postale 14343, 00149-Roma Trullo, Italy Email: mario.sacco@tin.it
SE APL Users Group	John Manges	413 Comanche Trail, Lawrenceville, GA 30044, USA Tel: +1 (770) 972-3755 Email: seapldoc@aol.com
Shepp & Associates LLC	Andrew Shepp	1312 Washington Avenue, 6th Floor St. Louis MO 63103, USA Tel: +1 (314) 621-3272 Fax: +1 (314) 621-4267 Email: ashepp@compuserve.com
Skelton Consulting GmbH	Charles Skelton	Weinklinge 9, 70329 Stuttgart, Germany. Email: c.skelton@skelton.de
Snake Island Research Inc	Bob Bernecky	18 Fifth Street, Ward's Island, Toronto, Ontario M5J 2B9 Canada Tel: +1 (416) 203-0854 Fax: +1 (416) 203-6999 Email: bernecky@interlog.com
SOCAL (South California)	Roy Sykes Jr	Sykes Systems Inc, 4649 Willens Ave, Woodland Hills, CA 91364-3812 USA. Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Soliton Associates		Soliton Inc, 44 Victoria Street, Suite 2100, Toronto, ON M5C 1Y2 Tel: +1 (416) 364 9355 Email: sales@soliton.com

SovAPL Russian Chapter of SIGAPL	Alexander Skomorokhov	PO Box 5061, Obninsk-5, Kaluga Region 249020, Russia Tel: +7(08439)47109 Fax: +1 (530) 6885510 Email: askom@obninsk.com
Strand Software Inc	Anne Faust	P.O. Box 330, Excelsior, MN 55331 USA Tel: +1 (952) 470-7345 Fax: +1 (952) 470-9202 Email: info@strandsoft.com
Rex Swain	Rex Swain	8 South Street, Washington, CT 06793 USA. Tel: +1 (860) 868-0131 Fax: +1 (860) 868-9970 Email: rex@rexswain.com
SwedAPL	Christer Ulfhielm	Novator Consulting Group AB, Svärdvägen 11C, S-182 33 Danderyd Sweden. Tel: +46 8 622 63 50 Fax: +46 8 622 63 51 CService: 100341.404
Swiss APL User Group	Urs Oswald (Chairman)	Nordstrasse 292, 8037 Zürich, Switzerland. Email: osurs@bluewin.ch
Sykes Systems Inc	Roy Sykes Jr	4649 Willens Ave., Woodland Hills, CA 91364, USA Tel: +1-818-347-5779 Email: roysykes@earthlink.net
Toronto SIG	Richard Procter	PO Box 55, Adelaide St. Post Office, Toronto Ontario M5C 2H8, Canada Email: info@torontoapl.org
ubJL GmbH (APL Software Team)	Kai Jäger	Thusneldastr. 22, 90482 Nürnberg, Germany. Tel + 49 911 482512; Fax: +49 911 482518 Email: jaeger@ubjl.de
Weighahead Systems	Philip Bulmer	Camberley House, 1 Portesbury Road, Camberley, GU15 3RB, UK. Tel +44 1276 20789 Email: sales@weighahead.com
Stephen Wynn		8 Clarence Gardens, Brighton, Sussex BN1 2EG, UK. Tel: 01273-327238 Email: centre@bolblue.com
Zark Incorporated	Gary A. Bergquist	23 Ketchbrook Lane, Ellington CT 06029, USA. Tel: +1 (860) 872-7806

FTP SITES

IBM APL2	ftp.software.ibm.com/ps/products/apl2
Waterloo Archive	archive.uwaterloo.ca/ftparch/languages/apl
APL-to-ASCII	archive.uwaterloo.ca/languages/apl/workspaces/aplascii

WORLD WIDE WEB SITES

ACM SigAPL	www.acm.org/sigapl/
AFAPL	www.afapl.asso.fr/ (Journal available on line)
APL2000	www.APL2000.com/
APL-385	www.demon.co.uk/apl385/
APL Forum	www.aplforum.com
APL Germany e.V.	www.apl-germany.de
APL Journal, Germany	http://www.rhombos.de/shop/a/show/article/7216
APL Borealis Inc.	www.aplborealis.com
APL Systems IDC SL	www.apl2000.net
Assured Systems	www.assuredsystems.com.au/
AUSCAN	http://home.ca.inter.net/rjp/auscan/
Eke van Batenburg	www.bio.leidenuniv.nl/~Batenburg/index.html
Carlisle Group	http://carlislegroup.com/
Causeway	www.causeway.co.uk/
CODEWORK	www.codework-it.com/tangram/eng/
CoSy (Bob Armstrong)	CoSy.com/
Dinosoft Oy	www.dinosoft.fi/
Dittrich & Partner	www.dpc.de ; www.apl-online.de
DMOZ - Open Directory	http://dmoz.org/Computers/Programming/Languages/APL/
Dyalog Ltd	www.dyalog.com/
DynArray	www.dynarray.com/
Dynamic Logistics Systems GmbH	www.dls-planung.de www.dls-planning.com
FinnAPL	www.pyr.fi/apl/

First Derivatives plc	www.firstderivatives.com
Godin London Inc	www.godin.com/
Houben	http://home.wanadoo.nl/houb/IDA.htm
Hoekstra Systems	www.HoekstraSystems.ltd.uk/
IBM APL2	www.ibm.com/software/awdtools/apl
Infostroy	www.infostroy.ru
Insight Systems ApS	www.insight.dk/
Japan APL Association	www.ae.keio.ac.jp/lab/soc/takeuchi/japla/
JSoftware Inc	www.jsoftware.com/
KJK-tieto Oy	www.kjk-tieto.com
kx systems	www.kx.com
Lambent Technology	www.lambenttechnology.com
Lescasse Consulting	www.lescasse.com/
Lingo Allegro USA Inc	www.lingo.com/
MicroAPL Ltd	www.microapl.co.uk/apl
Milinta Inc	www.milinta.com
Nussbaum gift	www.nussbaum-gift.de
Oasis b.v.	www.oasis.nl/
Optima Systems Ltd	www.optima-systems.co.uk
Qualedi, Inc.	www.qualified.com
Renaissance Data	www.aplbooks.com/
Resources & Results	www.decision-support.net
RE Time Tracker Oy	www.rett.fi/
Right Seat Software, Inc.	www.voxproxy.com
The Rochester Group Inc.	www.rochgrp.com/
Shepp & Associates	www.digitravel.com/
SigAPL	www.acm.org/sigapl/
Skelton Consulting GmbH	www.skelton.de
Snake Island Research Inc.	www.snakeisland.com
Soliton	www.soliton.com/
Strand Software Inc.	www.strandsoft.com/
Rex Swain	www.rexswain.com/
Toronto SIG (for Toolkit)	www.torontoapl.org/
ubJL GmbH (APL Software Team)	www.aplteam.de
Jim Weigang	www.chilton.com/~jimw/
Weighahead Systems	www.weighahead.com

Zark Newsletter Extracts

edited by Jonathan Barman

Utility Corner: Market Days

(The purpose of this column is to make you more productive by introducing you to utility functions. Think of utility functions as APL functions that have names instead of symbols. By expanding your function vocabulary, you'll be able to write APL code that's more concise, more efficient, and more readable.)

In last issue's *Limbering Up* column, you were asked to define a dyadic utility function `DAYSPLUS` that allows you to do "date addition" with a "Market Day" calendar.

When working with investment data, a special calendar must be used – the calendar that has one day per "Market Day," i.e. per day that the securities market is open for trading. The market is not open weekends nor on certain holidays. For certain analyses, it is necessary to move forward or backward a certain number of market days. How is this done?

Suppose the "Market Day" calendar is expressed in two arrays, `ciy` and `cid`:

```
      ciy
1994 1995
```

```
      pcid
2 12 31
```

```
      cid
WDDDDDDWDDDDDDWDDDDDDWDDDDDDWDD
DDDDWDDDDDDWDDDDDDWDDDDDDWDDDDW
DDDDWDDDDDDWDDDDDDWDDDDDDWDDDDW
HWWDDDDDDWDDDDDDWDDDDDDWDDHDDWX
WDDDDDDWDDDDDDWDDDDDDWDDDDDDWDD
DDDDWDDDDDDWDDDDDDWDDDDDDWDDDDX
DWWHDDDDWDDDDDDWDDDDDDWDDDDDDW
DDDDDDWDDDDDDWDDDDDDWDDDDDDWDD
DDWWHDDDDWDDDDDDWDDDDDDWDDDDDX
WDDDDDDWDDDDDDWDDDDDDWDDDDDDWDD
DDDDWDDDDDDWDDDDDDWDDDDHWWDDDX
DDWWDDDDDDWDDDDDDWDDDDDDWDDDDW
```

```

WHDDDDWDDDDWDDDDWDDDDWDD
DDDWDDDDWDDDDWDDDDWDDXX
DDDWDDDDWDDDDWDDDDWDDDD
WDDDDWDDDDHWWDDDDWDDDDWX
DDDDWDDDDWDDDDWDDDDWDD
DDWDDDDWDDDDWDDDDWDDDDX
WWDHDDWDDDDWDDDDWDDDDWDD
DDDDWDDDDWDDDDWDDDDWDDDD
DWWDHDDWDDDDWDDDDWDDDDWX
WDDDDWDDDDWDDDDWDDDDWDD
DDDWDDDDWDDDDWDDDHWWDDDX
DWDDDDWDDDDWDDDDWDDDDWDD

```

Here, *ciy* is a vector of the years in the calendar. *cid* is a 3-dimensional character array with one plane per year in *ciy*, 12 rows (months), and 31 columns (days). Thus, the ninth row of the second plane corresponds to September 1995:

```
DWWHDDDDWDDDDWDDDDWDDDDWX
```

The *Ds* correspond to market days, the *Ws* to weekend days, the *Hs* to holidays, and the *Xs* to nonexistent days. For example, September 4 is a holiday (Labor Day); September 2 and 3 are weekend days; and September 31 does not exist.

The left argument of the proposed utility function *DAYSPLUS* is an array of market day dates. The right argument is a corresponding array of numbers of days to add. The result is an array of the market day dates that come from the date "addition." For example,

```

19950906 DAYSPLUS 2 0 -1 -3
19950908 19950906 19950905 19950831

```

The arguments should be scalar conformable, you know, vector and vector returns vector, matrix and scalar returns matrix, and so on. Assume globals *ciy* and *cid*.

The solution from Rex Swain is direct and efficient:

```

▽ R←YMD DAYSPLUS N;C;Y;IO
[1]  A Returns date that is N market days
[2]  A after date YMD (yyyymmdd). Logic
[3]  A assumes YMD is a market day.
[4]  A Arguments are scalar conformable.
[5]  A Globals:
[6]  A   ciy - vec of years
[7]  A   cid - 3D char array of years by
[8]  A           12 months by 31 days
[9]  A   (D=mkt day; W=weekend;
[10] A   H=holiday; X=non-day)
[11] A
[12] IO←0 A Origin 0 is simpler
[13] Y←ciy[0]-1 A Base year
[14] R←YMD-10000×Y A YYYYMMDD → IMMDD
[15] R←0 100 100⊔R A IMMDD → I MM DD
[16] R←0 12 31⊔R-1 A I MM DD → cid inds
[17] C←+\\,'D'=cid A Cumulative mkt days
[18] R←C⊔C[R]+N A Perform "addition"
[19] R←1+0 12 31⊔R A cid inds → I MM DD
[20] R←0 100 100⊔R A I MM DD → IMMDD
[21] R←R+10000×Y A IMMDD → YYYYMMDD
▽

```

Let's walk through the logic in this function with an example:

```
19950906 DAYSPLUS 2 0 -1 -3
```

```

[13] Y←ciy[0]-1 A Base year
      Y : 1993

[14] R←YMD-10000×Y A YYYYMMDD → IMMDD
      YMD : 19950906
      R : 20906

[15] R←0 100 100⊔R A IMMDD → I MM DD
      R : 2 9 6

[16] R←0 12 31⊔R-1 A I MM DD → cid inds
      R : 625

[17] C←+\\,'D'=cid A Cumulative mkt days
      C : 0 0 1 2 3 4 5 5 5 6 7 ...

```

```

[18]  R←C1C[R]+N A Perform "addition"
      C[R] : 424
      C[R]+N : 426 424 423 421
      R : C1C[R]+N : 627 625 624 619

[19]  R←1+0 12 31⌈R A cid inds → I MM DD
      R : 2 2 2 2
          9 9 9 8
          8 6 5 31

[20]  R←0 100 100⌊R A I MM DD → IMMDD
      R : 20908 20906 20905 20831

[21]  R←R+10000×Y A IMMDD → YYYYMMDD
      R : 19950908 19950906 19950905 19950831

```

The APL "magic" that allows this function to work without looping takes place on lines [17] and [18]. The vector C designates, for every day in the calendar, the number of market days that have elapsed since the beginning of time (or, rather, since the beginning of the calendar). The expression $C1C[R]+N$ does the "addition" by extracting the number of cumulative days for each given date, adding N days, and searching for the market date that corresponds to that new number of cumulative days.

The beauty of the expression $C1C[R]+N$ is that it works for any shapes of R and N (as long as they are scalar conformable). For example, R can be a matrix and N a scalar, or R a scalar and N a matrix, or they can both be matrices of the same shape.

The submission from Jim Weigang used the same algorithm as above but refined the technique a bit by converting the year, month, and day values to origin 0 indices all at once. In the function above the adjustments occur on lines [13], [14], [16], [19], and [21]. In the function below, they occur only on lines [15], [17], and [23].

```

      ▽ R←YMD DAYSPLUS N;A;B;C;Y;⊞IO
[1]  A Returns date that is N market days
[2]  A after date YMD (yyyymmdd). Logic
[3]  A assumes YMD is a market day.
[4]  A Arguments are scalar conformable.
[5]  A Globals:
[6]  A   ciy - vec of years
[7]  A   cid - 3D char array of years by
[8]  A           12 months by 31 days
[9]  A   (D=mkt day; W=weekend;
[10] A   H=holiday; X=non-day)
[11] A
[12] ⊞IO←0 A Origin 0 is simpler
[13] A←0 100 100 A Date-packing radix
[14] B←0 12 31 A cid-indexing radix
[15] Y←A⊞ciy[0],1 1 A Origin adjuster
[16] C←+\,'D'=cid A Cumulative mkt days
[17] R←YMD-Y A YYYYMMDD → IMMDD (⊞IO=0)
[18] R←A⊞R A IMMDD → I MM DD
[19] R←B⊞R A I MM DD → cid inds
[20] R←C⊞C[R]+N A Perform "addition"
[21] R←B⊞R A cid inds → I MM DD
[22] R←A⊞R A I MM DD → IMMDD
[23] R←Y+R A IMMDD → YYYYMMDD

```

▽

While there is a certain aesthetic appeal to this refinement, the efficiency of the function improved by only a fraction of a percent.

A side note: Many APL beginners are lured by the appeal of writing code that is compact as well as powerful. On encountering a function like the one above, they are tempted to condense the function by combining lines [17] to [23] like so:

```
[17] R←Y+A⊞B⊞C⊞C[B⊞A⊞YMD-Y]+N
```

They may even include a comment for this line like, "Do the necessary conversion." Resist this temptation! The joyful feeling that comes from squeezing one more rabbit in the hat will be more than offset by the feeling of despair that comes from looking at this line of code in four months and having no idea what it does.

Happily, the function does not run faster by jamming all the code onto one line, so don't offer efficiency as an excuse. And the concept of saving storage by omitting comments is analogous to swallowing your food whole to conserve your teeth.

Enough preaching.

Another submission, from Bob Santos, takes the same approach as the above functions, but does the "addition" a bit differently.

```

      V R+YMD DAYSPLUS N;A;B;I;Y;[]IO
[1]  A Returns date that is N market days
[2]  A after date YMD (yyyymmdd). Logic
[3]  A assumes YMD is a market day.
[4]  A Arguments are scalar conformable.
[5]  A Globals:
[6]  A   ciy - vec of years
[7]  A   cid - 3D char array of years by
[8]  A         12 months by 31 days
[9]  A   (D=mkt day; W=weekend;
[10] A   H=holiday; X=non-day)
[11] A
[12] []IO←0 A Origin 0 is simpler
[13] A←0 100 100 A Date-packing radix
[14] B←0 12 31 A cid-indexing radix
[15] Y←A⊥ciy[0],1 1 A Origin adjuster
[16] I←,'D'=cid A Cumulative mkt days
[17] I←I/ιρI A Bits to indices
[18] R←YMD-Y A YYYYMMDD → IMMDD ([]IO=0)
[19] R←A⊥R A IMMDD → I MM DD
[20] R←B⊥R A I MM DD → cid inds
[21] R←I[N+I⊥R] A Perform "addition"
[22] R←B⊥R A cid inds → I MM DD
[23] R←A⊥R A I MM DD → IMMDD
[24] R←Y+R A IMMDD → YYYYMMDD
      V

```

Compare the "addition" methods

Cumulative market days —

```

C←+\\,'D'=cid
R←C⊥C[R]+N

```

Index of market days —

```

I←,'D'=cid
I←I/ιρI
R←I[N+I⊥R]

```

Both methods do, 'D'=cid. The former uses +\\ on the resulting bit vector. The latter uses /ιρ on it. The former method uses [] and then ⊥; the latter uses ι and then []. The two methods are similar enough that the relative efficiency will depend on the implementation of APL you use.

When timing these algorithms, it becomes apparent that much of the time consumed by these functions is spent on the π or π operations. Anything that can be done to reduce this time will improve the overall efficiency of the functions. For example, in APL*PLUS II/386, the π line of the latter function can be replaced by

```
[17] I←WHERE I A Bits to indices
```

where the WHERE function is a "compiled" function from the APL*PLUS workspace ASMFNS. The resulting DAYSPLUS function takes about 60% as long to run as the function without WHERE.

An even faster solution is to precompute the C or I variable once, and pass it as a global variable to the function. Here's how this approach looks (for the latter function):

```
▽ R←YMD DAYSPLUS N;A;B;Y;IO
[1] A Returns date that is N market days
[2] A after date YMD (yyyymmdd). Logic
[3] A assumes YMD is a market day.
[4] A Arguments are scalar conformable.
[5] A Globals:
[6] A   ciy - vec of years
[7] A   cid - 3D char array of years by
[8] A       12 months by 31 days
[9] A   (D=mkt day; W=weekend;
[10] A   H=holiday; X=non-day)
[11] A   cii - I/IpI←,'D'=cid (IO=0)
[12] IO←0 A Origin 0 is simpler
[13] A←0 100 100 A Date-packing radix
[14] B←0 12 31 A cid-indexing radix
[15] Y←A⊥ciy[0],1 f A Origin adjuster
[16] A cii←,'D'=cid A Cumulative mkt days
[17] A cii←cii/tpcii A Bits to indices
[18] R←YMD-Y A YYYYMMDD → IMMDD (IO=0)
[19] R←A+R A IMMDD → I MM DD
[20] R←B⊥R A I MM DD → cid inds
[21] R←cii[N+cii⊥R] A Perform "addition"
[22] R←B+R A cid inds → I MM DD
[23] R←A⊥R A I MM DD → IMMDD
[24] R←Y+R A IMMDD → YYYYMMDD
```

▽

This function runs faster than any other function we could devise. It runs in about 75% the time of the original (Swain) function with its C variable precomputed. The reason for this difference is that the vector you get from the π is about 67% the

length of the vector you get from `+ \` (because weekends, holidays, and invalid days are eliminated).

As a mental exercise, how would you modify the above function to develop another dyadic function name `DAYSDIFF` whose arguments are both arrays of dates, and whose result is an array of the number of market days between those dates? Which method (`+ \` or `/:p`) would be faster? How about a monadic function named `DAYSVALID` whose argument is an array of dates, and whose result is a Boolean array flagging the dates that are valid market days? Which method would be faster?

Limbering Up: Market Days

(The purpose of this column is to work some flab off you APL midsection. Like muscles, your APL skills can atrophy if not exercised with adequate frequency and variety. This column presents a task for you to perform. Set aside a few minutes from your busy schedule and work the task. Mail your solution and stay tuned for the results.)

One of the features of APL is that you can use exponential, or scientific, notation to express numbers. For example, you can type `1E6` instead of `1000000`. This convenience is, however, a two-way street. The computer also has the right to speak to you in exponential notation. For example, you can try the following in immediate execution mode:

```
.00000123
0.00000123
.000000123
1.23E-7
1230000000
1230000000
123000000000
1.23E10
```

It appears the computer, too, grows weary of typing too many zeros. From a fairness point of view, we are comforted. However, at times you will find exponential notation unacceptable.

For example, suppose you develop an application that prompts the user for a "Daily Declared Rate" for a Mutual Fund:

```
Daily Declared Rate: .000000125
```

Your program digests the number and files it. Later that day, the user wants to review the rate for subsequent modification. Here's what he sees:

```
Daily Declared Rate: 1.25E-7
```

Is this the number he entered or not? The answer is not immediately obvious. What do you, as the programmer, do?

One simple approach is to tell the computer to stop using exponential notation:

```
      .000000125
1.25E-7
      □EN
1
      □EN+0
      .000000125
.000000125
      ‡.000000125
.000000125
```

Don't you wish? Unfortunately, there is no such □EN system variable.

That's where you come in. Your task is to write a monadic function ENOFF (Exponential Notation OFF) that behaves exactly like monadic ‡ except it never returns its formatted numbers in exponential notation.

Send your solution to:

Vector Production
 Brook House
 Gilling East
 York YO62 4JJ
 UK email apl385@compuserve.com

The notable functions and their authors' names will be printed in the next issue of *Vector*. Good luck and happy limbering.

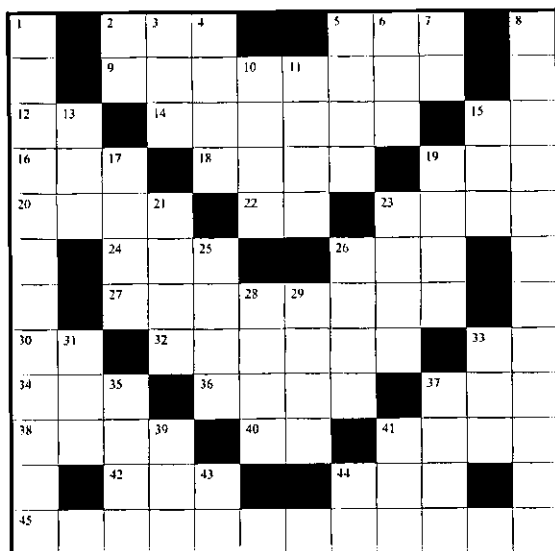
Reprinted with kind permission from Zark APL Tutor News, a quarterly publication of Zark Incorporated, 23 Ketchbrook Lane, Ellington, CT06029, USA

21.3 Solution, Puzzle, and New Crossword

Solution to Crossword in 21.2

I	N	S	E	R	T	A	N	A	O	R	P
N	×	-	?	L			C	P	×	ρ	R
N	φ	L	B		×	Z		L	ι	K	O
E	I	M		÷	\	I	Y		3	†	D
R	D		√	/		W	×	Q		Q	U
P		L	.	5	+		S	I	C		C
R		K	≠	2		⊗	B	÷	T		T
O	R		V	?	U		*	N		S	I
D	ι	D		N	×	o	D		1	€	V
U	Γ	-	N		P	P		F	⊘	B	I
C	/	ι	ρ	G			A	Γ	Γ	/	T
T	R	I	G	O	N	O	M	E	T	R	Y

New Crossword



Across

2. Centre of the universe
5. $S - V \times \{ S \div V$
9. Set V as a bit vector flagging rows of CI that contain the vector S
12. $\square$ to vaporize unwelcome objects
14. $\{AD \uparrow MBT\}$, $AD \uparrow MBT$ for positive AD
15. $\square$ tells where to break long output lines
16. 1 , $(A-1) \rho 0$ for scalar A
18. $,1 \quad \neg 1 \circ . \times V$ for vector V
19. Flag leading 1 s in bit vector E
20. OK different random numbers from $1 \ 8$
22. $\square$ to define a local function
23. $MR[;1] - MR[;2]$ for two-column MR
24. Number of elements in vector $V1$
26. $K \times K > 0$
27. Index of 1st element of vector V that is twice as large as the biggest number in vector NV
30. $\times i$ as in $\rightarrow END \times i A \in E$
32. Number of columns in MAT
33. $(F > 0) - F < 0$
34. B formatted to $R[2]$ decimal places within width $R[1]$

36. Stick V as a new column to the front of matrix 1÷B
37. $(5\rho F[1]), (5\rho F[2]), (5\rho F[3]), \dots$
38.)_____ to bring in more functions
40. □_ to get a list of vars and/or fns
41. √/GI for vector GI
42. Symbols that convert bits to indices
44. $(\sim S \in C) / C$
45. Raise to powers

Down

1. $\circ R * 2$ for radius R
2. □_, the list of available characters
3. Make a copy of A and name it P
4. $(L + DV) \geq 1$ for Boolean L and DV
5. Flag with bits elements of vector BV that begin subvectors that match V
6. $(CT * 2) * 0.5$
7.)__ to see what's not finished
8. ... 0.2 1 5 25 125 625 ...
10. ~_____ is the alternating sum of the elements of matrix F from end to start
11. X below SM
13. The first X elements of vector K
15. The value of Boolean P with the elements at indices P/ιρP replaced by M
17. One random index, ravelled, from vector V
19. $\sim 0 \in KV$ for bit vector KV
21. ' ' ρBV / ι ρBV for bit vector BV
23. | -NT
25. All but the first 12 elements of V
26. $((1 \uparrow \rho AB), 0) \rho AB$ for matrix AB
28. Is N non-empty?
29. The quotient of M and L, rounded up
31. Format values in O (not zero) to F decimal places
33. $G[1] * G[2] * G[3] * \dots$
35. $P\{BP / \iota \rho BP\}$
37. Does SA contain a 5?
39. Where O first occurs in vector Y
41. $(\sim 1 \in I) \rho 1$
43. Number of rows, columns in matrix N
44. □_ to define "close enough" when deciding whether numbers are equal

J-ottings 43: A Rippling Good Yarn!

by Norman Thomson

A "ripple shuffle" of a deck of cards consists of dividing it into two halves (or as nearly as possible if there is an odd number of cards) and taking one card in turn from each half. How many shuffles will it take for a deck of, say, 52 cards to return to its original state?

Suppose that the cards are numbered consecutively from zero and that the total number of cards is even. Then if all the even-numbered cards are placed together in order on top of all the odd-numbered ones, a ripple shuffle would exactly restore the original order. To put this in another way, a ripple shuffle is the inverse of "all evens first" sort. The dyadic form of the primitive verb *grade-up* is very convenient for the purpose of describing the "all evens first" sort, or equivalently the inverse ripple shuffle. To quote from the J help file, $x/:y$ is defined as $(/:y)\{x$, that is x is sorted to an order specified by y . Thus if y is set up as a vector of alternating 0s and 1s the effect is to obtain all the items of one parity followed by all the items of the other. The hook $\$~\#$ applied as $0(\$~\#)v$ is convenient for generating a vector of zeros equal in length to an arbitrary vector, and can readily be adapted to generate alternating 0s and 1s, leading to the following inverse ripple shuffle verb:

```
irs=.: 0 1&($~)@#      NB. inverse ripple shuffle
irs i.10
0 2 4 6 8 1 3 5 7 9
irs 'abcdef'
acebdf
```

The power adverb makes repeated application of this process a trivial matter:

```
irs^(i.7)i.10          NB. 7 inv. ripple shuffles
0 1 2 3 4 5 6 7 8 9
0 2 4 6 8 1 3 5 7 9
0 4 8 3 7 2 6 1 5 9
0 8 7 6 5 4 3 2 1 9
0 7 5 3 1 8 6 4 2 9
0 5 1 6 2 7 3 8 4 9
0 1 2 3 4 5 6 7 8 9
```

Once the original order is restored, a succession of ripple shuffles is obtained by reading the rows of the above table from bottom to top. To count the number of shuffles required to restore the original order, the direction in which the table is read is immaterial, so define a straightforward iterative counting verb:

```

counters=.3 :0      NB. no of ripples to restore original order
r=.y. [ i=.1
while. -.y.-:irs r do. r=.irs r [ i=.i+1 end. i
)
counters i.10
6

```

For a deck of 52 cards the number of ripple shuffles required to restore the original order is

```

counters i.52
8

```

A range of even card numbers can be explored, and in some cases the results may be a little surprising at first sight:

```

EACH=.&.>
w,:>counters EACH i. EACH w=.2+2*i.20
2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40
1 2 4 3 6 10 12 4 8 18 6 11 20 18 28 5 10 12 36 12

w,:>counters EACH i. EACH w=.22+2*i.20
22 24 26 28 30 32 34 36 38 40 42 44 46 48 50 52 54 56 58 60
6 11 20 18 28 5 10 12 36 12 20 14 12 23 21 8 52 20 18 58

w,:>counters EACH i. EACH w=.64+4*i.15
64 68 72 76 80 84 88 92 96 100 104 108 112 116 120
6 66 35 20 39 82 28 12 36 30 51 106 36 44 24

```

Intuitively it might be expected that numbers which are relatively rich in factors would have relatively short run-lengths, however this is not in general the case. For example, compare 60 with 92.

A variation on the ripple shuffle is to do a final exchange of the two half-decks. This is achieved by simply changing 0 1 to 1 0 in the verb `irs` :

```

irss=.: 1 0&($~)@#

```

After defining `counters` by changing `irs` to `irss` in `counters`, some corresponding results are :

```

w,:>counterss EACH i. EACH w=.2+2*i.20
2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40
2 4 3 6 10 12 4 8 18 6 11 20 18 28 5 10 12 36 12 20

```

The bottom row of this table is, not surprisingly, the same as that of the first part of the previous table, only shifted one place to the left. Thus if $rs(n)$ is the run length for a pack of n cards without final half-deck exchange, and $rss(n)$ is the run

length with exchange, then $rss(n)=rs(n+2)$. Thus overall run-length patterns to restore do not change as a result of the final half-deck exchange.

Apart from the powers of two for which $rs(n)$ is simply $\log_2 n$, the pattern of $rs(n)$ is an irregular one with some abrupt changes. In general numbers with low values in the rs table have low values in the rss one. However, this is not always the case, and in particular $rs(54) = rss(52) = 52$, that is the effect of the seemingly innocent final switch of the two half-decks is that for a 52-card deck, 52 rather than 8 runs are needed to restore the original state.

Another way in which the problem of order-restoring shuffle run lengths can be addressed is to observe the position of, say, the number 1 in successive ripple shuffles in the table of shuffles above (remember to read the table upwards!). Its successive positions are 1, 2, 4, 8, then it wraps around to position 7 which = 16 in modulo 9 arithmetic. The next position is 5 which = 32 in modulo 9 arithmetic and so on. Then look at the number originally in position 2. After one shuffle it moves to position 4, then to 8, 7 and 5, the last two being equivalent to 16 and 32 in modulo 9 arithmetic. Thus the number 2 will be restored in the same number of shuffles as it takes to restore 1, and similarly for all the other numbers.

The problem of counting the number of shuffles needed to restore the original order is equivalent to that of obtaining a second 1 in the sequence $2^i \cdot 10$ in modulo 9 arithmetic, thus

```
9|2^i.13
1 2 4 8 7 5 1 2 4 8 7 5 1
```

shows the 6-cycle which has already been observed with 10 cards. Since the run-length to restore cannot exceed the number of cards the number of shuffles to restore in general comes from observing 1s in $(n-1) | 2^i \cdot (n+1)$, leading to

```
(<:|2&^@>:@i.)10
2 4 8 7 5 1 2 4 8 7
```

following which the number of shuffles for various cases is given by

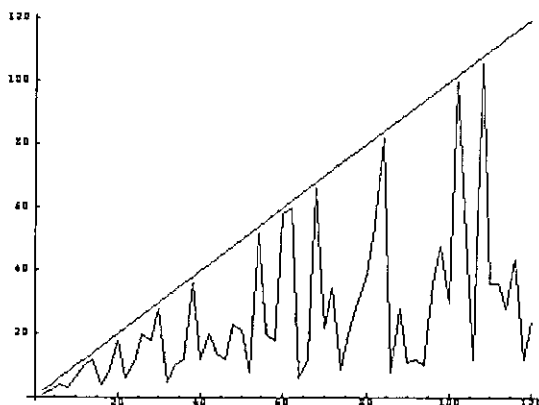
```
count=.:>:@i.&1@(<:|2&^@>:@i.)
count 10
6
w,:>count EACH w=.12+2*i.15
12 14 16 18 20 22 24 26 28 30 32 34 36 38 40
10 12 4 8 18 6 11 20 18 28 5 10 12 36 12
w,:>count EACH w=.44+4*i.15
44 48 52 56 60 64 68 72 76 80 84 88 92 96 100
14 23 8 20 61 6 69 35 20 39 85 28 12 36 30
```

Sharp-eyed readers may have noticed a few values such as that for 60 which are inconsistent with the previous table. This is because 2^{60} is rather a large number, which requires extended precision for exact integer comparison. Thus in the event of discrepancy it is `count rs` which delivers the correct values.

To deal with the variation in which the two half-decks are exchanged at the end of the shuffle, simply replace the `<:` in the middle with `>:`:

```
counts=.:>:@i.&1@(>:|2&^@>:@i.)
counts 10
10
```

But what does the function `rs` look like? Here is a graph of `rs(n)` from $n=2$ to 120:



As noted earlier, the powers of two are easy to deal with – they simply form the series $\log_2 n$. But why do 22, 52, 74 and 92, for example have short cycles, whereas 20, 30, 50, 60 have long ones? And why are there regions of relatively short run-lengths such as

```
w, :>count rs EACH i. EACH w=.82+2*i.10
82 84 86 88 90 92 94 96 98 100
54 82 8 28 11 12 10 36 48 30
```

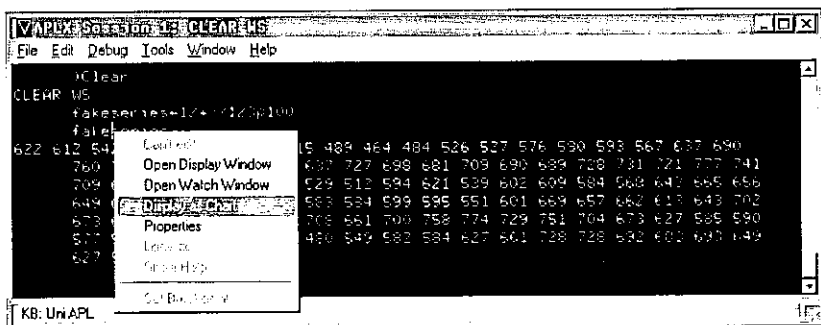
I simply don't know. Perhaps numbers have hidden personality streaks that I never realised, in which case the phrase "well, he's a good sort" takes on a whole new meaning!

Built-in Charting in APLX Version 3

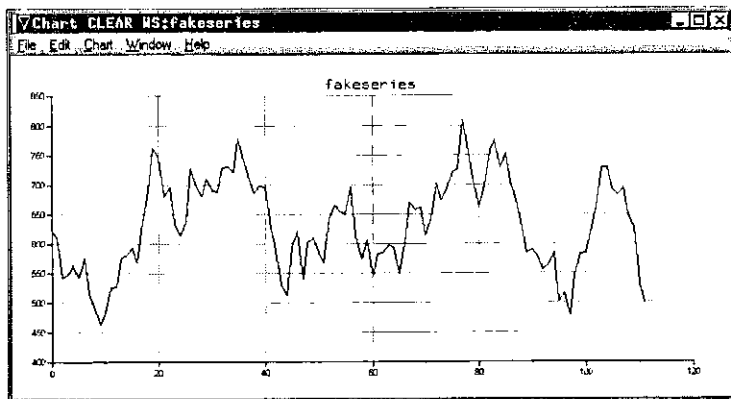
first impressions by Adrian Smith (adrian@apl385.com)

Just Show the Data!

APL is often described as a super-powerful pocket calculator. Great for fooling with numbers, but missing one key ingredient - visualising the numbers once you made them. Only A+ (in the Unix build) and J come with this charting capability as a native part of the interpreter - until today that is. Now MicroAPL have added `CHART` as a system function, and "Display as Chart" to the right-mouse menu which shows up on any (numeric) array.

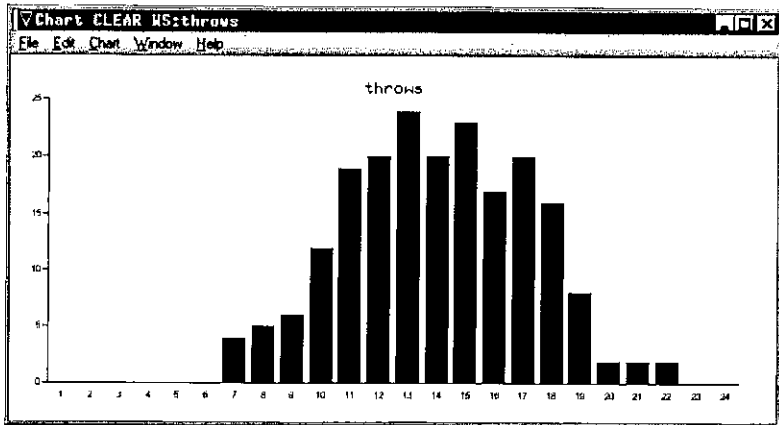


The nice thing about this is that it requires exactly zero programming effort to get a perfectly adequate visualisation of the result. Let go of the mouse, and up pops a new window ...



If all you want is some test data for a sales-forecasting system, then away you go. Seriously, this is very well done indeed - the scales are nicely auto-ranged and the tick-mark intervals take good default values. Basically, this just works 'out of the box' so let's see what else it can do! How about some simple statistics ...

```
fourdice←?200 4p6  
throws←+/ (124)°. ,=+/fourdice
```



Throw 4 dice 200 times for a nice Normal curve!

Did someone mention the possibility of using APL in schools? Maybe this is just the missing ingredient that we need. It is certainly a major selling point for J, and puts APL back on a level field with Excel and all those fancy Casio calculators that you can't take into exams these days.

Quibbles? Well the second thing I tried (as you do) was to regenerate my fake series with some new numbers with the chart window still open. I know that in A+ the chart always watches the variable automatically (Sasha showed some splendid animations last time he talked to the FinnAPL meeting) and I rather expected this to work in the same way. But it doesn't - so please can we have a 'Refresh' option so I can fool in the session and see the results without having to open lots of new windows? Maybe a few more options (for example to force the Y-scale through zero) could creep into the drop-down menu on the chart window? I realise that there is a lot more power available in the `⎕CHART` system function, but it would be good if just a little more of this was accessible here.

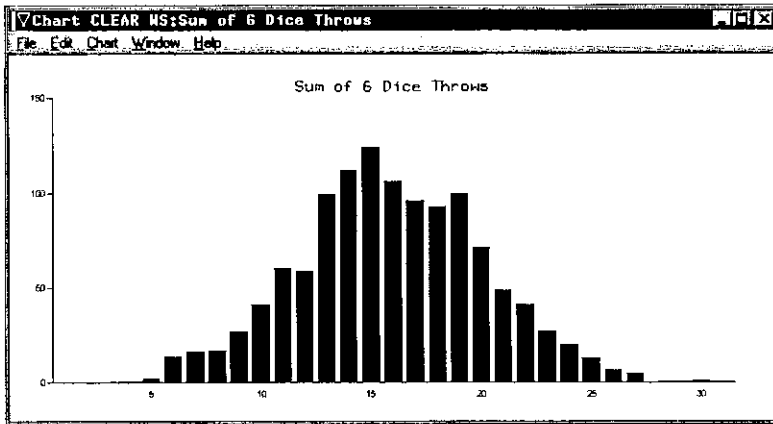
To summarise - if I had had this capability in any of the mainstream APLs I have worked in over the years, I would have definitely done my job better. It would

have saved time and eyestrain looking for outliers in sales forecasts (at Rowntrees in my mainframe era), and given me just the tool I needed to tune Causeway in the days when we Dyalog programmers still cared about speed.

Deeper into `CHART`

The right-mouse capability is great, but (obviously) severely limited in what it can do. If you need to use charts in a real application then there is a much more comprehensive interface available to you. For starters:

```
sixdice+=?1200 6p6
throws+=(5+36)*./sixdice
'type=bar' 'title=Sum of 6 Dice Throws' CHART throws
```

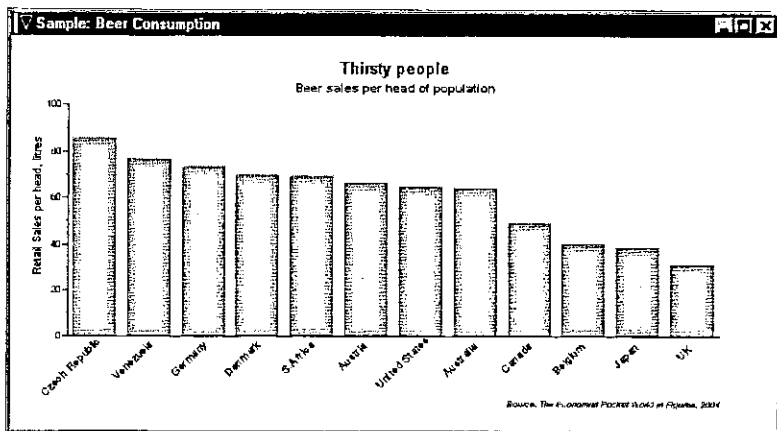


Throw 6 dice 1200 times for a very nice Normal curve!

Essentially, you can drive any of the menu options with a simple keyword interface, provided as the left argument to `CHART`. This is a nice way of short-circuiting the pull-down menus (and adding a few extra tweaks like titles and legends) but for real power I am afraid you have to open up the help file and explore the properties and methods of the chart object. Here is one of the examples from the supplied demo workspace:

```
Chart_Beer;labels
A Sample bar chart
'Beer' OWI 'Create' 'Window'('title' 'Sample: Beer Consumption')('visible' 3)
'Beer.Chart' OWI 'New' 'Chart'('align' -1)('type' 'bar')
'Beer.Chart' OWI 'title' 'Thirsty people'
'Beer.Chart' OWI 'subtitle' 'Beer sales per head of population'
```

```
'Beer.Chart' [WI 'note' 'Source: The Economist Pocket World in Figures, 2004'
'Beer.Chart' [WI 'ylabel' 'Retail Sales per head, litres'
'Beer.Chart.s1' [WI 'new' 'series' ('color' 374479)
'Beer.Chart.s1' [WI 'values' 85.8 76.7 73.9 69.9 69.7 66.6 64.7 64.5
49 40.2 38.6 31.7
labels='Czech Republic' 'Venezuela' 'Germany' 'Denmark' 'S.Africa'
labels=labels, 'Austria' 'United States' 'Australia' 'Canada' 'Belgium'
'Japan' 'UK'
'Beer.Chart' [WI 'xlabels' labels
```



Beer Sales with Custom Colours and Text

So far, so good. Now for a few little challenges. Can I change the Y-axis range to run from 0 to 150, for example? A quick dig in the help file gives me a very APL syntax for this – rather than setting the Y-range (and having APL figure out the tickmarks) I simply provide a vector of tick-mark positions directly:

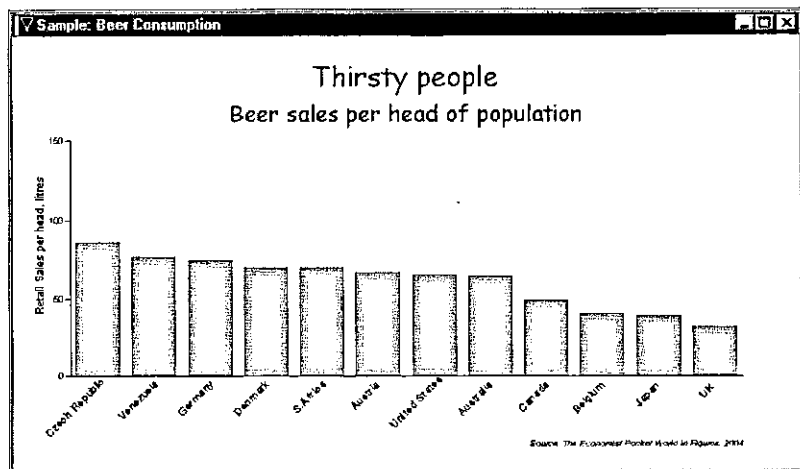
```
'Beer.Chart' [WI 'ymajorticks' (0 50 100 150)
```

... and let's have a less ugly title font while we are about it:

```
'Beer.Chart' [WI 'fonttitle' 'Comic Sans MS' 0.1
```

The font size is given as a fraction of the window width, so that it resizes nicely as the chart is scaled. This time things do change 'as you type' so it is easy to play around in the session until the chart looks OK, then copy and paste the statements into a function to preserve it for posterity.

Here is the output from the amended code:



Beer Sales with Y-tickmarks and Heading Font

Styles can be set with a collection of flags, for example to add some gridlines:

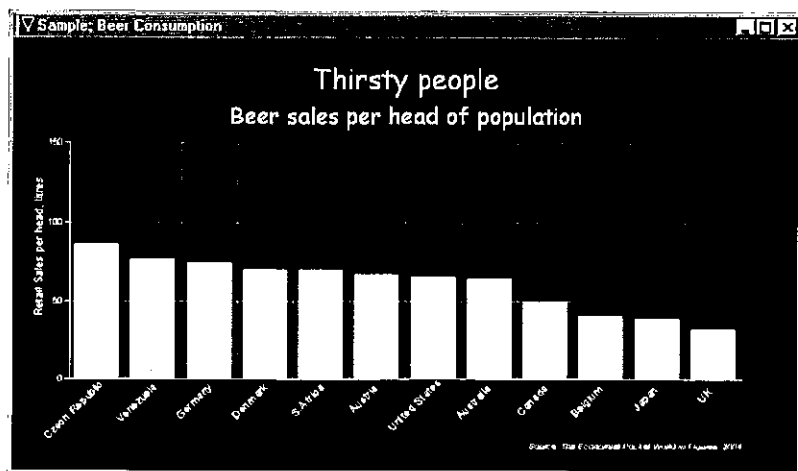
```
'Beer.Chart' [WI 'style' (2+8)
```

Colours can be set for many of the individual chart elements, such as the background and gridlines:

```
'Beer.Chart' [WI 'color'(255 255 128 0 0 64)
'Beer.Chart' [WI 'colorgrid'(128 128 128)
```

This gives us a very nice dark blue background, with yellow text (and axes) but with the gridlines specifically set to mid-grey.

Best to look at this one in colour on the Vector website, but here is a poor monochrome reproduction:



Beer Sales with a more restful colour scheme

I think there is enough here to satisfy most reasonable graphics needs, and the functionality is a good compromise between power and usability. It is also (of course) very fast and copes with arrays of up to a million points with no trouble at all. If you really run out of options, MicroAPL have thoughtfully provided the ultimate 'power user' option of setting the Draw method scale to 7 (which uses the chart axes) and simply adding arbitrary artwork to any existing chart object.

Summary

I think that this is a really worthwhile addition to the APL environment. If APL*PLUS/PC had offered something similar back in 1989, I would probably never have gotten around to writing *RainPro*. Of course there are plenty of things *RainPro* can do that `CHART` probably never will, but there is also a much longer learning curve to ascend before you really feel that you have the software completely under your control. I think I would like a few more styles (e.g. for legend positioning) and I could use a few extras like custom dash-repeats, but these can come along later without wrecking the basic simplicity of the interface.

Dyalog and APL2000 and APL2 please copy!

How to Write Computer Programs

by John Scholes, Dyalog Ltd (john@dyalog.com)

Apology

I apologise to those of you who already know how to write computer programs; please skip to the next article. Anyone who has any doubts might find something of interest in what follows. First, indulge me by listening to a parable:

A Parable

Identical twins Isambard and Deuteronomus (Izzy and Dewy to their friends – see note [1]) were separated at birth and both grew up to become APL programmers. By a strange twist of fate, for the last year or so, they have independently been experimenting with D, a functional subset of Dyalog APL; because they have heard that it is “cool”.

On the face of it, the code they produce is very similar, but closer examination of *how* they produce it, reveals some profound differences.

Dewy frowns and mutters while he codes, whereas Izzy seems to fizz slightly while maintaining a Zen-like tranquillity.

When faced with an unfamiliar and complex APL expression, Dewy tackles it from the right, while Izzy tends to start from the left. Sometimes Dewy even *writes* APL expressions right to left by backing up and prefixing extra bits.

Dewy uses *temporal* words when thinking about programming: “*First* do this, *then* do that ...”. Izzy doesn’t.

Dewy says: “*N gets one*”, while Izzy says: “*N is one*” and Dewy refers to *N* as a *variable*, while Izzy calls it a *definition*. Dewy sees assignment as working right-to-left: a value is *put* into a named pigeonhole, whereas Izzy sees a name *indicating* an existing value to its right (see note [7]). Izzy sometimes wonders why APL uses a left, rather than a right-pointing arrow for assignment.

Dewy sees the statement: $i \leftarrow 3 \quad j \leftarrow 3$ as assigning two *instances* of the number 3 to variables *i* and *j*, whereas Izzy considers that both names indicate the same “Platonic constant in the sky” (see note [8]). The same goes for assignments such as $sum \leftarrow sum + 1$ and even: $gcd \leftarrow \{ \omega = 0 : \alpha \quad \omega \nabla \omega | \alpha \}$.

Dewy comments code with injunctions: "A Initialise result", while Izzy tends to use descriptive noun phrases: "A Mean value".

Dewy calls the first four primitive functions "plus, minus, times and divide", whereas Izzy says "sum, difference, product, and quotient".

Dewy wonders how to simulate for and while loops in D, while it never occurs to Izzy to think about this. (When control structures were introduced, their great uncle Gottlieb (Go-ey) used to wonder how to simulate branch arrows. He still maintains that control structures are thinly disguised branches and when, for example, he looks at an :If :EndIf structure, he sees through it to the underlying go-to).

Dewy finds the flowchart an indispensable tool of thought; Izzy doesn't.

In general, Dewy finds programming a difficult (though rewarding) experience, while Izzy wonders what all the fuss is about.

Dewy finds that, more often than not, his programs run a little quicker than those of Izzy, but that they are buggier, so he spends more time fixing them.

Although raised in separate homes, their upbringing was remarkably similar, except for a single life-changing event. One day, at a formative age, while Izzy was exploring the attic, he found a piece of framed embroidery, which proclaimed:

Describe the result
in terms of the arguments,
using only the present tense
of the verb "to be".

Picked out in smaller stitches around the border, was the additional rubric:

You may use
conditional if clauses and
supplementary **where**
clauses
to define terms.

And

Only when you have a complete description,
should you transliterate it into code.

End of Parable

(The characters in this story are fictional. Any resemblance to person or persons living or deceased is purely coincidental.)

So what?

D, a subset of Dyalog APL is a modest attempt to provide a back end for is-y programming. It is hard to **do** anything in D, but easy to define what anything is.

A surprisingly large number of problems are amenable to is-y programming, as is a surprisingly large proportion of the components of an otherwise **do-y** program.

Perhaps also surprising: is-y programs have no need of:

- Loops
- Partial assignment]← .. or)←..

- Modified assignment: $I \leftarrow I + ..$ (or $I \leftarrow I + ..$)
- Variables (which change, as opposed to definitions, which do not).

Is-y programs have no “moving parts” in the same sense that this program for the mean item of a vector has no moving parts:

```
mean ← { (+ / ω) ÷ ρ ω }
```

“The **mean** is the quotient of the **sum** and the **number** of items.”

Eh?

Do-y and Is-y programming are also known as *procedural* and *declarative* programming respectively. Declarative languages and in particular *functional* languages are generally accepted to be easier to program and produce more stable code, than procedural ones.

But you have to treat them right.

Some of us are old enough to remember the disappointment of people who translated Basic programs into APL on the understanding that something magic would occur to make them easier to understand and go a lot faster. Not so. To reap APL's benefits, you have to think arrays *before*, rather than *after* designing your code.

An Example

Many years ago, before there were control structures (and by the look of it, lower-case function names), I wrote a program to generate random maze puzzles:

```
MAZE 5 10
+ +---+---+---+---+---+
| | | | | | | | | |
+ +---+---+ +---+ +---+ +
| | | | | | | | | |
+ + + + + +---+---+---+
| | | | | | | | | |
+ +---+ + + + +---+---+ +
| | | | | | | | | |
+---+ +---+ +---+---+ +---+ +
| | | | | | | | | |
+---+---+---+---+---+---+
```

As a Christmas gift to the dfns@dyalog.com email group, I translated MAZE into D. This was not a trivial task, as the original code had a fairly spaghetti-like

branching structure. Only after I had finished the translation, did it occur to me to wonder what the program would look like had I (like Izzy) *thought about the problem* in a declarative way, in the first place.

Let's contrast the two approaches to this problem:

MAZE: procedural approach

- Initialise an w -sized grid of cells.
- Starting from opposite corners of the grid, alternately extend a random path by breaking down cell walls, until the paths collide. The resulting path, corner-to-corner, is the solution path of the maze.
- Starting from each unvisited cell, extend a random path until it collides with a path other than itself.
- When extending a random path, if all cells adjacent to the active (foremost) cell belong to the path being extended, choose a cell at random from those in the path and continue extending from this point.

MAZE: declarative approach

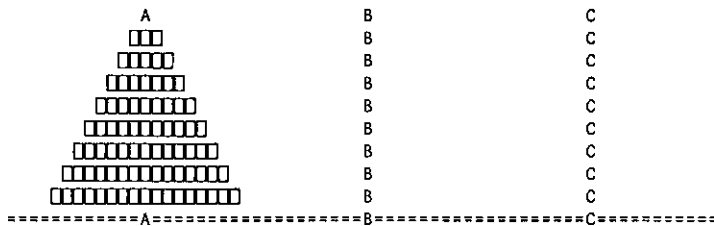
An w -maze is the fill of a solution for an initial w -grid. Where:

- An w -grid is an w -sized grid of cells.
- A solution through a grid is
 - If the half-paths collide, then the grid.
 - Otherwise, the solution of the extension of each half-path.
- The fill of a solution is:
 - If there are no unvisited cells, then the solution.
 - Otherwise, the fill of the extension of a path that is an unvisited cell.
- The extension of a path is:
 - If the active cell has an adjacent cell that is not within the path, then the path extended with a random selection from such cells.
 - Otherwise, the random extension of the path with a randomly chosen active cell.

Another Example

The **Tower of Hanoi** problem may be stated and solved in both a procedural and declarative fashion:

Disks of decreasing sizes are placed on one of three spikes A, B and C, as shown:



Procedural statement and solution of the problem

How *do* you move all of the disks, one at a time, from spike A to spike B, while avoiding placing a larger disk above a smaller one?

If *n* is greater than 0:

Move *n*-1 disks from spike *i* to spare spike *k*, then

move disk *n* from *i* to *j*, then

move *n*-1 disks from spare spike *k* to spike *j*.

```

[0]  {ftv}hanp n;fm;to;via
[1]  :If n>0
[2]      :If 0=NC'ftv' ◊ ftv◊'ABC' ◊ :End
[3]      fm to via◊ftv
[4]      (fm via to)hanp n-1
[5]      [◊←(¿n),':',fm,'→',to
[6]      (via to fm)hanp n-1
[7]  :End
  
```

Declarative statement and solution of the problem

What is the sequence of moves that transfers disks, one at a time from spike A to spike B, while avoiding placing a larger disk above a smaller one?

If *n* is 0, then the null sequence.

Otherwise, the concatenation of:

The sequence for moving $n-1$ disks from spike i to spare spike k ,

The move that transfers disk n from spike i to spike j , and

The sequence for moving $n-1$ disks from spare spike k to spike j .

```
hand←{
  ω=0:0 0ρ''
  α←'ABC' ◊ fm to via←α
  seq1←↑(fm via to)∇ ω-1
  seq2←↑(↖ω),': ',fm,'→',to
  seq3←↑(via to fm)∇ ω-1
  tseq1,seq2,seq3
}
```

The Mathematicians' Con Trick

Simple declarative statements, such as the solutions above, sound too good to be true. They remind us of an elegant mathematical proof that was a long time in the finding and then stated very simply. Of course the mathematician who discovered the proof didn't do so in its final neat form; it would have been refined over time from an initially much messier version. So it is with the statement of the result in terms of its arguments. We can work with this statement, refining it over and over until we are satisfied that it is robust, simple and sufficiently general for our needs. At various points, we may commit this statement to code for testing purposes. When we find problems that cause us to amend the code, we should first correct the description accordingly, as the statement will form a powerful piece of system documentation for our program, when we later forget how it works.

Full Circle

When we become proficient in describing the result in terms of the argument, using only the present tense of the verb *to be*, we could experiment with writing the description in a different language (French, Latin, Esperanto, ...) before transliterating it into code. At some point, we might even find that a declarative *programming* language (such as appropriately commented D) is suitable for our needs.

What's the Catch?

One significant downside of *is-y* programs is that in general they run slower than corresponding *do-y* programs. For example, an indexed assignment is

significantly quicker than a function to mesh together the parts of two arrays to form a third. However, because APL interpreter mongers tend to keep an eye on the way we code, the more we use such language constructs, the faster they tend to become.

Notes

- [1] Dewy pronounces his name Do-y, rather than Due-y. American readers, for whom *do* and *due* sound the same, will have the same take on this statement as with "you say tomato and I say tomato".
- [2] On a personal note: I was brought up firmly in the do-y school of programming. I find it extremely hard to change and when the going gets tough, I slip back into the old ways. It may be too late for me, but younger minds may be able to make the switch.
- [3] The functional programming community has spent the last couple of decades inventing languages in which *all* programming is conducted in the is-y style. As an example, they even propose implementing operating systems this way. APL is a forerunner of the functional programming movement and many of its exponents started life as APLers.
- [4] You might review the above parable as a sort of "teen magazine personality test" to see how is-y or do-y you are. Invent your own scoring system.
- [5] Exercise: Old habits die hard. You will not believe the advantages of is-y programming unless you take the time to try it one time "for real". Rewrite your favourite function from scratch using is-y techniques.
- [6] See Alan Perlis' Aphorisms: <http://www.cs.yale.edu/homes/perlis-alan/quotes.htm>.
- [7] At a baby-naming ceremony, such as a christening, we (with the possible exception of some folk in the pop music industry) usually think of assigning the name to the baby, rather than the baby to the name.
- [8] Another school of thought holds that this stuff is ref-counted. The number 3 will remain in existence as long as someone, somewhere in the world, bears it in mind. This is why it is important to teach our children to count.
- [9] "Yes, but computers *do* things don't they?" Well yes, but a lot of what they do is maths, and maths doesn't *do* anything, it just *is*.

Appendix 1 - A note on the conditional construct

Although they often use the same keyword: **if**, conditional execution differs between procedural and declarative languages.

Both constructs tend to have three working parts: the **test**; the **true** part; and the **false** part. For example:

```
if    test
then true
else false
fi
```

In both cases, **test** is an expression that evaluates to a Boolean scalar but the similarity stops there:

In a procedural language: both **true** and **false**, together with the construct as a whole are non-result-returning (void) *do-y statements*; in a declarative language, all three are result-returning *expressions* and in a strongly typed language, all three must be of the same type.

In natural language terms, the procedural **true** and **false** are *clauses*: "If it's raining, put on a mackintosh, otherwise (else) wear a jacket"; in a declarative language, they are *noun phrases*: "I would like: if you have milk, then tea, otherwise coffee".

In a procedural language, the **else false** part of the construct may be omitted: "if it's raining, put on a mackintosh"; in a declarative language such an omission would render the sentence incomplete: "I would like: if you have milk, then tea" ("Else what?").

The C language has distinct procedural and declarative conditionals:

```
if (test) true; else false; // procedural.

(test ? true : false)      // declarative.
```

Classical APL may use indexing or pick as a declarative conditional, although as the language is *strict*, both sides of the condition are evaluated and then one is discarded:

```
(⊖io+test)>false true
```

In D, the declarative conditional is implemented by the guard:

```
test:true ◊ false
```

(*) Strictly speaking, a declarative conditional could omit the `else` clause if the type of the conditional were `(void)`. However, the market for void-returning clauses is fairly slim: given that there are no side effects, it is hard to imagine the purpose of such a clause.

Appendix 2: A short note on the Declarative Programming Style

Declarative Programming, of which Functional Programming is a subset, of which *pure D* is a sub-subset, differs from the more traditional Procedural Programming style in that, in the latter we are obliged to provide a procedure for the construction of a result, whereas in the former, we merely declare what the result is to be. APL was one of the first languages to show that the declarative style is feasible and was cited as such in John Backus' seminal paper [Backus 1978].

An example of the declarative style is the APL "program" for the arithmetic mean of a numeric vector:

```
mean←{          A The arithmetic mean of a vector IS the:
      (+/w)÷p w  A quotient of the sum and number of its items.
}
```

Notice that the function comments form a single sentence of the form: subject, copula, predicate. The "body" of the function is a noun phrase. Contrast this with the comments for a primitive procedural language:

```

v z←mean vec;i;tot  A The mean of a vector IS
[1] tot←0           A FIRST, ZERO total, THEN
[2] i←pvec          A SET index to number of items, THEN
[3] loop:→i+done    A BREAK loop if finished, otherwise
[4] tot←tot+i+vec     A INCREMENT total by next item THEN
[5] i←i-1            A DECREMENT index and THEN
[6] →loop            A LOOP back for next item.
[7] done:z←tot÷pvec  A The result IS the quotient of ...
v
```

Notice that the way we read procedural programs, tends to be rife with temporal words: *first* do this, *then* do that; *repeat* so-and-so *until* such-and-such; *variable* what's-its-name *becomes* something-or-other; ...

The temporal sense of the code is further reinforced by the use of 'destructive' verbs such as *zero*, *set*, *increment* and *decrement*. You might think this a little unfair

and that we could have commented the first couple of lines with: “Total is 0; index is the number of items ...”, but we would then be forced to contradict this, for example in line[5] by trying to say: “index is one less than index”, which is clearly nonsense.

Declarative programs, on the other hand, have no sense of the passage of time; everything just *is*.

Not that any of this is in itself a bad thing, but such words are indicative of the program’s having *state*. The beauty of a program without state is that, as there are no “moving parts”, little can go wrong.

A similar point is made about the background of the little Min language (see www.dyalog.com/dfnsdws/min_index.htm). With its roots in Church’s Lambda Calculus [Church 1941], Min uses the terms *successor* and *predecessor* in preference to: *increment* and *decrement*. Min’s primitive function $+$ is equivalent to D’s $\{1+\omega\}$:

“Incidentally, using the terms *successor* and *predecessor* as opposed to *increment* and *decrement* stems from having a *denotational* rather than *operational* view of a function. In other words, a function is seen as denoting the mapping between the sets that comprise its domain and range, rather than as a prescription for the operation that converts one into the other. To take a specific example: Min’s $+$ doesn’t increment its argument; it leaves it alone and denotes its successor as result. Given this view, words from the subset of nouns that can be used as determiners (first of, reverse of, ...), seem more appropriate than transitive verbs as names for functions. Specifically, *sum*, *difference*, *product* and *quotient* would appear preferable to: *add*, *subtract*, *multiply* and *divide*”.

It is important not to misunderstand all this emphasis on language. Our choice of words in describing a process is often indicative or symptomatic, of our internal mental model. However, being careful to use the “right” words, does not of itself, improve the model: curbing the symptoms doesn’t cure the disease, although there is some evidence that rationalised behaviour is gradually incorporated. Avoiding sexist, racist, ageist ... language does not automatically qualify me as being free of that particular -ism, but it’s a good start.

A first step might be to read APL expressions, and write APL expressions that can be read, from left to right. Again taking $\text{mean} \leftarrow \{ (+/\omega) \div \rho \omega \}$ as an example: the declarative left-to-right reading of the code is in the end simpler than the procedural right-to-left reading: “Take the shape of the vector and divide it into the sum of the items”.

Notice that the declarative version of `mean` isn't compromised by splitting it up a little with local definitions (strictly speaking, not "local variables" as nothing varies).

```
mean←{      A The arithmetic mean of a vector IS:
  tot←+/ω   A (Let <tot> BE the sum of the items and
  num←ρω    A      <num> BE the number of items) in
  sum÷num    A the quotient of the two".
}
```

A reasonable definition of a **D** function in this context is "an expression of α and ω preceded by 0 or more local definitions". Given this, it is sometimes helpful to read the code backwards, bottom to top: "The arithmetic mean of a vector *is* the quotient of *sum* and *num*, where *num* *is* the number and *sum* *is* the sum of the items". In fact, because there are no side-effects in such a definition, it doesn't really matter in which order the expressions are evaluated, so long as all referenced values are available at the time of the evaluation. In the early 1970s there were a number of rather sensationalist statements in the computer press, along the lines of: "Researchers are designing programming languages in which you can shuffle the source card deck".

References:

Backus, J., 1978. "Can Programming Be Liberated from the Von Neumann Style? A functional Style and Its Algebra of Programs". *Comms of the ACM* 21, No. 8, pp 613-641 (1978).

Church, A., 1941. *The Calculi of Lambda Conversion*. Princeton University Press.

http://www.dyalog.com/dfnsdws/min_index.htm

Acknowledgements

Thanks are due to:

John Daintree for pointing out that declarative (is-y) `ifs` must have an `else`.

Stephen Taylor for suggesting that APL might have chosen the *right arrow* for assignment.

Maria Wells for overhearing note [9].

Revisions

This note was first published at the Dyalog APL conference in Naples Beach, Florida in November 2004 and revised in April 2005.

Industrial FP: A Case Study

by Stephen Taylor (stj@5jt.com)

*Jars of spring water are no longer enough;
take us down to the river.*

— Rumi

APL and Functional Programming are kissing cousins. Long before FP was elaborated, APL was focused on the transformation of arrays of data by primitive, defined and derived functions. In turn, seminal work on Functional Programming influenced the development of D in Dyalog APL and tacit programming in J.

What does this mean to a working programmer? Substantial software has been written in pure FP form, including compilers. But does FP remain an academic fad, a cunning trick — look, Ma, no variables? What can we take and use from FP, and how might it help us in everyday programming?

Elsewhere in this issue (“How To Write A Computer Program”) John Scholes offers his view of the nature and value of FP. In this article I will take a single substantial example drawn from my own programming practice, explain how it works and how it was developed. I’ll also review what differences doing it in FP made, and what debugging and maintaining it have been like.

This example has been in production use at a large UK pension provider for the last year. It is fast, reliable and ran about 10,000 times in the last three months.

The problem

As part of our development style with this customer (I would blush to use so elevated a word as ‘methodology’) we minimise our use of their IT people. We are often reminded how scarce their time is. So when we were offered an interface to mainframe data records and asked in what format we would like the data arranged, we said: Anything, really. We didn’t want to add to their already heavy burdens. Just do it, we said, in any way that suits you, and tell us afterwards what you did. We’ll figure out how to read it. We just want you to do it whichever way is easiest for you. They looked at us as if we were mad.

Perhaps we were. When we got the file formats, tables were nested inside tables within tables within... Perhaps this is a common layout for mainframe files. Would I know? Anyway the data had to be unpacked from this, and in the future

from files of similar structure. I did not want to revisit this problem; I wanted a general approach.

Brief thought distinguished the syntax from the semantics of the file. The files alternate definition lines with one or several data lines. (See Exhibit1 or download the query file (100Kb) from www.vector.org.uk/archive/v213/query.txt). The definition lines name and describe the fields represented in the data lines that follow. The semantics of the file involve type conversions, discarding padding and so on. The syntax was a matter of picking the data fields out.

The first lines in the file are encouragingly easy. For example:

```
D011 #PAIDTODATE(N5),1 #ISSUEDAY(N8),1 #STATUSCODE(A2) ...
A01 1248          1220 1180 456 20 1248 1248GPSN ZS488133CYNN ...
```

This is knockabout stuff. Definition line 1 (D01) specifies PAIDTODATE(N5) — a numeric field requiring 5 characters. Well, 6, as it turns out that N fields always get an extra character width. Still, they are consistent, no problem. It's followed by 9 characters of ISSUEDAY and 2 (really 2) for an alphabetic STATUSCODE. This gets the first three fields of the line: '1248' '1' '22'. If only it were all that easy.

The second definition line introduces a complication.

```
D021 RET-IDAS-INFO(1:700),2 #COVCONNECT(A2)
A020707080819
```

RET-IDAS-INFO is not a field, but announces a 700-row table. The following fields all contain 700 values, alpha-fields padded with blanks, and numeric fields padded with zeros. So we get the COVCONNECT column as '07' '07' '08' '08' '19' and the blanks are presumably 695 unused rows.

We also notice the numeric prefix to the field name. In the first example, the fields were all prefixed with a 1, locating them at the root of a 'nesting tree'. The columns of the RET-IDAS-INFO table are all prefixed with 2s, indicating another level of nesting.

The RET-IDAS-INFO table gives another couple of blank-padded alpha columns, then hits its first numeric column.

```

D052 #DETAILEFFDATEMYC(N5)
A05 1180 1204 1180 1204 1237 0 0 0 0...
A06 0 0 0 0 0 0 0 0 0 0...
A07 0 0 0 0 0 0 0 0 0 0...
A08 0 0 0 0 0 0 0 0 0 0...
A09 0 0 0 0 0 0 0 0 0 0...
A10 0 0 0 0 0 0 0 0 0 0...
A11 0 0 0 0 0 0 0 0 0 0...

```

Once we realise the zeros are just the padding on the numeric column we can see there is no difficulty here. The table continues with further columns, then ends with a new kind of field declaration.

```

D261 #ERRORMESSAGE(A30/1:10)
A26

```

We know the table has ended because `ERRORMESSAGE` is prefixed with a 1, putting it back in the root of the nesting tree. If it had a 2 prefix, it would be part of the `RET-IDAS-INFO` table. The `A30` speaks for itself, but `1:10` looks like a table declaration. Almost: we could say this a list of ten 30-character strings. (We're pleased to see the error message is blank.)

After the error message we plunge straight into a new table, `SVDATA`.

```

D011 SVDATA(1:2),2 #CASHVAL-1(N9.2),2 #SURRVAL-1(N9.2),2 #SURRCHG-1(N7.2)+
B01 46989.35 49766.26 46989.35 49766.26 0.00+

```

(We notice the data lines are now prefixed with B; it turns out we're into part B of the query. And the definition line numbers have restarted at 1 again.) Lines D01 and B01 describe the left end of the 2-row table `SVDATA`:

CASHVAL-1	SURRVAL-1	SURRCHG-1
46989.35	46989.35	0.00
46989.35	46989.35	0.00

But just a little way further along the same definition line, things start to get more interesting.

```

2 #REASONCOUNT-1(N2),2 RETLPIENTRY-1(1:3),3 #LPISURRVAL-1(N7.2)

```

While still inside table `SVDATA` we've declared a new 3-row table `RETLPIENTRY`. Now further 'columns' of `SVDATA` are no longer 2-lists, but 2x3 tables. To mark this, the contents of `RETLPIENTRY` are prefixed with a 3. The first column of `RETLPIENTRY` is `LPISURRVAL`; this is now nesting level 3.

That's pretty much it. The file has no more surprises. But it makes liberal use of the table trick. Before emerging from RETLPIENTRY it's opened (line D18) the 3-row RETPHLPIENTRY table at level 4, and we can see (definition line D02 in part C) that we reach level 5 at the deepest level of nesting.

If I have to unwrap query files like this, I want a general solution. I want a function that will read the definition lines and work out where the data is in the data lines and break it out and remember what shape and type it's supposed to be. I'll go nuts if I try to work this stuff out myself and I'm blown if I want to repeat the exercise for the next kind of query file we use. (Hence the epigraph from Rumi at the head of this article.)

Parsing the query file

I decided to parse the query file into a 4-list. The first element of the list will be a list of nesting stacks. For example, the field LPISURRVAL is within the 3-row table RETLPIENTRY, which is itself within 2-row table SVDATA. So the nesting stack for LPISURRVAL is 2 3. (Or 1 2 3 if you count the root of the nesting tree.) An APL way to say it: the first element of my result will be a list of the shapes of the fields.

The second element will be a list of the field names, and the third a list of their formats. The fourth and final shall be a list of the data 'windows', that is to say, the text (possibly truncated) from the query file that contains the data for the field.

Since each of the four lists will have an entry for each field, they will be of equal length. So we could equally well describe the result as a 4-column table.

Strategy

The job is an obvious candidate for FP. All the information required is on the query file. Retrieve that as a single character string, and return the required 4-list. The query files run to about 100Kb, which can be easily handled within memory. There are no side-effects required. I set out to write `parseFile` as a D function using FP style.

The first thing to notice about `parseFile` (Exhibit 2) is that it consists entirely of definitions. Three of these define data, which is to say names are assigned to constants. The remaining lines all define functions and operators, except the last line, which defines the result. It is also the only definition that involves the function argument. So we can see that, as `parseFile` is interpreted, not much actually happens until the last line, then a lot happens at once. For debugging this

looks like a disaster, but I'll return to that subject later. At any rate, it makes the strategy of `parseFile` very clear.

Looking at the last line, the `lineate` function is dyadic; `CrLf` is its left argument. Functions `nstate`, and `separate` are monadic, and `nstate` and `separate` both return paired lists, which dyadic function `parse` zips together.

So we can see that the result is defined as something like the following: we `parse` between the two lists returned by `nstate` after separating segment headers and bodies from the segmented file string `lineated` by `CrLf`. Well, it's a start.

Read from right to left: we `lineate` the file string using `CrLf`. That sounds as if we sensibly divide it into lines marked by CR-LF. Then we `segment` it, and a quick glance at the definition of `segment` confirms this involves testing for the initial `D` that we know distinguishes the definition lines. So we can guess `segment` too: it looks as if it groups the lines into segments of a single definition line followed by 1 or more data lines.

Ah, we can then guess what `separate` does. It turns a segment into a pair: a definition string (header) and a data string (body).

The `nstate` function does the tricky thing in the whole setup.

The file lines have been truncated, so the segments have to be parsed individually, lest we fail to add enough padding between segments. We have to calculate the data windows for each segment, overtaking to pad with blanks, before starting the next segment. So we need to get all that worked out while preserving the segment structure.

But there is not enough information in each segment to calculate its data window. (That is, how many characters the whole segment should occupy.) We need the whole nesting structure to calculate the windows of the individual fields. The `nstacks` function takes a list of segment headers and returns the nesting stacks for each field, re-nested into segments. (The `onfl` at operator that does this came from a casual remark by Ray Cannon that defined operators are handy anytime you want to transform something, apply a function, and then reverse the transformation.)

The `nstacks` function works out the repetition (nesting) stacks for each field. That means running forward through all the definitions, working out the cumulative effect of the various table declarations and ends of tables. Back to this later. For now, `nstate` does this, and infixes the nesting stacks with the field definitions.

After that it's all downhill. We've got the field names and type/widths and by now, the nesting stacks which tell us how many times to replicate the width to get the 'data window' for each field. Simple: multiply the field width by the product of its nesting stack. You can see this happening at the right end of the `parse` function.

```
parse←{r f←α ◊ (c1↑"r),ns,cω fretby×/"r,"2>ns←nmshp f}"
```

We're nearly home and dry. We came out of `nstate` with the nesting stacks infix in the field definitions. All that remains in `parse` is to derive the data window for each field and use it to fret the segment's data string. (You don't have to write `J` to learn from it.)

Broadly, `parseFile` has three roughly equal parts. The first part covers transforming the file string first into lines, then segments, then separating the segments into pairs of definition and data strings. Nothing very hard there. The second part (`nstate`) does the hard work of calculating the nesting stacks. The third part (`parse`) uses the field widths and nesting stacks to calculate the data windows, and so fret the data strings.

Tactics

An undeserved pleasure of displaying code like this is leaving the impression that it just flowed off the pen, or in this case the keyboard. Well, some of it did. The first third was fairly obvious in two ways. It was fairly obviously a good way to start on the file string, and it was fairly obvious how to do it.

The last third required more fiddling to convert the various field definitions to widths, but the general line of it was clear from the beginning. I recall trying many ways of passing arguments between the functions before settling on this version.

Half the work went into the middle section, especially if you include lying on the sofa with a wet cloth on my head.

Of course, it's by no means pure FP. Although `parseFile` defines no variables other than a couple of constants, three of the functions it defines do. Arguably their short length limits whatever damage this does.

My `J` is nowhere near good enough to attempt one, but I would expect a `J` version to look broadly similar with perhaps 50-70% of the code volume. `D` is not, I think, quite as compact, but still a fine vehicle for FP.

Against utilities

The function is monolithic and has no external references. There are obvious opportunities to use utilities for familiar functions such as removing trailing blanks, partitioning a string. I used none. Why?

First, I was indulging a love of the monolithic, self-contained function. D functions are such an opportunity to write functions that are simultaneously monolithic and modularised, without importing the rubric of object-oriented code.

One of my principles of coding clarity is to minimise the distance between definition and use of a name. A reference to a term defined in the previous line is clearer than to one ten lines earlier. A reference to a term defined in the same function is clearer than a reference to one defined elsewhere. So I favour local definitions. I might think I know what a utility function does, but its definition is not right in front of my eyes, so there is some uncertainty.

Worse, utilities usually do too much. When I wrote utilities I wrote as if I were extending the set of primitive functions. I designed them to handle boundary conditions predictably and consistently, and never suspend. Ideally they would select the most efficient algorithm for their arguments. But reading later the code that uses them, you can't see which parts of the utility are expected to be useful. Is that boundary condition the utility takes such trouble to handle actually expected to occur in this application?

These days utilities have a more limited place in my work, encapsulating pesky interfaces to other systems and a bunch of stuff I'd prefer not to know about. But for simple transformations, such as lineating a file string, I use the simplest D function I can write. Simplest, so I can read exactly what it's intended to do.

Utilities handle boundary conditions I expect not to encounter. If I'm wrong about the argument I pass it, a utility is likely to do something clever and return an unexpected result to cause trouble further down the line. Better to use a simple D function that breaks in place. Think of this as a fuse, a circuit breaker; code that breaks when given unexpected arguments.

Better off trad?

Did I do a good thing writing this in FP style? The middle third of `parsefile` took a lot longer to write than I care to remember. The whole function represents several days of work. The middle section, in particular, I could only work on when fresh and alert and able to stay 'in the zone'. (I believe pair programming here would have slashed the development time. Ray, where were you?)

Perhaps I would have done better to write a less condensed, tightly-wrought solution? I haven't tried the exercise for comparison, but I don't think so. In the first place, it made a huge difference that I could get all the source code on a single screen. Eyeballs track closer to the speed of thought than fingers do.

Secondly, while developing I stopped `parseFile` on its penultimate line. (You can't set a stop on a D function, but a judiciously placed period will do it.) At that point I had all its local functions and operators defined and could debug the last line in immediate execution. That was pretty much what I did. I got lineate working to my satisfaction, then segment, and so on.

The tiny D functions could all be edited until they did the right thing, then displayed in canonical representation, and the revised definitions pasted back into the suspended `parseFile` function. Nothing could be simpler. Despite the complexity of the whole problem, refining the components of `parseFile` was pleasantly like doing textbook APL.

When `parseFile` worked on my folder of sample query files I took it back to the office to play with live query files. Proudly I showed the source to my colleagues, who paled. Surely this was the infamous 'write-only code'? It turned out not to be so. The live query files twice displayed characteristics I hadn't originally seen or catered for. In each case I was able to identify and amend the relevant fragment of `parseFile` in under twenty minutes.

That was a year ago; never had to touch it since. Some time last year we got the second kind of query file, formatted on the same principles. We had five problems parsing it, all of them arising from errors in the query file format. Once they were corrected, `parseFile` handled everything smoothly.

Refactoring

In fairy tales the good people live happily after. Or something happens after a year and a day.

A year and a day later, while preparing this article, I noticed some unhappy-looking expressions. For example, the original function defined

```
wdws←{×/"ω,"1↑"α}
```

Surely those three eaches might be refactored? Indeed yes, and `wdws` disappeared into `×/"` and a `,`. And then a couple of defined operators looked vulnerable and had to go as I saw patterns I hadn't been able to see a year earlier.

Two things to notice. One: I'm still learning how to write APL.

Second, refactoring was a snack. We have thousands of query files, but only two varieties. I took the file strings from an example of each and assigned them names F1 and F2. Now

```
R←parseFile"F1 F2
```

gave me R as a model answer. Then it was time for substitutions. I opened `parseFile`, duplicated a line, commented out the first copy and edited the second. Exiting the editor,

```
R ≡ parseFile"F1 F2
```

told me whether the new line worked. Changing and testing in small steps like this, I was able to work confidently, never more than one change away from a working version. Code and test, code and test, with a cycle time measured in seconds — XP on amphetamine!

Alternative ending

Fairy tales can have alternative endings too. Reviewing the code for this paper, I was unhappy with a certain amount of shuffling I saw going on, with arrays being packed and unpacked primarily to keep them in the FP pipeline. Refactoring removed much of this, but you can still see some juggling going on in `nstate` and `parse`. I tried another round of refactoring: could I lose the juggling? Alternatively, if I did assign a (presumably minimal) set of intermediate results to variables, how bad would that now look?

You can see the result in Exhibit 3. (I've arranged the lines so that both functions start with all the common lines.) It's a line or two longer. The last line expresses the result more clearly: four lists with the segment structure removed. The penultimate section is, I think, clearer about the tactics of the function, less clear about its strategy. You pays your money...

More soberly: FP seems not the way to write (any more than OO is) but a Very Useful Style worth practising. Working functionally kept the code terse and free from distractions.

Exhibit 1: top left corner of query file

```

S00GU8035950U 04/11/03 10:50:10.6 10.0 12202003
D011 #PAIDTODATE(N5),1 #ISSUEDAY(N8),1 #STATUSCODE(A2)
A01 1248 1220 1180 456 20 1248 1248GPSN Z5488133CYNN
D021 RET-IDAS-INFO(1:700),2 #COVCONNECT(A2)
A020707080819
D032 #REASONCODE(A2)
A03K1K1K1K1J2
D042 #ACTIVITYTYPE(A1)
A0433333
D052 #DETAILFFDATEMYC(N5)
A05 1180 1204 1180 1204 1237 0 0 0 0 0 0
A06 0 0 0 0 0 0 0 0 0 0 0 0
A07 0 0 0 0 0 0 0 0 0 0 0 0
A08 0 0 0 0 0 0 0 0 0 0 0 0
A09 0 0 0 0 0 0 0 0 0 0 0 0
A10 0 0 0 0 0 0 0 0 0 0 0 0
A11 0 0 0 0 0 0 0 0 0 0 0 0
D122 #DETAILFFDATEDAY(N3)
A12 1 5 1 5 27 0 0 0 0 0 0 0 0 0 0 0
A13 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
A14 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
A15 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
A16 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
A17 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
A18 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
D192 #DETAILGROSSPAYMENT(N7.2)
A19 1000.00 2000.00 1000.00 2000.00 12820.00 0.00
A20 0.00 0.00 0.00 0.00 0.00 0.00
A21 0.00 0.00 0.00 0.00 0.00 0.00
A22 0.00 0.00 0.00 0.00 0.00 0.00
A23 0.00 0.00 0.00 0.00 0.00 0.00
A24 0.00 0.00 0.00 0.00 0.00 0.00
A25 0.00 0.00 0.00 0.00 0.00 0.00
D261 #ERRORMESSAGE(A30/1:10)
A26
D011 SVDATA(1:2),2 #CASHVAL-1(N9.2),2 #SURRVAL-1(N9.2)
B01 46989.35 49766.26 46989.35 49766.26 0.00
D024 RETRCLPIENTRY-1(1:3),5 #RCLPISURRVAL-1(N7.2)
B02 0.00 0.00 0.00 0.00 0.00 0.00
D035 #RCLPISVUNITS-1(N7.6)

```

Exhibit 2

```

parseFile={
  A w is a string representing lines of a LIFE70 seam file
  A result is 4 same-length lists describing the fields:
  A (nesting levels)(names)(formats)(data as text)
  to+{(a w)w w} A apply w according to a w
  part+{1+((a)to a,w)} A partition by a
  crlf+TC[2 3] A CONSTANT: CR, LF
  rtb+{w/~/\φw+ ' } A remove trailing blanks
  lineate+{(a)part w~1+a} A distinguish lines by a
  segment+{( 'D'==o(α))to c w} A each segment: (descs)(data)(data)
  defl dat+3 3 A CONSTANTS: defn and segment labels
  flatten+{α,/dat+1 w} A flatten segment body
  parsehd+{ ' , 'part defl+w~ ' } A parse segment header
  separate+{(parsehd+ w)(flatten+1 w)} A separate segment headers and bodies

  re+{(a w){ '#'=w:0 o ~: 'εw:1 o w+~w:1: '2+w~ '}} A repn effect of defn
  nrs+{(w+~w:0,0~2α)} A next rep stack: w mod by rep effect α
  crs+1+{1+~nrs/((-1+pw)+cφ(cα),w)} A cum rep stacks from seed stack 1
  onflat+{(α,/1+~p w) cα α, w} A α on flat w, then re-enclose result

  blank+{r+w o m+~v/0 ~1 ~2φ'c'/1: 'εw o (m/r)+ ' o r} A blank out '/1: '
  fp+{x1|w:1+~0 10+10×w o w} A float: convert real to integer
  tbl+{(w^.= ' ')v0 !≡: 'εw} A tbl declaration
  fshp+{tbl w:0 o φfp('N'=w){w+at~pw}, blank 1+w} A 'N' have extra blank

  flip+~φ+1 A invert order
  fretby+{w1'(0,~1+~w)1~cα} A fret data seg α by field lengths w

  nstacks+{(crs+(re))onflat w} A nesting stacks
  nmshp+~o+{(α){fshp w})/φflip+{(2+('part w))} A field names and shapes
  nstate+{f d+w o ((nstacks f){α w}~2+~f)(d)} A infix stacks:
  A ((rs def)(dat)) ((rs def)(dat))
  parse+{r f+α o (c1+r),ns,cw fretby~r,~2ns-nmshp f}
  A ((rs defs) data) to (rss names shapes data)

  α,~/>parse/nstate separate segment crlf lineate w
}

```

Exhibit 3: Alternative ending to parseFile

```

nstacks+1+~o+{(crs+(re))onflat w} A nesting stacks: discard initial 1
heads bodies=separate segment crlf lineate w A retain segment structure
A (segs are truncated)
stacks+nstacks heads A nesting stacks
namshp+~o+{(2+w)(fshp w))/2+('part w} A field name and shape
names shapes=flip flip+namshp~heads A field names and shapes
full+bodies fretby~x/~stacks,~shapes A undo trunc and fret to fields

α+{,/}~stacks names shapes full A remove segment structure
}

```

How To Rig An Election

by Ian A. Clark (earthspot@yahoo.com)

My nominations for the two most beneficial inventions of all time are:

- the telephone, because it enables people to talk to each other who hitherto could not,
- the computer, because it enables the people to express their political will where hitherto they could not.

But both inventions have their dark side. The telephone gave rise to the loudspeaker and talking films, exploitation of which in the first half of the 20th century enabled the fascists to win power and consolidate their support in many European countries. In the first half of the 21st century, when computer technology (IT) can be expected to reach maturity, some feel that the world is yet again at risk of absolute power falling into the hands of the first group of people who exploit the new technology to its fullest extent.

The maintenance of democracy in the rapidly-growing population of the USA (for which the VAP or “voting age population” currently stands at around 200 million [4]) has historically depended on IT. An early application of the Hollerith punched card system processed census data and ran elections. But IT, formerly so beneficial, may itself pose a threat to the democratic process if it is allowed either to stagnate in public use or develop out of control.

The Model used in the Workspace: VOTE

The workspace VOTE offers a model of the voting process for the newly-enfranchised citizen to play with, in order to gain an understanding of its possibilities and abuses.

Our “toy” democracy has just two parties: the Apples and the Pears. We are to imagine that the Apples are in power and are seeking re-election. To achieve this they must win a majority of seats (one seat per ward, in this simple model). But this time their prospects are not rosy: the Pears have mobilised enough support to outvote them. What can the Apples do to stop the country going pear-shaped?

Well, quite a lot – and not all of it honest campaigning. Though if one party doesn’t put much effort into campaigning, it will warn the other side that it is confident it has things nicely stitched-up. But in the light of what we have to say,

the reader may well wonder why any party in power need ever relinquish it. George Orwell [1] thought it could only come about because the party lost the will to govern.

Ignoring countless variations and hybrids, we recognise four basic election rigging techniques:

- R1: Gerrymandering
- R2: Selective disqualification
- R3: Stuffing the ballot box
- R4: Selective discouragement.

We must remember that the lawyers for the Pears know these tricks too and will be watching the Apples closely. So success will largely depend on whether the Apples manage to disguise what they are doing.

VOTE isn't claimed to be a faithful model of any given democracy. But it does allow us to conduct *in-silico* experiments (as bioengineers call them) on how to rig an election. We can use it to demonstrate a variety of rigging techniques and take a guess at their relative cost-effectiveness.

We will then discuss the morality and the politics of what we have done (which are actually two different things). Demonstrating how to achieve a reprehensible goal is a Machiavellian thing to do. But we should recall that the chief concern of Machiavelli (in [2], if not so evidently in *The Prince*) was to draw the fangs of political abuses he identified in his native land by bringing them out into the open.

The Structure of the Workspace: VOTE

Uppercase names like `VOTERS` and `WARDS` are used for global variables, lowercase names like `support` and `vote` for APL functions. We shall refer to these global variables as the *parameters* of the model.

The model manages a collection of "wards", the number being determined by the value of parameter: `WARDS` (typically 100). Eschewing nested variables, we set up separate "parallel" arrays: `POPN[i]` `PRO[i]` `CON[i]` ... all referring to ward `i`.

The workspace VOTE is designed for experimentation by a non-APL programmer, simply by tweaking the parameters. These parameters are initialised by an ordinary textfile (`VOTE.INI`), which presses into service a simple text editor

(MS NOTEPAD) as a rough-and-ready interface. The same parameters have also been brought out to a crude Windows GUI interface, so the model can be adjusted by clicking combos and option buttons, once you understand what they do.

The model breaks down into these main functions, which may be rerun in turn from any stage onwards:

data	gets initial values for all model parameters.
populate	fills array: POPN which counts the voters in each ward.
support	allocates voters in POPN to either PRO or CON.
poll	fills arrays: VPRO & VCON with votes actually polled, handling turnout, defections and lost ballot-boxes.
vote	counts, analyses and displays the votes in VPRO & VCON.

Some of these functions are governed by a “mode” parameter, viz POPULATE_MODE SUPPORT_MODE POLL_MODE respectively, which select a :Case-block inside the said fn to define different implementations of that function, so they can be viewed as pluggable “building blocks” of the model. Thus when POPULATE_MODE equals 1, we say that fn: **populate** is running in mode 1.

Fn: data

Fn: **data** gets the values of the model parameters from VOTE.INI, echoing them in the APL session log to provide a visual confirmation, This also permits them to be re-entered by someone competent to use the APL session log. Use of the Windows GUI makes the APL session log redundant.

Fn: populate

In mode 1, fn: *populate* simply assigns a fixed number: VOTERS (=1000) to each of the (100) wards by the statement:

```
POPN[←]VOTERS
```

In mode 2, *populate* assigns a random number of voters to each ward by Zipf’s Law, which governs the distribution of individuals among such classificatory groupings as that of sharing the same surname. We achieve a Zipfian distribution by:

```
POPN←rounded VOTERS÷iWARDS
```

The result of this is that if $POP_N[1]=VOTERS$, then $POP_N[2]=VOTERS \div 2$, $POP_N[3]=VOTERS \div 3$... $POP_N[i]=VOTERS \div i$... which imposes Zipf's Law. The *fn* then goes on to scale-up POP_N in proportion to give us a total population ($+ / POP_N$) which is the same as that in mode 1, viz. 100,000 – a value which changes of course if you alter either *VOTERS* or *WARDS*.

Fn: support

Fn: support now allocates voters to *PRO* and *CON*. The workspace currently only models a two-party system, and we are to imagine the Apples are in power. Therefore *PRO* is the number of Apple supporters in each ward and we must suppose everybody else (*CON*) supports the Pears (we shall handle non-voters and defectors later).

This is implemented by the statement:

```
CON+POP_N-PRO
```

But first we create *PRO*. In mode 1 this is done by:

```
PRO+PERCENT[1] percent_of POP_N
```

where $PRO[i]$ gets a (rounded) fixed percentage of $POP_N[i]$.

In mode 2, *fn: support* allocates individuals according to a binomial distribution. This is achieved by *fn: coins*, which postulates that each voter in a ward tosses an unbiased coin to vote.

```
PRO+POP_N coins**50
```

Notice that for ease of handling the model interface, all probabilities, proportions and tendencies are expressed in percentages, and usually integer percentages at that.

In mode 2 we give ourselves the option to split the total population evenly between *PROs* and *CONs*, although some wards may have more of one than the other. Then, as we investigate spooky effects which win the *PROs* more wards than we think they deserve, we can be sure it's not due to the *PROs* having an overall majority as things are. *Fn: balance* sprinkles the necessary additional (or "subtractional") *PRO* members among the wards at random. It is switched off by $BALANCE_MODE+0$, and switched on by $BALANCE_MODE+1$.

Further modes offer increasingly complex cases, including ones where the percentage support for the CONs depends on the size of population in the ward. See the partial code listing at the end of the article.

Fn: poll

Fn: **poll** now determines how the voting goes on polling day. It derives VPRO and VCON from PRO and CON, using percentage turnout rates and defections to adjust a random element in the decision of a PRO (or CON) supporter to cast his or her vote.

Mode 1 completes the polling in one shot. It then runs **fn: vote** (see next).

Mode 2 has a more complex behaviour. Here polling is reported and analysed one ward at a time. A combo-controlled timer releases successive results either slowly or fast. The user now has the option of intervening to rig the result according to how badly things are going. See its use below when we discuss voting machines and postal votes.

Fn: vote

Fn: **vote** now analyses VPRO and VCON to report the results. It does not change any arrays, so each time it is run it will give the same result.

Redrawing Boundaries: is it just to ensure fair Representation?

Zipf's Law tends to be obeyed by those population groupings in which the rate of growth of a given group (we are interested in "wards" here) is proportional to its size. This is something that might be expected of wards if they were allowed to grow in an uncontrolled fashion – which in general they are not. Instead most democracies have provisions to redraw ward boundaries every ten years to equalise the populations within them, the idea being to achieve fairer representation of voters in the legislature.

In the USA, by a ruling of the Supreme Court, "districts" (UK: "wards") must be "re-zoned" (UK: "boundaries redrawn") every ten years at least. In the UK the Boundaries Commission undertakes this task every ten years. However in the USA it is invariably performed by locally-elected officials or political appointees, who are far from being disinterested agents. In the past, both Democrats and Republicans have been accused of taking advantage of this legal requirement to move enough of their opponents out of a given district to capture it for their own side. Recently, in the State of Texas, Republican-appointed officials were alleged

to have done this using sophisticated software, one district (#25) getting itself known as the "Bacon Strip" for its egregious shape. Since 2000, re-zonings in Texas and Pennsylvania have been hotly contested in a series of bizarre actions, including one by Texas Democrats (the "Killer D's") who delayed the 2003 Restructuring Bill (which they claim was a Republican gerrymander) by fleeing to neighbouring Oklahoma in order to make the legislature inquorate.

To be fair, Republicans claim the said Bill was necessary to undo a Democrat gerrymander in 2001. But... when does a venial sin become a mortal sin? When it stops being hard work and starts being effective?

Even before the November 2004 elections, the lawyer for the Democratic challenge in the High Court went on record as saying: "it is now possible to take a mildly Democratic state and gerrymander it to return a strong Republican majority". The Democrats insist that the Republicans have engaged in wholesale gerrymandering across the USA, a project they say has been coordinated by House Leader Tom DeLay. But this will be hard to establish without producing the modern software that did the job.

But there is no doubt such software exists. Cognos Planning, formerly Adaytum Planning, formerly Kunzle Planning System (KPS) [7], is quite up to the job, though it lacks built-in geographic functionality. Of interest to *Vector* readers: this product happens to be written in Dyalog APL. The author would not be at all surprised to learn that this, or a competing product, had been used to gerrymander a re-zoning exercise. What would surprise the author would be to learn that the tool employed had been written in Visual Basic or Java.

Gerrymandering software exploits the unprecedented detail of present-day census data to permit a far more effective gerrymander than hitherto without gross distortion of a district's boundaries.

R1: Gerrymandering

The Gerrymander is named after Elbridge Gerry, whose re-zoning activity in the Commonwealth of Massachusetts was so blatant as to create a salamander-shaped district. This was in 1812 – which shows how long the practice has been going on.

Here's how it works.

Consider two adjacent wards, W1 and W2, which are scheduled for re-zoning. Let's suppose they have absurdly small populations to highlight the effect, viz. 21 and 33 inhabitants respectively.

This expression models the allegiance of voters to Apples and Pears respectively:

```

+/'Z+(W1 W2)+(10 11)(16 17)
21 33
x-/'Z A 1 supports Apples, -1 supports Pears
-1 -1

```

Notice that both W1 and W2 are currently Pear wards, the Pear majority in each being precisely 1 vote. Now, W2 being the larger ward, (W2-W1)=12, we are legally required to redraw the boundary to even-up the populations by identifying an area containing half that number (6 voters) for transfer from W2 to W1. But suppose we are at liberty to choose these 6 voters with as skewed an allegiance as we can manage, in this case 2 voting Apple and 4 voting Pear. It's not hard to achieve if (say) Pear voters predominate in poorer neighbourhoods.

This expression performs the required population transfer of 6 voters:

```

+/'TRANSFER+2 4
6
[]+Z+(W1 W2) + (1 -1)x''=TRANSFER
12 15 14 13
x-/'Z A 1 supports Apples, -1 supports Pears
-1 1

```

but whilst performing our legal duty we have deliberately contrived to capture one of the two Pear wards for the Apples. This is what is meant by gerrymandering.

R2: Selective disqualification

Selective disqualification is when the Apples try to stop given groups of people from casting votes that count, in a manner which stops more Pears than Apples. This can be by:

- direct targeting, by disqualifying a given group of people from the official lists of those eligible to vote (like those with criminal records)
- indirect targeting, by losing or rejecting all the votes from a given set in which there are more votes cast for the Pears than the Apples (e.g. those cast at a given polling station or voting machine).

Direct targeting is something the Pears will be watching for. The Apples know they won't get away with disenfranchising someone (striking them from the voters' list) just because they've said they'll vote Pear. It's got to be for some reason which most sane people will accept, e.g. the voter is insane.

The 2000 election in Florida was notorious for the poor usability of the voting machinery (e.g. the “butterfly ballot”) and the selectivity of the people affected by this – e.g. older people, the less well-educated, or recent immigrants. When new digital voting machines were installed prior to the 2004 elections, those suspecting a conspiracy had cause to wonder. Was their deployment conspicuously free from political interference? Well – no. What might be going on inside those machines? Hard to say.

The Apples, too, have installed new voting machines and have control over their operation on polling day. The Pears know this, so the Apples deliberately draw their fire by spreading a rumour that the machines contain illicit code to disqualify a selection of Pear votes. They don’t, of course: the political fallout from the Pears finding any such code would be far too great to bear. There’s a simpler trick the Apples can play.

The Apple election boss (you) can trigger a software glitch which disqualifies all the votes from any selected ward that has yet to declare its results. It doesn’t need hi-tech to apply this principle. You can also lose selected ballot boxes, or delay bags of postal votes from overseas. You won’t get away with it on a large scale, but – hey! – the voting machines are new and the occasional glitch may pass scrutiny. Demographic software allows the Apples to predict which wards are likely to fall to the Pears. Those wards which are also likely to deliver their results late in the day are the ones to target.

Fn: poll in mode 2 starts a timer which delivers results ward-by-ward, once every two seconds. The GUI’s “delay” combo can speed this up or slow it down. Set the “poll” combo to 2 (signifying mode 2) and click button “poll”. As voting proceeds, the figure labelled “PRO AHEAD/BEHIND” (i.e. <PRO seats> – <CON seats>) executes a drunkard’s walk towards what you’re hoping will end up as a positive number, signifying an Apple victory. If you don’t like the way things are going, then click: “knockout”. This runs **fn: ko**, which disqualifies the ward with the greatest predicted Apple defeat, so it’s worth one ward to you each time you click it.

With judicious use of the “knockout” button you can usually steer the Apples into a marginal victory. You may not have to use it at all, if luck (in the form of “?” or [roll], the APL random number generator) is acting in your favour. If you over-use it and you win by a much greater margin than the exit polls predict, the Pears will ask why so many voting machines went wrong – and they might see a pattern and rumble you.

This is a stealth tactic. It is only feasible because partial results are analysed and broadcast by the networks as they come in, leaving those wards which have yet to declare their results vulnerable to what casinos call "black action".

R3: Stuffing the ballot box

You can artificially disqualify your opponents' voters – and you can artificially inflate your own. The Caltech/MIT study [4] examined 3rd-party registration: there are numerous organisations which register voters en-bloc – people who have not voted before, residents of old-folks' homes, immigrant groups, etc. – and handle the voting process for them. It stands to reason that such blocs of voters are more likely to vote the way they are told than free independent citizens. There is also the matter of sheer fraud. Over a 20 year period, 5,400 dead people were discovered to have voted in the state of Georgia alone [4], however it's not reported whether the dead mostly vote Republican or Democrat. One needs a sixth-sense to know that.

Registration is designed to combat blatant fraud, but the problems of scale with the registration system is identified as one of the major issues bedevilling the voting process in the USA. The statistics quoted in [4] are disturbing. Thus the National Voter Registration Act of 1993 (called "Motor Voter" for its linking of voter registration with the renewal of drivers' licences) led to a big increase in "inactive" registrants, from 1.7 million in 1994 to 14.6 million in 1998. That's a lot of spooky voters out there. It just needs some Aragorn to go exploring these "Paths of the Dead" for a legion of hitherto unsuspected supporters to burst upon the scene and tip the balance for one party or the other.

VOTE does not try to model this issue. Mathematically it is a trivial one, but in practice it is vastly important.

R4: Selective discouragement

Unlike the R3 techniques, seeking to discourage your opponent's supporters has the advantage of being free from the smell of fraud. But running VOTE in cases where the PROs and CONs are evenly balanced shows just how sensitive the outcome is to even a single-point change in the turnout rate of your supporters. With the model parameters set up as shown in VOTE.INI below, increasing the PROs' no-vote rate from 10% to 11% (click combo "nv/PRO", then click "poll" and "vote" again) tends to cost you around 10 wards, the random swing being about 5-6 wards either way, so it signifies almost certain defeat. Tony Blair has recently warned his supporters that the outcome of the May 2005 election is likely to be decided on a few hundred votes in a handful of seats.

The Week [3] quotes Michael Howard's advisers as telling him he needed just 838,000 people in 165 marginal Labour seats to switch their allegiance from Labour to Conservative in order to win an overall majority. You can be sure that these figures have not been dragged out of the air. But if they are wise they'll advise him to pursue a policy of discouragement through "negative campaigning" – e.g. impugning the character of his opponents – rather than trying to woo the opposition, to whom he may well have nothing to offer.

The gerrymander in Texas and elsewhere was a costly thing to do if its only aim was to capture a few districts. Of far more value was the effect it undoubtedly had on the poorer voters, predominantly Democrat supporters, to convince them that everything was stitched up and they hadn't a chance of their votes counting (even if they were actually counted!) So it may actually have been an application of R4 rather than R1.

The Apples likewise think it is too costly to woo voters, who might demand that they introduce measures they'd rather not as the price of defection. Each defector is worth two votes, one lost to Pears, one gained by Apples. A couple of Pear supporters who've been given the pip is worth exactly the same and is a lot cheaper to achieve. It is known, for instance, that younger people are more likely to be impatient with the status-quo and thus vote for change, so it serves the purpose of the Apples to spread the word that politics is a dirty game and there's far more fun to be had on a rainy afternoon than queuing for hours at some draughty polling station.

Shady tactics – but is it rigging, or just robust campaigning? Arguably it's a case of R4, if you use demographic data to target your selective discouragement activities, even if the tactics you then employ are a battery of miscellaneous measures. If you don't act heavy in too many wards, international observers invited-in from Poland and elsewhere will have little cause to accuse you of "serious" or "widespread" abuse (to them that means bodies with bullets in the back) so R4 can be a stealth tactic too.

Why Should we Worry?

In July 2001, the Caltech/MIT Voting Technology Project delivered its report [4] on the state of the nation's voting arrangements. One of the intriguing things they found was that the number of lost votes across the nation was 2%-3% in presidential elections, and they compared this to the fact that in 2000 the winner's margins in 4 states: Florida, Iowa, New Mexico and Wisconsin, was less than 0.5%. In response the federal government saw fit to mount a massive program to restore

public confidence in the electoral process (HAVA: Help America Vote Act, 2002 [5]).

There will be calls for much the same in Britain after the May 2005 elections. But perhaps we should not be demanding yet another exercise in restoring public confidence, but instead aim to construct a voting system which merits our confidence. This should not be beyond the wit of man to devise, considering the clever things that man can already do to subvert the existing apparatus. We should consider whether the following activities are now so dangerous to our democratic way of life, thanks to advances in IT, that they should either be outlawed or be strictly reserved to an impartial and independent agency:

- third-party registration of voters (R2, R3)
- collecting and processing data on individuals' party-political preferences and voting behaviour (R1, R2, R3, R4)
- partial publication of results while voting is underway (R2, R3)
- exit polls and other unofficial polls while voting is underway (R4)
- issuing new postal votes (R2, R3)
- canvassing postal votes from people who have never yet voted (R2, R3)
- publicly inviting certain classes of people to stay away from the polling booths on voting day (R2, R3, R4)
- publicly impugning the character (as distinct from the policies) of a candidate in the two weeks preceding an election (R4).

After each suggestion the rigging techniques it combats are shown in brackets.

Conclusion

Democracy. For all its imperfections, it is still the most effective way mankind has discovered to curb the persistent abuse of power – and free and fair elections are the sole means to preserve it. You either believe this or you don't.

Loss of belief in it led Benito Mussolini, an ardent socialist, to forsake the people's cause and become the first fascist. He was, and still remains, fascism's most articulate theorist, in spite of his more notorious (and undeniably articulate) imitator in Germany. It is good to read Mussolini's article [6] defining the nature of fascism: the notion being touted in the USA of running government like a business corporation is clearly seen to be his – and it is a central one to his ideas.

But the democratic process, as we show with our VOTE model, is a chain which is no stronger than its weakest link. We should not wait for voting abuses to become "widespread" before we start to worry, but attack irregularities wherever they occur, not leave the fight to a few locals who realise what is going on. Otherwise who's to blame if we wake up to find ourselves no longer living in a democracy, but an oligarchy flaunting a few democratic customs?

There is no such thing as a small fire in the kitchen. There is a major case for stern measures whenever voting abuses occur: they should be investigated for complicity at the highest level – and those found guilty never again entrusted with public office. This may mean disqualifying one or two charismatic leaders, but others will rise to fill their places. All too readily, the leaders may think. We should reject the claim some will make that the brilliant brains of certain individuals would be a loss to the nation – this is to succumb to the cult of personality. It must remain a central principle of democracy that the will of the people, fairly expressed, takes precedence over the will of any one individual, however well-endowed, well-educated, or well-connected.

References

- [1] Orwell, George (1948), *Nineteen Eighty-Four*.
<http://www.abebooks.com>
- [2] Machiavelli, Niccolò, *Discourses on Livy*. University of Chicago Press (1996). ISBN 0-226-50036-5
- [3] THE WEEK (9 Apr 2005) Blair calls an election, p3.
<http://www.theweek.co.uk>
- [4] Caltech/MIT Voting Technology Project (2001), *Voting: What Is /What Could Be*.
<http://www.vote.caltech.edu>
- [5] Help America Vote Act (2002).
<http://www.fec.gov/hava/hava.htm>
- [6] Mussolini, Benito, (1932), *The Doctrine Of Fascism*.
<http://www.worldfuturefund.org/wffmaster/Reading/Germany/mussolini.htm>
- [7] Clark, I.A. (1996), KPS: Beyond the Spreadsheet. VECTOR 12.3, p55.

Partial listing of the VOTE model

VOTE is available by email from the author. It is a workspace running under APL+Win v4.

```

      v populate;totpop
[1]  Apopulate the wards
[2]  POPN←PRO+CON+UND+WARDSp0 A--setup the working arrays
[3]
[4]  :Select POPULATE_MODE
[5]
[6]  :Case 1
[7]      mode 'equal number of voters in each ward'
[8]      POPN[]←VOTERS
[9]
[10] :Case 2
[11]     mode 'ward populations skewed by Zipf Law'
[12]     totpop←WARDS×VOTERS
[13]     POPN←rounded VOTERS÷iWARDS A--Zipf
[14]     POPN←rounded POPN×(totpop÷+/POPN) A--boost in proportion
[15]     POPN←totpop padup POPN A--make up the small discrepancy to totpop
[16]  A POPN←shuffled POPN
[17]     POPN←⊖POPN A--biggest at the back, for ko to work well
[18]
[19] :EndSelect
      v
      v support;t;z
[1]  Aallocate support for/against
[2]  :Select SUPPORT_MODE
[3]
[4]  :Case 1
[5]      mode 'fixed % of voters in each ward are PRO'
[6]      PRO←PERCENT[1] percent_of POPN
[7]
[8]  :Case 2
[9]      mode 'balanced: all voters toss a balanced coin'
[10]     PRO←POPN coins" 50
[11]     balance
[12]
[13] :Case 3
[14]     mode 'fixed % support for PRO & CON, residue toss (fixed-bias) coin'
[15]     PRO←PERCENT[1] percent_of POPN
[16]     z←POPN-PERCENT[2] percent_of POPN
[17]     PRO←PRO+(z coins" TENDENCY[1])
[18]
[19] :Case 4
[20]     mode 'max % PRO support irrespective of size of ward'
[21]     PRO[]←POPN [ PERCENT[1] percent_of VOTERS
[22]
[23] :Case 5
[24]     mode 'fixed PRO support is all of smallest ward'
[25]     PRO[]←⌊/POPN
[26]

```

```

[27] :Case 6
[28]   mode 'voters toss a coin biased by size of ward'
[29]   PRO+POPN coins" TENDENCY percent_by POPN
[30]
[31] :EndSelect
[32]
[33] CON+POPN-PRO A--CON is everybody else (2-party only)
[34]
[35] report logex '+/PRO' '+/CON'
  ▽

  ▽ poll
[1] Aconduct the poll (c/f: vote)
[2] default 'POLL_MODE=1'
[3] mode 'conducting the poll'
[4] pll POLL_MODE
  ▽

  ▽ pll flag;z
[1] A send voters to the polls, in mode: flag
[2] A(called by: poll. Runs: vte, timer)
[3] ANB: Cases 1 and 2 are POLL_MODE settings, for call with a single button
[4] A Other cases are service-calls, recursive calls, or calls by the timer
[5]
[6] :Select flag
[7]
[8] :Case 1 A--one-shot poll (POLL_MODE setting)
[9]   pll "vars"
[10]   vote
[11]
[12] :Case 2 A--stepped poll (POLL_MODE setting)
[13]   timer "toggle" A--timer callback fn: timer_action calls: pll
[14]
[15] :Case "vars" A--compute working vars
[16]   z+DEFECT+NOVOTE
[17]   NPRO+PRO coins" 1>z A--loss to PRO by defection & non-voting
[18]   NCON+CON coins" 2>z A--loss to CON by defection & non-voting
[19]   z+100*DEFECT÷z A--of which defectors AS A %age
[20]   DP2C+NPRO coins" 1>z A--of which: defections PRO+CON
[21]   DC2P+NCON coins" 2>z A--of which: defections CON+PRO
[22]   VPRO+0{PRO+DC2P-NPRO A--actual turnout in favour of PRO
[23]   VCON+0{CON+DP2C-NCON A--actual turnout in favour of CON
[24]   {VPRO0 VCON0}-VPRO VCON A--saved for debugging: ko
[25]
[26] :Case "part" A--zero the future part of VRPO VCON...
[27]   {VPRO1 VCON1}-WARDS+"NTICK:"{VPRO VCON}
[28]   vte 2 A--partial estimate
[29]
[30] :Case "final"
[31]   vte 1
[32]   timer "stop"
[33]
[34] :Endselect
  ▽

```

```

    ▽ ko;i
[1]  Adiscard votes from ward with biggest CON majority
[2]  i←+VCON-VPRO
[3]
[4]  :If VPRO[i]=0
[5]  :AndIf VCON[i]=0
[6]      report myname ">>> no votes anyway from ward:" i
[7]  :Else
[8]      timer "pause"
[9]      VPRO[i]+VCON[i]←0
[10]      report myname ">>> all votes discarded from ward:" i
[11]  :End
    ▽
```

Contents of: VOTE.INI as distributed

```
[GENERAL]
WARDS=100

VOTERS=1000
;--basic number of voters in each ward

INTERVAL=2000
;--stepping interval in millisecs

[PERCENT]
;Vector %s for allegiance turnout and defection of PRO & CON respectively...
PERCENT=50 50
NOVOTE=10 10
DEFECT=5 5
TENDENCY=50 50
;--tendency to vote PRO (for smallest/largest wards)

[MODE]
BALANCE_MODE=1
POPULATE_MODE=2
SUPPORT_MODE=2
POLL_MODE=1
```

Queens and Knights

by Roger K.W. Hui

The Problem

The webpage <http://www.itasoftware.com/careers/programmers.php> often contains interesting programming puzzles. The "Queens and Knights" puzzle was posted in 2003-2004:

In 1850, Carl Friedrich Gauss and Franz Nauck showed that it is possible to place eight queens on a chessboard such that no queen attacks any other queen. The problem of enumerating the 92 different ways there are to place 8 queens in this manner has become a standard programming example, and people have shown that it can be solved using many different search techniques.

Now consider a variant of this problem: you must place an equal number of knights and queens on a chessboard such that no piece attacks any other piece. What is the maximum number of pieces you can so place on the board, and how many different ways can you do it?

The `n` queens problem can be solved succinctly and efficiently. A solution is presented in the `for .` page of the J dictionary.

The queens and knights problem is considerably more difficult.

Configs

A *config* is an `n*n` (`n=:8`) element string recording the placement of queens (`Q`) and knights (`N`) together with the coverage, the squares under attack by pieces already on the board and the squares from where additional pieces would attack an existing piece.

`x` below is a config. `Q` indicates a square where a queen is placed, on row 2 and column 6. `x` indicates a square under attack. `n` indicates a square where a knight, if there placed, would attack `Q`. A `"."` indicates a free square.

A queen can be placed on a `"."` or `q` square. A knight can be placed on a `"."` square.

```
n=: 8
```

```
x=: '....xqxq....qxxxxxxxxxQx....qxwx....xqxq...x..x.
     ..x...x..x...x.'
```

```

      (n,n) $ x
....xqxq
....qxqx
xxxxxxQx
....qxqx
....xqxq
...x..x.
..x...x.
.x....x.

```

y below is another config. N indicates a square where a knight is placed, on row 6 and column 1. x indicates a square under attack. n indicates a square where a queen, if there placed, would attack N. A "." indicates a free square. A knight can be placed on a "." or n square. A queen can be placed on a "." square.

```

y=: '.n....n.n....n...n...n...n...xnxn....nnnx....
     nNnnnnnnnnnnx....'

```

```

      (n,n) $ y
.n....n
.n....n.
.n...n..
.n..n...
xnxn....
nnnx....
nNnnnnnn
nnnx....

```

Our plan for solving the queens and knights problem is as follows: starting with the empty config $((n*n)$ ' ')$, place queens and knights one piece at a time on all possible squares. As a piece is placed the config is updated. Placing a piece and updating the config means merging the config with a config that represents putting a piece on a particular square, to derive a new config.

Consider again the configs x (a Q on the (2,6) square) and y (an N on the (6,1) square). How do we merge them? That is, what is the config for a board with a Q on (2,6) and an N on (6,1)?

```

      (<n,n) $&.> x;y;'?'
+-----+-----+-----+
|....xqxq|.n....n|????????|
|....qxqx|.n....n|????????|
|xxxxxxQx|.n...n.|????????|
|....qxqx|.n..n..|????????|
|....xqxq|xnxn...|????????|
|...x..x.|nnnx....|????????|
|..x...x.|nNnnnnnn|????????|
|.x....x.|nnnx....|????????|
+-----+-----+-----+

```

A Calculus of Configs

We have seen that a config can contain the letters .NQxnq. To merge two configs, we need to determine what letter should be the result for every combination of two letters.

```

.   free
N   N sits here
Q   Q sits here
x   under attack
n   Q here would attack N
q   N here would attack Q

```

It is clear that "." (a free square) with any letter should result in that letter. N (a knight on this square) with n (queen here would attack a knight) should be N, but N with anything other than "." or n should be ?, an error.

The combinations (n, q) or (q, n) merit some comment. A queen can be placed on a q square but not on an n square, and a knight can be placed on an n square but not on a q square. Therefore, a square that is both n and q is equivalent to an x square, "under attack", because no piece (neither queen nor knight) can be placed there.

The results for other combinations are derived similarly.

	free	.	N	Q	x	n	q
N	N sits here	N	?	?	?	N	?
Q	Q sits here	Q	?	?	?	?	Q
x	forbidden	x	?	?	x	x	x
n	Q here would attack N	n	N	?	x	n	x
q	N here would attack Q	q	?	Q	x	x	q
?	error						

The list MT records the ravelled operation table. The rows and columns are in the order .NQxnq. Having MT, it is straightforward to write a verb that merges configs x and y: index into MT using letters in x as row indices and letters in y as column indices.

```

MT   =: ' . - . ~ ' .NQxnq N???N? Q????Q x???xxx nN?xnx q?Qxxq'
MTy  =: (%: #MT){.MT
MTx  =: (##)]MTy

```

```
merge=: MT"_ {~ MTx&i.@[ + MTy&i.@[
```

```

      (,~%:#MT) $ MT
.NQxnq
N???N?
Q????Q
x???xxx
nN?xnx
q?Qxxq

      (-: |:) (,~%:#MT) $ MT
1

      (<n,n) $&.> x;y;x merge y
+-----+-----+-----+
|...xqxq|.n....n|.n..xqxx|
|...qxqx|.n....n|.n..qxqx|
|xxxxxxQx|.n....n|.n..xxxxQx|
|...qxqx|.n..n..|.n..xxxx|
|...xqxq|xnxn...|xnxnxqxq|
|..x..x.|nnnx...|nnnx..x.|
|..x..x.|nNnnnnnn|nNxnnnnn|
|.x...x.|nnnx...|nxxn..x.|
+-----+-----+-----+

      x (merge -: merge~) y
1

```

The table QT is an $n*n$ row table of configs, one for each possible placement of a queen. The table NT is an $n*n$ row table of configs, one for each possible placement of a knight.

```

UI  =: >, {;~i.n
NI  =: _2]\_2_1_2 1 _1_2_1 2 1 _2 1 2 2 _1 2 1
QI  =: 0 0 -~ (0&, , ,&0 , ,~ , (.,|.)) i: n
mask= 3 : 'UI e."2 UI +"1"1 2 y.'
QT  =: '.Qqx' {~ (=i.n*n)+(2*mask NI)+3*mask QI
NT  =: '.Nxn' {~ '.Qqx'i.QT

      $ QT
64 64

      $ NT
64 64

```

```

(<n,n) $&. <"1 (16 17 18 19){QT
+-----+-----+-----+-----+
|xqx.....|qxqx.....|xqxqx...|xqxqx...|
|xxq.....|xxxq.....|qxxxq...|qxxxq...|
|Qxxxxxxxx|xQxxxxxxxx|xQxxxxxxxx|xxxQxxxxx|
|xxq.....|xxxq.....|qxxxq...|qxxxq...|
|xqx.....|qxqx.....|xqxqx...|xqxqx...|
|x...x....|x...x....|...x...x|x...x...x|
|x...x....|x...x....|...x...x|...x...x|
|x...x....|x...x....|...x...x|...x...x|
+-----+-----+-----+-----+

```

```

(<n,n) $&. <"1 (16 17 18 19){NT
+-----+-----+-----+-----+
|nqn.....|xnqn.....|nqnqn...|.nqnqn...|
|nnx.....|nnnx.....|xnnnx...|.xnnnx...|
|Nnnnnnnnn|nNnnnnnnnn|nnNnnnnnn|nnnNnnnnn|
|nnx.....|nnnx.....|xnnnx...|.xnnnx...|
|nqn.....|xnqn.....|nqnqn...|.nqnqn...|
|.n...n...|.n...n...|.n...n...|.n...n...|
|.n...n...|.n...n...|.n...n...|.n...n...|
|.n...n...|.n...n...|.n...n...|.n...n...|
+-----+-----+-----+-----+

```

Queens and Knights

With verb `merge` and tables `QT` and `NT`, the queens and knights problem is ready for solution.

`q1` applies to a table of configs and produces new configs by placing one more queen in all possible (". " or `q`) squares. Similarly, `n1` applies to a table of configs and produces new configs by placing one more knight in all possible (". " or `n`) squares.

`x qx y` puts `x` additional queens on the configs `y` by calling `q1` `x` times, at each step removing configs with less than the required number of ". " or `q` squares. Similarly, `x nx y` puts `x` additional knights on the configs `y`.

`qn y` finds all solutions of putting `y` queens and `y` knights on a chessboard, starting from the table of the empty config `((1,n*n)$'.'`). Since a queen attacks many more squares than a knight, thereby reducing many more possibilities, it is more efficient to place the queens first, then the knights.

```

q1=: 3 : '~.((+/"1 b)#y.) merge QT {~ (n*n)|I.,b=.
      y.e.''.q'''

```

```
n1=: 3 : '~.((+/"1 b)#y.) merge NT {~ (n*n)|I.,b=.
      y.e.''.n'''
```

```
qx=: 4 : 'for_j. (-i.)x. do. y.=. q1 y.#~j<:+"1
      y.e.''.q''' end.'
```

```
nx=: 4 : 'for_j. (-i.)x. do. y.=. n1 y.#~j<:+"1
      y.e.''.n''' end.'
```

```
qn=: 3 : 'y. nx y. qx (1,n*n)$''.'''
```

It is obviously not possible to place 8 queens and 8 knights, since 8 queens alone already attack the entire board. We start with 7 queens and 7 knights and work down from there.

```
# t=: qn 7
```

```
0
```

```
# t=: qn 6
```

```
0
```

```
# t=: qn 5
```

```
16
```

That is, up to 5 queens and 5 knights can be placed on an 8-by-8 chessboard, and there are 16 solutions.

The solutions can be displayed by mapping the letters Q and N to themselves and other letters to ".". Boxing and reshaping make a pleasing display.

```
$ t
```

```
16 64
```

```
(n,n) $ {. t
```

```
xQxxxxxx
```

```
xxxxxxQx
```

```
xxQxxxxx
```

```
xxxxxxQxx
```

```
xxxxxxxQ
```

```
Nxxxxxxx
```

```
NxxNNxxx
```

```
xxxNxxxx
```

```
see=: ,~@%:@# $ 'QN'&i. { 'QN.'"_
```

[illegible]

Variants

Let s be one of the solutions as displayed in the previous section. It is clear that reflections and transpositions of s , and compositions thereof, are also solutions.

```
s=: see { . t
      (] ; |: ; |. ; |."1) s
+-----+
|.Q.....|. ....NN.|...N....|. ....Q.|
|. ....Q.|Q.....|.N..NN...|.Q.....|
|. .Q.....|. .Q.....|.N.....|. ....Q.|
|. ....Q.|. ....NN|. ....Q.|. ....Q.|
|. ....Q.|. ....N.|. ....Q.|Q.....|
|.N.....|. .Q....|. .Q....|. ....N|
|.N..NN...|.Q.....|. ....Q.|. ....NN..N|
|. .N....|. .Q....|.Q.....|. .N...|
+-----+
```

```
NB. self ; transposition ; reflection 0 ; reflection 1
```

`var` computes all reflectional and transpositional variants of a solution. There can be up to 8 variants.

```
var=: ~.@(] , |:&.> , |.&.> , |."1&.>)^:3
      var <s
+-----+-----+-----+-----+-----+-----+
|.Q.....|. ....NN.|...N....|. ....Q.|.NN.....|. ....Q...|
|. ....Q.|Q.....|.N..NN...|.Q.....|. ....Q.|Q.....|
|. .Q.....|. .Q.....|.N.....|. ....Q.|. ....Q.|. ....Q...|
|. ....Q.|. ....NN|. ....Q.|. ....Q.|.NN.....|. ....N.|
|. ....Q|. ....N.|. ....Q.|Q.....|. .N.....|. ....NN|
|.N.....|. .Q....|. .Q....|. ....N|. ....Q....|.Q.....|
|.N..NN...|.Q.....|. ....Q.|. ....NN..N|. ....Q.|Q.....|
|. .N....|. .Q....|.Q.....|. .N...|. ....Q....|. ....NN.|
+-----+
```

To find the unique solutions with respect to reflections and transpositions, we can proceed as follows:

- find all the variants of each solution;
- sort the variants;
- select the "smallest" of each set of variants;
- find the nub of these smallest variants.

```
uvar=: ~. @: ({.@(/:~)@var&>) @: (<@<@see"1)
```

```
uvar t
```

```
+-----+-----+
|.....Q.|.....NN|
|.Q.....|.Q....|
|.....Q..|.Q.....|
|..Q.....|......NN|
|Q.....|......N|
|.....N|Q.....|
|...NN..N|.Q.....|
|...N...|......Q..|
+-----+-----+
```

Summary

- At most 5 queens and 5 knights can be placed on an 8-by-8 chessboard so that no piece attacks another.
- There are 16 different ways to place the 5 queens and 5 knights.
- If reflectional and transpositional variants are considered to be the same, then there are 2 different ways to place the 5 queens and 5 knights.

The following are the collected definitions for the queens and knights problem. The material in this article is available in the J 5.04 distribution as the "Queens and Knights" lab.

```
NB. . free . N Q x n q
NB. N N sits here N ? ? ? N ?
NB. Q Q sits here Q ? ? ? ? Q
NB. x forbidden x ? ? x x x
NB. n Q here would attack N n N ? x n x
NB. q N here would attack Q q ? Q x x q
NB. ? error
```

```
n =: 8
MT =: ' _ _ ~ ' .NQxnq N???N? Q????Q x???xxx nN?xnx q?Qxxq'
MTy =: (%:#MT){.MT
MTx =: (##)MTy
```

```
merge=: MT"_ {~ MTx&i.@[ + MTy&i.@]
```

```

UI   =: >, {;~i.n
NI   =: _2]\_2 _1 _2 1 _1 _2 _1 2 1 _2 1 2 2 _1 2 1
QI   =: 0 0 ~.~ (0&.,. ,.,.&0 ,.,.~ , (.,.|.)) i: n
mask =: 3 : 'UI e."2 UI +"1"1 2 y.'
QT   =: '.Qqx' {~ (=i.n*n)+(2*mask NI)+3*mask QI
NT   =: '.Nx'n' {~ '.Qqx'i.QT

q1   =: 3 : '~.((+/ "1 b)#y.) merge QT {~ (n*n)|I.,b=.
      y.e.''.q''

n1   =: 3 : '~.((+/ "1 b)#y.) merge NT {~ (n*n)|I.,b=.
      y.e.''.n''

qx   =: 4 : 'for_j. (-i.)x. do. y.=. q1 y.#~j<: +/"1
      y.e.''.q'' end.'

nx   =: 4 : 'for_j. (-i.)x. do. y.=. n1 y.#~j<: +/"1
      y.e.''.n'' end.'

qn   =: 3 : 'y. nx y. qx (1,n*n)$''. ''

see  =: ,~@%:@# $ 'QN'&i. { 'QN.'"_

var  =: ~.@(] , |:&.> , |:&.> , |."1&.>)^:3
uvar =: ~. @: ({.@(/:~)@var&>) @: (<@<@see"1)

```

Time(r) for the Game of Life

by Cliff Reiter

Introduction

The Game of Life is a process that updates a rectangular array of Boolean valued cells according to simple local rules. The process was introduced by John Conway and popularized by Martin Gardner [3,6,7]. In particular, a cell with value 1 is considered alive and at the next generation an alive cell remains alive if it has 2 or 3 alive neighbours among its closest 8 neighbours. A dead cell becomes alive if it has exactly 3 alive neighbours. The rich behaviour that results from these simple rules is fantastic. For example, Figure 1 shows the evolution of a Pulsar Puffer Train which is a configuration that moves to the right, leaving a new pulsar in its wake after every 36 steps. The alive cells are shown in colours that depend upon the number of neighbours they have. The Game of Life has been shown to be a universal computer [5] and considerable activity remains in identifying configurations that evolve in these fantastic ways [4,8,16].

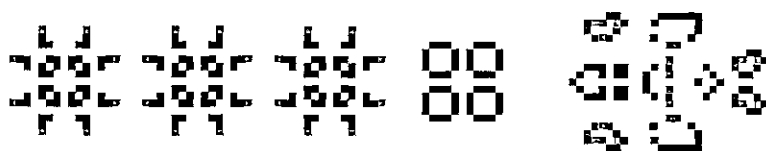


Figure 1. The Trail Left by a Pulsar Puffer Train

Because the process is so simple and the results are so intriguing, implementing the Game of Life is a popular programming exercise. Writing the "fastest" life program has a long history with some implementations using heap-sorted blocks of local activity to allow portions of the configuration to drift away without hindering performance [8].

There have been many discussions of the Game of Life in APL including a survey by McDonnell [10]. Interest in the APL community continues, recently including [1]. K implementations may be found at [2]. Implementing the Game of Life has arisen several times on the Jforum [9]. During the [9b] thread, Ewart Shaw [15] presented an interesting implementation and his amazing signature appeared. His signature implements and runs the game of life on a simple configuration.

Here is his signature and its result.

```

      3
      ((4&({*(.=+ /))++/=3: )@([:/0&,^:(i.3)@|:"2^:2))&.>@]^:(i.@[)
      <#:3 6 2
      +-----+
      |0 1 1|0 0 0 0 0|0 0 0 0 0 0|
      |1 1 0|0 1 1 1 0|0 0 0 1 0 0|
      |0 1 0|0 1 0 0 0|0 0 1 1 0 0|
      |   |0 1 1 0 0|0 1 0 0 1 0|
      |   |0 0 0 0 0|0 0 1 1 0 0|
      |   |   |0 0 0 0 0 0|
      |   |   |0 0 0 0 0 0|
      +-----+

```

We will say more about his signature later.

My interest in the Game of Life began around 1977 when a mentor told me that it could be easily implemented by making 9 copies of the cellular arrangement as a three dimensional array and then rotating the planes -1, 0, or 1 units in appropriate directions to organize each cell and its neighbours along one axis. I took the bait and implemented the Game of Life in APL using that idea. I also faced the issue of implementing the Game of Life for my visualization text and have been interested in generalizations for some time [11, 12, 13]. It recently occurred to me [9e] that algorithms for the Game of Life, such as the one in Shaw's signature, are fast enough in J that producing a real time GUI-based version of life is practical. GUI based versions have appeared based on Tcl/Tk [9a], but I have a special interest visualizing processes, and wanted to at least display cells with colour encoding their number of neighbours (as in the 3-dimensional representations in [11] which were not created in real time). The script I developed is available from [14].

Thus, the purpose of this note is to describe a few of the implementations of Life in J that I considered as I developed the script. I also briefly discuss using the system timer event, since that event was new to me and is a key to having "start" and "pause" buttons on the form.

Variations on Life

There certainly is no one "best" algorithm for life. One might be interested in speed, memory usage, brevity, readability, subtle beauty or something else, whether it is objective or subjective. Furthermore, context matters. For example, when I introduce the Game of Life in [13] it is in the context that some, but not all, of the features of J have been introduced. Furthermore, I know that in subsequent

sections I will consider fuzzy generalizations of Life, so it is natural to aim for coherence between those developments.

I have mentioned that rotating planes was the first approach I used in APL. In writing *Fractals*, *Visualization*, and *J*, I really wanted to use tessellations as provided by cut. It seems to me that tessellations separate the local processing from neighbourhood organization in a beautiful way. Here we see all the 3 by 3 overlapping neighbourhoods of *i.4 6* using cut (*:_3*).

```

3 3 <:_3 i.4 6
+-----+
| 0 1 2| 1 2 3| 2 3 4| 3 4 5|
| 6 7 8| 7 8 9| 8 9 10| 9 10 11|
|12 13 14|13 14 15|14 15 16|15 16 17|
+-----+
| 6 7 8| 7 8 9| 8 9 10| 9 10 11|
|12 13 14|13 14 15|14 15 16|15 16 17|
|18 19 20|19 20 21|20 21 22|21 22 23|
+-----+

```

In [13], the Game of Life was implemented by defining a local sense of Life and then applying it to the 3 by 3 tessellation. However, in the first edition of the book, infixes were used to spread the local behaviour since at that time cut was too inefficient (more on that later). Since there are fewer cells of the tessellation than original cells, we also need to deal with the boundary conditions. We will take periodic boundary conditions. First we implement the local life rule.

```

]L=:1,1 9 1,:1
1 1 1
1 9 1
1 1 1

]n1=:0 1 1,0 1 0,:0
0 1 1
0 1 0
0 0 0

L*n1
0 1 1
0 9 0
0 0 0

llife=: +/@:,@:(L&*) e. 3 11 12"_
llife n1
1

```

We encode the local processing using a matrix L . The sum of the entries in $L * n1$ is enough to determine the number of neighbours and whether the central cell is alive. Here the sum is 11 which corresponds an alive cell with 2 neighbours which is a configuration that yields life. Next we implement the periodic extension and then distribute the local life function using infix (`life1`) and cut (`life2`)

```

perext=: {: , ] , {:
perext2=: perext"1@:perext
perext2 n1
0 0 0 0 0
1 0 1 1 0
0 0 1 0 0
0 0 0 0 0
1 0 1 1 0

life1=: 3&(llife\)|: "2@(3&([\])@:perext2

life2=: 3 3&(llife;._3)@:perext2

life1 n1
1 1 1
1 1 1
1 1 1

```

Table I shows timing tests from 1995 [12] that show `life2` was infeasible in that era. By the second edition of book, `life2` became the method used. Notice that in order to have subsecond screen updates, we would have been restricted to around a 10 by 10 array in 1995. Before looking at more modern performance, we discuss the signature.

n	life1		life2	
	time	space	time	space
5	0.12	2212	0.32	8344
10	0.75	2812	1.66	45684
20	3.57	4564	8.34	266248
40	15.65	12972	44.1	1679530

Table I. Performance of Life, Circa 1995.

Ewart Shaw's Signature

In the text of [9b] Ewart Shaw defines `test=: (4&{ *. +/=4:) + +/=3:` while in his signature he uses a variant. We define the following.

```
test=: 4&({*(=+/))++/=3:
```

```
shift=: [: ,/0&,^:(i.3)@|:"2^:2
```

With those definitions his signature reduces to the following.

```
3 test@shift&.>@|^:(i.@[] <#:3 6 2
```

Except for `test@shift`, this is iteration control, boxing and creating the initial configuration. Thus, `lives=:test@shift` implements the Game of Life with expanding boundary conditions. We will see how those two pieces of the function work.

First, `shift` appends a number of zeros (from `i.3`) along each axis and the results are assembled into nine planes which contain the original data padded with zeros. Those nine planes are created along two axes, so `[: ,/` is used to organize the planes along one axis. The new axis of length nine is such that all nine values from the 3 by 3 tessellations appear along that axis.

```
$shift i.3 3
9 5 5
```

```
3 3$<"2 shift i.3 3
```

```
+-----+-----+-----+
|0 1 2 0 0|0 0 0 0 0|0 0 0 0 0|
|3 4 5 0 0|0 1 2 0 0|0 0 0 0 0|
|6 7 8 0 0|3 4 5 0 0|0 1 2 0 0|
|0 0 0 0 0|6 7 8 0 0|3 4 5 0 0|
|0 0 0 0 0|0 0 0 0 0|6 7 8 0 0|
+-----+-----+-----+
|0 0 1 2 0|0 0 0 0 0|0 0 0 0 0|
|0 3 4 5 0|0 0 1 2 0|0 0 0 0 0|
|0 6 7 8 0|0 3 4 5 0|0 0 1 2 0|
|0 0 0 0 0|0 6 7 8 0|0 3 4 5 0|
|0 0 0 0 0|0 0 0 0 0|0 6 7 8 0|
+-----+-----+-----+
|0 0 0 1 2|0 0 0 0 0|0 0 0 0 0|
|0 0 3 4 5|0 0 0 1 2|0 0 0 0 0|
|0 0 6 7 8|0 0 3 4 5|0 0 0 1 2|
|0 0 0 0 0|0 0 6 7 8|0 0 3 4 5|
|0 0 0 0 0|0 0 0 0 0|0 0 6 7 8|
+-----+-----+-----+
```

The function `test` tests whether the centre (plane with index 4) is alive with 4 alive cells among the 9 cells, or if there is a total of 3 alive cells among the 9 in a neighbourhood. While the wording is different, these conditions are equivalent to

those given originally, but this view is used since computing sums of all 9 cells is convenient.

Notice that the 4 in the definition of test is used in two quite different ways when it is spread upon the verb train. It is both an index (plane 4) and a potential sum. Since the same word is used in two distinct ways in the sentence in which it is embedded, I believe Shaw's signature qualifies as humour. It certainly made me laugh out loud.

But his signature is much more than a joke. Table II shows the performance of Life implementations using J 5.03 on a 1.7GHz PC. Cut (l i f e 2) is now wonderfully efficient, but Shaw's signature (l i f e s) is more than ten times faster than that. Moreover, using subsecond updates, we can deal with universes that have more than 10,000 times more cells than in 1995.

n		life1		life2		lifes	
time	space	time	space	time	space	time	space
256	0.2516	534272	0.2243	265152	0.0125	2754688	
1024	4.0536	8423168	3.6037	4203456	0.3145	44042368	

Table II. Performance of Life, Circa 2005.

Time for the Game of Life

In preparing my remarks for [9e] I used a variant of Life that used table look-ups for neighbour sets and configurations that would lead to life. It was competitive with the signature method and both were fast enough to quickly update universes at least 1000 by 1000. I decided it was time to write a GUI version of the Game of Life in J [14]. I eventually grew to prefer the signature method for the style of updating generations, but still preferred the periodic boundary conditions. Slight changes to shift, which we call `shiftv`, give those boundary conditions. We also use a slight variant on the test function.

```
shiftv=:[:,/1&|.^(i:1)@|:"2^:2
```

```
testv=: (4&{((3:=])+. [* .4:=]) +/)
```

```
Lifev=:testv@shiftv
```

Note that I have gone back to my roots. Instead of adjoining zeros, the function `shiftv` rotates planes -1, 0 or 1 units. You might think that I used `Lifev` in my

implementation. Actually not, I used `shiftv@testv` instead of `testv@shiftv`. That is because the colour information requires knowing the number of neighbours each alive cell has and that is the same information needed to update a generation. The expensive part of the computation is the `shiftv` portion; thus, rather than compute it twice, we store all 9 planes as the representation of the universe at any iteration. That is my attempt at humour.

Figure 2 shows the form created by running the *vlife2.ijs* script. The configuration was initially a 64 by 64 random configuration embedded in a 128 by 128 universe. It is shown after 4 and 117 iterations. Notice that even after 4 iterations some organization is apparent. After 117 iterations there is much more organization, the information has expanded, but the random quality is not gone.

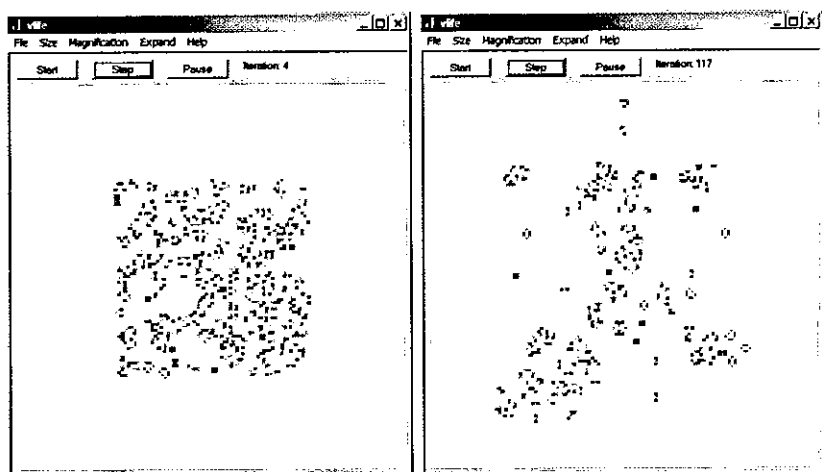


Figure 2. Evolution of a random configuration

On the web one can find many sites with very interesting patterns. A classic collection is Al Hensel's *lifep.zip* collection. These use the Life 1.05 format which may be imported into our life form using the file load menu. Watching the dynamics is far more interesting than viewing static images, but we can nonetheless get some insight into these striking behaviours. Figure 1 showed one stage of the evolution of *psrtrain.lif*. Glider guns such as *gun30.lif* produce streams of gliders; *lwssgun.lif* produces the stream of lightweight space ships seen in Figure 3. Figure 4 shows *spiral.lif* which must be watched to be fully appreciated. Four configurations move away from the centre leaving a small trail of stable configurations that are destroyed by a glider that moves around in a spiral. Figure 5 shows a configuration arising from *max.lif* that grows outward leaving

the interior filled with the green strips. The configuration *zips.lif* shows an empty pattern moving inside strips in a fashion similar to a zipper. The Game of Life offers many surprising and beautiful configurations.



Figure 3. A configuration resulting from *lwssgun.lif*

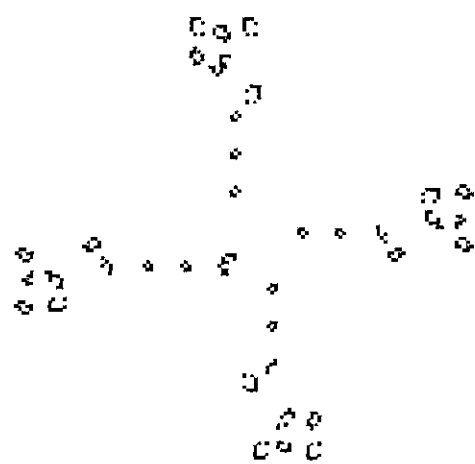


Figure 4. A configuration resulting from *spiral.lif*

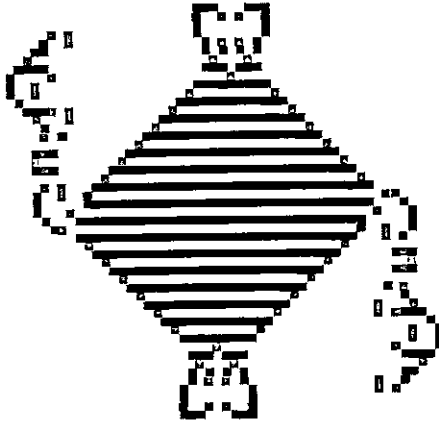


Figure 5. A configuration resulting from *max.lif*

Timer for the Game of Life

In order to implement the Game of Life in a manner that I desired, I wanted "start" and "pause" buttons as can be seen on the forms in Figure 2. Of course, if the "start" button implements an unbounded loop, there is no obvious way for an event to trigger a change that could break the loop. However, the timer event gives the facility required. The "start" button runs one step of life, and sets a timer event to occur later. If some other event happens first, then that event is executed. For example if the "pause" button is pressed, then its event is defined to cause the timer event to be cancelled. If no other event has occurred, then the timer event executes `sys_timer` which is defined to run one more step of life and the next timer event remains pending. This is repeated until the timer is halted by the "pause" button or an error.

```
life_start_button=:3 : 0
life_step_button ''
wd 'timer 100'
)

sys_timer=: 3 : 0
try. life_step_button '' catch.
  wd 'timer 0'
end.
)

life_pause_button=:3 : 0
wd 'timer 0'
)
```

The function `life_step_button` runs one step of life and updates the form. Its details are available in the script [14].

Conclusion

We have seen that the Game of Life can be run very effectively in J. While many details have a new flavour, the fundamental ideas of using infix, cut, or multiple planes are old [10]. Nonetheless, combined with timer events and a colouring scheme, a GUI allows us to investigate the beautiful evolution resulting from the Game of Life.

Cliff Reiter
Department of Mathematics
Lafayette College, Easton, PA, 18042 USA
<http://www.lafayette.edu/~reiterc>

References

- [1] M. Alfonseca, M. de la Cruz, and A. Ortega, Complex Systems in APL: Fractals, Evolving Cellular Automata and Artificial Life, *APL Quote Quad*, 32 4 (2002) 17-26.
- [2] S. Apter, Conway's Game of Life in K, <http://www.nsl.com/papers/life.htm> and <http://www.nsl.com/papers/plife.htm>
- [3] E. Berlekamp, J. Conway, and R. Guy, *Winning Ways For Your Mathematical Plays*, Academic Press, 1982.
- [4] P. Callahan, Patterns, Programs, and Links for Conway's Game of Life, <http://www.radicleye.com/lifepage/lifepage.html#catback>

- [5] P. Callahan, What is the Game of Life?
<http://www.math.com/students/wonders/life/life.html>
 - [6] M. Gardner, The fantastic combinations of John Conway's new solitaire game of "life", *Scientific American* 223, (1970) 120-123.
 - [7] M. Gardner, On cellular automata, self-replication, the Garden of Eden and the game "life", *Scientific American*, 224 4 (1971) 112-117.
 - [8] A. Hensel, Conway's Game of Life, <http://www.ibiblio.org/lifepatterns/>
 - [9] Jforum list server, <http://www.jsoftware.com>. Messages dated (a) June 24, July 6, 2001, (b) Aug 29, 2001, (c) Jan 31-Feb 7, 2003; (d) Aug 20-21, 2003, (e) Dec 12-16, 2004.
 - [10] E. E. McDonnell, Life: Nasty, Brutish, and Short, *APL Quote Quad*, 18 2 (1988) 242-247.
 - [11] J. E. Pulsifer and C. A. Reiter, One Tub, Eight Blocks, Twelve Blinkers and Other Views of Life, *Computers & Graphics*, 20 3(1996) 457-462.
 - [12] C. A. Reiter, Infix, Cut and Finite Automata, *APL Quote Quad*, 25 4 (1995) 162-169
 - [13] C. Reiter, *Fractals, Visualization and J*, 2nd edition. Jsoftware, Inc, Toronto, 2000.
 - [14] C. Reiter, Viewing Color Life,
http://www.lafayette.edu/~reiterc/j/vector/vlife_index.html
 - [15] E. Shaw, J page, <http://www.ewartshaw.co.uk/jwhat.html>
 - [16] J. Summers, Jason's Life Page, <http://entropymine.com/jason/life/>
-

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

APL2000 + Web Component (part 1 of 2)	Eric Lescasse	100
Notes on Virtual Memory	Bill Rutiser	115
Master and Slave: Dyalog APL Directs Excel!	Kai Jäger	125
Conformability to ISO APL Extended Standard	Eke van Batenburg	135
A Skeleton Key to Windows Performance Monitoring	Bill Rutiser	141

APL2000 + Web Component (Part 1 of 2)

by Eric Lescasse (eric@lescasse.com)

Introduction

Do you want to port your APL+Win application to the Web? Or do you want to write an APL+Win application which will be usable on the Web?

It is now feasible with the new APL2000 **APL+WebComponent** product.

What we mean by "deploying your APL+Win application to the Web" is that you'll make it run within the browser and that, once deployed on your Server, users from all around the world may start using it right away: all they will need to use it is a fast Internet connection and Internet Explorer!

Required Software

In order to use APL+WebComponent to deploy your APL+Win application to the Web, you will need the following software:

Windows 2000 Pro or
Windows XP Pro (or
Windows 2003 Server)

It should also be possible to use Windows 98, Me or NT but we have not tested APL+WebComponent on these systems.

Internet Explorer 5 (or 6)

The later version, the better

IIS Internet Information
Services (5 or 6)

You may install IIS only if you run Windows 2000 Pro or Windows XP Pro. It is not available for Windows XP Home. To check if it is installed, click Start/Control Panel/Administrative Tools and check if Internet Information Services is displayed. If not, you can install IIS by running Start/Control Panel/Add or Remove Programs/Add Remove Windows Components: check the Internet Information Services (IIS) check box and proceed.

Microsoft .Net Framework
1.1

If not yet installed on your computer download the Microsoft .Net Framework 1.1 now (free)
(Note: to find this, start Google and search for: "Download Microsoft .Net Framework 1.1")

APL+Win 5.0

Always best to run the latest version, but the one you have will do (to download the latest version you must be an APLDN Subscriber: a login and password is required)

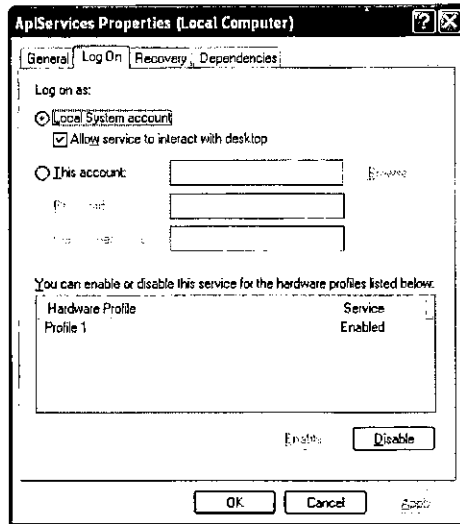
APL+WebServices	APL+Web Services are part of APL+Win 5.0. However you will need the latest version as downloadable from the APL2000 Web Site (you must be an APLDN Subscriber: a login and password is required)
APL+WebComponent	APL+WebComponent are not part of APL+Win 5.0: the latest version of APL+WebComponent may be downloaded from the APL2000 Web Site (you must be an APLDN Subscriber: a login and password is required)
Microsoft Office Web Components v11 (or v10)	Only required for APL+WebComponent (part 2) article. The latest version of this Microsoft spreadsheet object, made for the Web, can be downloaded from the Microsoft Web Site. (Note: to find the link, start Google and search for: "owc11.exe" and "microsoft")
Microsoft Office Web Components Toolpack	Only required for APL+WebComponent (part 2) article and want to learn how to program the OWC Spreadsheet. The latest version of this learning tool can be downloaded from the Microsoft Web Site. (Note: to find the link, start Google and search for: "Office Web Component Toolpack")

Note: most APL+Win 5.0 users should already have the required components installed except maybe the APL+WebComponent. The last 2 items in the list above are only required if you want to run the example workspace in APL+WebComponent (part 2).

Installation

Here are special instructions about installing and configuring some of the above products:

APL+Win 5.0	Even though you may have already installed APL+Win 5.0, you will need to register it as a COM Server. To do so, be sure to run the 2 following commands from Start/Run : <code>regsvr32 "c:\aplwin50\aplwco.dll"</code> <code>c:\aplwin50\aplw.exe /regserver</code>
APL+WebServices	Run the APLServicesSetup.msi installation Once the APL+Web Services are installed, check installation by clicking Start/Run/Services.msc : APLServices should appear in the list of installed services. You need to right click on APLServices , click on Properties and then on the Log On tab and check both the Local System Account radio button and the Allow service to interact with desktop check box. Click OK .



APL+WebComponent

Create a directory called LC under C:\INETPUB\WWWROOT

Create a subdirectory of LC called
C:\INETPUB\WWWROOT\LC\WEBSERVICES

Note: LC and WEBSERVICES are not fixed names but we need to use these names here so that the examples cut & pasted from my machine match.

The following files can be downloaded from the APLDN Web Site (you must be an APLDN Subscriber: a login and password is required):

Download and unzip the JSAVESDKFILES.ZIP file into
C:\INETPUB\WWWROOT\LC\WEBSERVICES

Download and copy the latest JSAVESDK.W3 file into
C:\INETPUB\WWWROOT\LC\WEBSERVICES

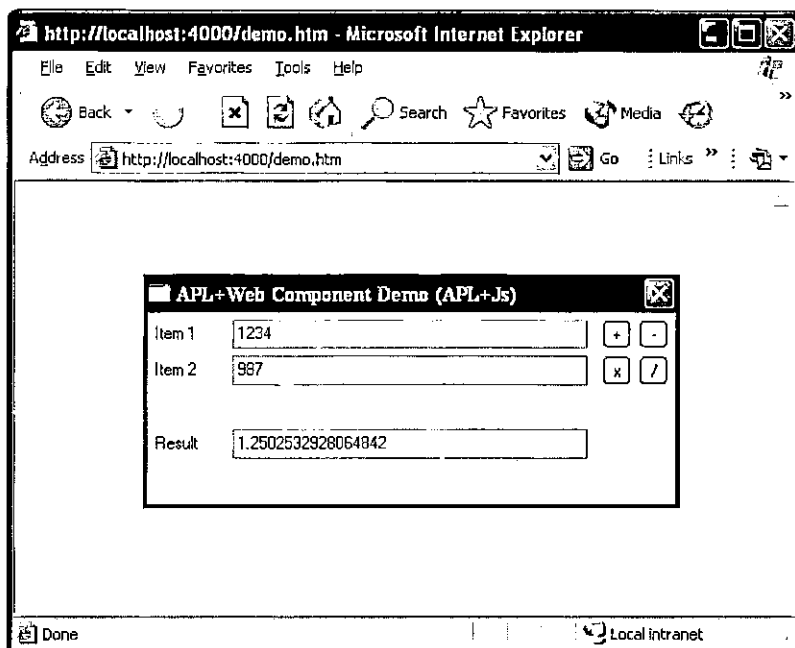
Download and copy the latest APLWS.W3 file into
C:\INETPUB\WWWROOT\LC\WEBSERVICES

Download and copy the latest JSCRIPT_FILES.ZIP file into
C:\INETPUB\WWWROOT\LC\WEBSERVICES

The Sample Application

We will write a very simple APL application and then publish it to the Web. While doing so we will explain in detail how to proceed.

Here is the sample application as it will appear in the browser :



This is a simple calculator capable of computing the 4 basic arithmetic operations on 2 numbers. It is made of a form containing 3 labels (Item 1, Item 2 and Result), 3 Edit controls and 4 buttons.

And here is all the code you need to write for this simple application:

```

      ▽ Main B;2;op1;op2;res;op
[1]
[2]   :select B
[3]   :case''
[4]     A Build the interface
[5]     Z←'fmMain'□wi>Create' 'Form'('border'1 16)'Hide'
[6]     Z←'fmMain'□wi'caption'('APL+Web Component Demo ('□sysid,')')
[7]     Z←'fmMain.1Op1'□wi>Create' 'Label'('where'(8 5 15 50÷16 8 16 8))
      ('caption' 'Item 1')
[8]     Z←'fmMain.edOp1'□wi>Create' 'Edit'('where'(5 60 21 250÷16 8 16 8))

```

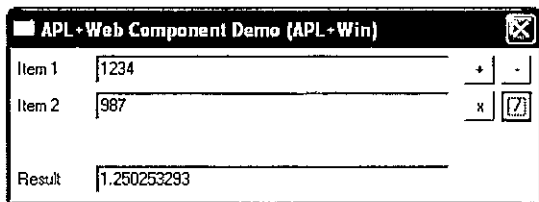
```

('text' '')
[9]  Z+fmMain.10p2'wi>Create' 'Label'('where'(34 5 15 50÷16 8 16 8))
('caption' 'Item 2')
[10] Z+fmMain.edOp2'wi>Create' 'Edit'('where'(31 60 21 250÷16 8 16 8))
('text' '')
[11] Z+fmMain.1Res'wi>Create' 'Label'('where'(86 5 15 50÷16 8 16 8))
('caption' 'Result')
[12] Z+fmMain.edRes'wi>Create' 'Edit'('where'(83 60 21 250÷16 8 16 8))
('text' '')
[13] Z+fmMain.bnAdd'wi>Create' 'Button'('where'(5 320 21 21÷16 8 16 8))
('caption' '+')('onClick' 'Main"bn.onClick"')
[14] Z+fmMain.bnSub'wi>Create' 'Button'('where'(5 346 21 21÷16 8 16 8))
('caption' '-')('onClick' 'Main"bn.onClick"')
[15] Z+fmMain.bnTim'wi>Create' 'Button'('where'(31 320 21 21÷16 8 16 8))
('caption' 'x')('onClick' 'Main"bn.onClick"')
[16] Z+fmMain.bnDiv'wi>Create' 'Button'('where'(31 346 21 21÷16 8 16 8))
('caption' '/')('onClick' 'Main"bn.onClick"')
[17] Z+fmMain'wi'size'((83+21+5)(346+21+5)÷16 8)
[18] Z+fmMain'wi>Show'
[19] :case'bn.onClick'
[20]   A Buttons handler
[21]   op1+fi'fmMain.edOp1'wi'text'
[22]   op2+fi'fmMain.edOp2'wi'text'
[23]   op+fi'caption'
[24]   :select op
[25]   :case+'  ¢ res+op1+op2
[26]   :case-'  ¢ res+op1-op2
[27]   :case'x' ¢ res+op1×op2
[28]   :case '/' ¢ res+op1÷op2
[29]   :end
[30]   'fmMain.edRes'wi'text'($res)
[31] :end
[32]
v

```

If we called this application from the APL+Win Session, here is how it would look:

Main''



Notes about the APL code :

1. When an empty vector is passed as an argument to Main, then we fall into :case" and lines 4 to 18 are executed. These lines build the application interface.

2. When the user clicks one of the 4 buttons, the buttons handler **Main'bn.onClick'** is called. In order to keep the whole application within the scope of one APL function, we have embedded this handler within the Main function which we call with an argument of 'bn.onClick' and the code from line 20 to 30 gets executed. The caption of the clicked button is retrieved and we use the :select :case control structure to execute the right operation depending on the caption found

Step by step Setup

Here is the step by step procedure you need to follow to deploy this simple APL application on the Web:

Preparing the workspace

1. Load the C:\INETPUB\WWWROOT\LC\WEBSERVICES\APLWS.W3 workspace
2. Rename it C:\INETPUB\WWWROOT\LC\WEBSERVICES\DEMO.W3
3. Copy the **Main** function displayed above into this workspace
4. Add an **AutoStart** function reading as follows:

```

      ▽ AutoStart
[1]   Main''
      ▽

```

5. Set the workspace □LX to be AutoStart

```
□lx←'AutoStart'
```

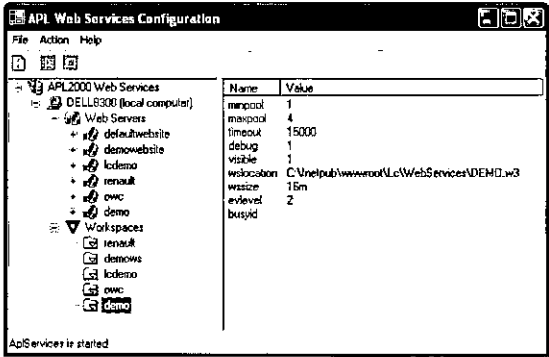
6. Save the workspace

Setting up the APL+WebService

Next you need to configure an APL+Web Server for this application. Proceed as follows:

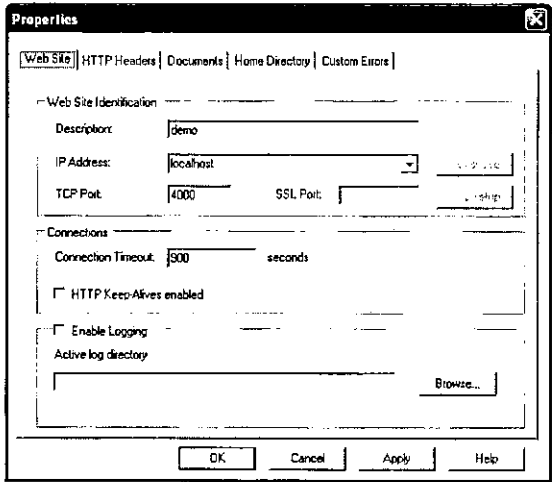
1. Click **Start/All Programs/APL2000 Web Services/AWS Admin** to start the APL Web Services Configuration console
2. Right click on **Workspaces** and select **New Workspace** to add your application
A new workspace called **defaultworkspace** is created
3. Right click this **defaultworkspace**, select **Rename** and rename it **demo**

4. Click the **demo** workspace and then set the various values in the right hand pane to match the ones in the following screen shot:

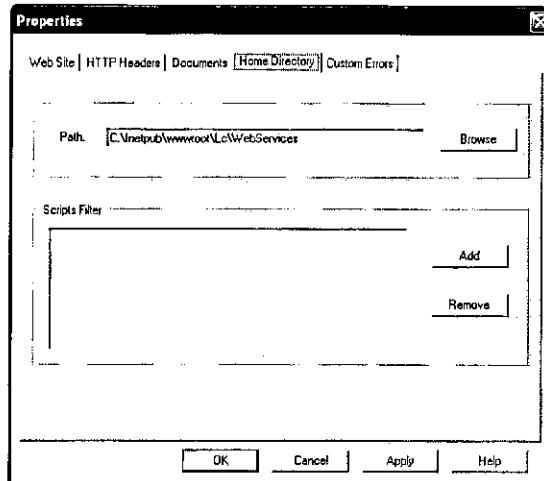


Note: as soon as you have defined the **wslocation** to be your **DEMO.W3** workspace, an **APL+Win ActiveX Server** session starts with your workspace loaded!

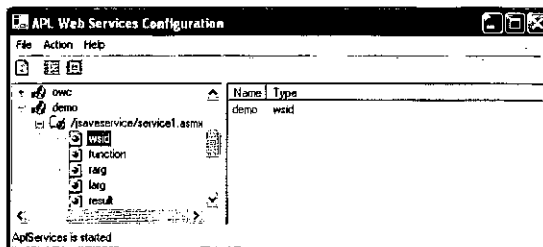
5. now right click on **Web Servers** and select **New Server** to create a new **APL+Web Server**. A new server called **defaultwebsite1** is created
6. right click on this **defaultwebsite1** server and select **Properties** the **Properties** dialog box gets displayed: fill its first tab as follows:

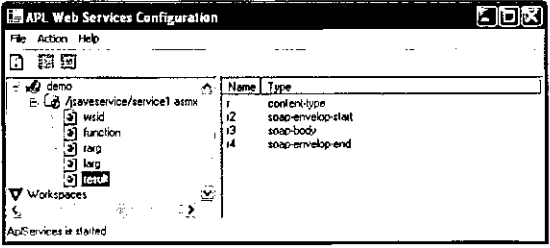
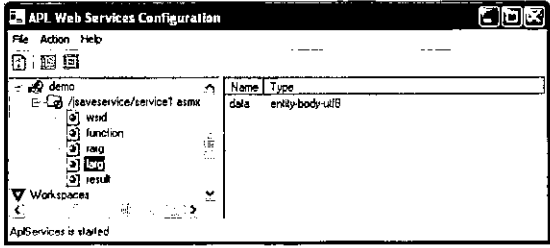
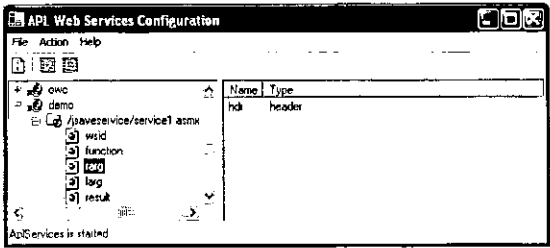
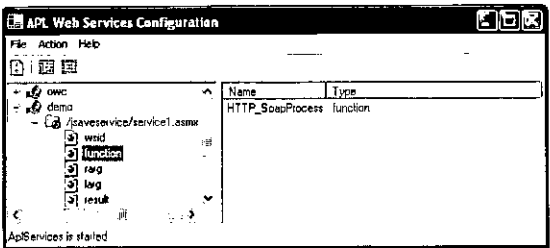


7. click the **Home Directory** tab and fill it as follows:



8. click the **Apply** button and then the **OK** button to close the dialog
9. next right click on the **demo** Web Server and select **New Virtual Directory**
a new virtual directory called **/default** is created (click on the + sign next to **demo** to see it)
10. right click this **/default** virtual directory , select **Rename** and rename it to:
/jsaveservice/service1.asmx
Note: this **/jsaveservice/service1.asmx** name is fixed and reserved for
APL+Web Component
11. next click or right click on the items below **/jsaveservice/service1.asmx** and set
their properties in the right hand pane to be the same as the ones displayed in
the following screen shots:





12. last but not least, you need to start your new demo APL+Web Server
To do that click on the demo Web Server once and then click the Start
Srvtoolbar button

The toolbar buttons change from    to    ■

Note: also, be sure that the console Status Bar displays **AplServices is started**. If it's not the case click **Start/Run** and run **Services.msc** then right click **APL Services** and start them.

Publishing the workspace with JSaveSDK

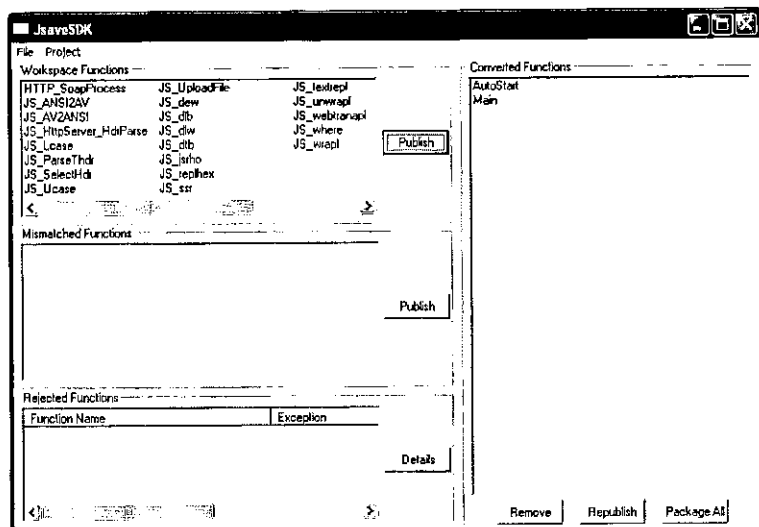
In order for the browser to be able to display your workspace, you now need to publish the functions which represent its interface, using the **JSaveSDK** software. The process of publishing a workspace is simple: it consists of selecting the APL functions of your workspace which will run on the Client side of the application (i.e. in the browser). These APL functions are automatically translated from APL to JScript so that they can run in the browser when you run the application. This translation is "transparent" for you: you never see JScript code and in fact, you could deploy APL applications to the Web without even knowing that this translation has occurred. I just explain this for the sake of understanding **JSaveSDK**.

But this translation has some implications, one of which being that JScript is much slower than APL and therefore you will generally want to limit as much as possible the amount of APL code that you want to publish.

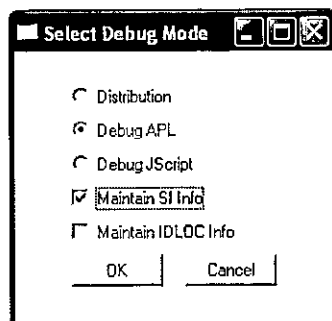
All the rest of your application code will run on the Server Side of the application. We will explain this in much more detail in the **APL+Web Component (part 2)** page. For now, just remember that almost everything related to your application Interface needs to be published. For our small application, everything will run on the Client side of the browser and therefore we will publish the only 2 functions we have written so far: **AutoStart** and **Main**.

1. Start another APL
2. Load the **C:\INETPUB\WWWROOT\LC\WEBSERVICES\JSAVEDSK.W3** workspace. The **JSAVEDSK** window pops up
3. Click **Project/Import Workspace** and select your **C:\INETPUB\WWWROOT\LC\WEBSERVICES\DEMO.W3** application workspace. The functions contained in this workspace get displayed
4. Click on **AutoStart** and then **Ctrl+Click** on **Main** to select these 2 functions

5. Click the Publish button. What you should see so far is:

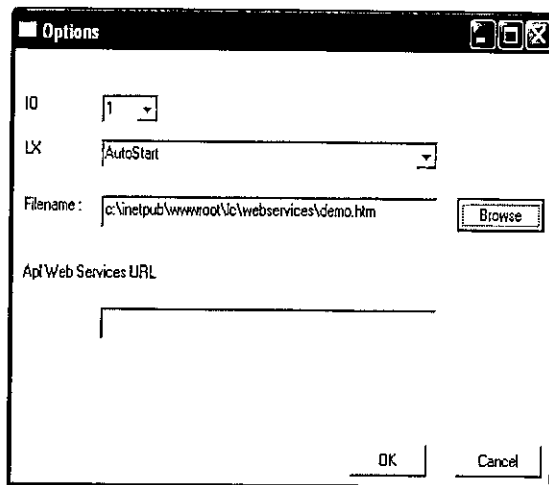


6. The Select Dialog Mode window pops up
Check the Maintain SI Info check box in the following dialog then click OK



7. Then select File/Save and save the JSaveSDK Project as
C:\INETPUB\WWWROOT\LC\WEBSERVICES\DEMO.WJS

8. Finally click the Package All button and fill the next dialog as follows:



Congratulations! You have successfully configured your first APL+Web Component application. Yes it is a little long and cumbersome the first time, but the good news are that you must only do this setup once for a given application and that you quickly get used to it. Also the setup process described above is the same whether for a small APL application or for a very large one.

Testing the application

It is now time to test our application.

As a good habit, it is always recommended to test it first in standard APL+Win mode. For that you should use the APL+Win ActiveX Server session where your DEMO workspace is loaded.

Check that your application is working fine in APL and correct any bug you may encounter. Then save the workspace.

The fact that your application runs fine in APL does not mean that it will necessarily run smoothly in the browser: there are many reasons for that which we explain in the APL+WebComponent (part 2) article. One of the reasons is that JSaveSDK is not able to translate all APL primitives, operators and constructs in JScript.

For example, when developing the Main function I had first written my buttons handler as follows:

```
[19] :case'bn.onClick'
[20]   A Buttons handler
[21]   op1←t⊞fi'fmMain.edOp1'⊞wi'text'
[22]   op2←t⊞fi'fmMain.edOp2'⊞wi'text'
[24]   :select t⊞wi'caption'
[25]   :case'+' ⊞ res←op1+op2
[26]   :case'-' ⊞ res←op1-op2
[27]   :case'x' ⊞ res←op1×op2
[28]   :case'/' ⊞ res←op1÷op2
[29]   :end
[30]   'fmMain.edRes'⊞wi'text'(⌞res)
```

Everything was running fine when testing in standard APL mode, but the buttons would not operate when running the application in the browser. I have had to split line 24 into 2 lines as follows to correct the problem:

```
[19] :case'bn.onClick'
[20]   A Buttons handler
[21]   op1←t⊞fi'fmMain.edOp1'⊞wi'text'
[22]   op2←t⊞fi'fmMain.edOp2'⊞wi'text'
[23]   op←t⊞wi'caption'
[24]   :select op
[25]   :case'+' ⊞ res←op1+op2
[26]   :case'-' ⊞ res←op1-op2
[27]   :case'x' ⊞ res←op1×op2
[28]   :case'/' ⊞ res←op1÷op2
[29]   :end
[30]   'fmMain.edRes'⊞wi'text'(⌞res)
```

This is just one example of the many things which may happen when publishing your functions. The bad news are that, when something like that happens, you do not get any precise error message pointing you to the culprit line. As a matter of fact in this case, I did not get any error message at all: the buttons would just not operate.

The good news are that, with a little habit, and some time spent working with this new technology, you most often quickly guess what's going wrong. The other good news are that these little debugging occur only with the functions you are publishing on the Client side, not at all with the ones you run on the Server: these latter ones use the full APL syntax and debugging on the Server side is easy.

In any case it is best to process as follows:

- write a few lines of code (or sometimes just one line)
- test in standard APL mode and debug
- republish your functions
- test in the browser

If it does not run fine in the browser, and if it was working fine the previous time, you know the problem occurs in the line (or small bunch of lines) you have added. This development method may seem slower, but in fact gets you more quickly to a running application in the browser.

Also remember that every time you make the smallest change to one of the APL functions which you have selected to publish in JSavedSDK, you **MUST Save** the workspace and then **Republish** with JSavedSDK. If you don't do so your change will not be taken in account when testing your application in the browser.

Running the APL application in the browser

We are at last ready to run our APL application in the browser.

Start an instance of Internet Explorer and enter the following URL:

```
http://localhost:4000/demo.htm
```

Here are some explanations:

- we use **localhost** to mean that we want to test our application on this current machine (not on a remote server)
also remember that **localhost** is what we entered in Setting up the APL+Web Service step 6 above
- we specify that we call on the **4000** port
according to the APL+Web Service Step 6 setup (see above), this Web Server port is associated with the following Home Directory :
C:\INETPUB\WWWROOT\LC\WEBSERVICES
- then we specify the **demo.htm** page
The Web Server associated with port 4000 therefore fetches the
C:\INETPUB\WWWROOT\LC\WEBSERVICES\DEMO.HTM page and loads it. This page is the page built by JSavedSDK when we published the application.

Conclusion

In this first article on APL+Web Component we have showed the basics which should allow you to publish a simple APL application on the Web.

However, when deploying a real life application, things gets a bit more complicated: in APL+Web Component (part 2) we go much deeper into this new technology through a more complex example involving:

- using the Microsoft OWC Spreadsheet object
- running APL functions on the Server
- deciding what to run on the Client and what on the Server
- debugging
- transferring data from the Server to the Client using XML
- publishing your application on a Real Server

If you succeed in publishing a small APL application on the Web after reading this article, please send me an E-mail. Also, if I have omitted anything important in my Description above and if you get stuck, please tell me.

Notes on Virtual Memory and the APL Workspace

by Bill Rutiser (from a talk given at the 2003 APL2000 Conference)

A Little History

1950s

Machines were logically small but physically big. They were very expensive to buy and operate. Programs had full access to the entire machine. Programmers were responsible for all details of a program's operation. Machine time was scheduled and paid for in blocks.

1960s

Simple operating systems mediated I/O operations and transitions from job to job. Turn-around times under batch or job stream operation were measured in hours or days. Experimental time-sharing systems provided more or less immediate response to users.

1970s

Advances in hardware technology reduced the cost and size of computer hardware, which permitted logically larger mainframe systems and physically smaller mini-computers. Operating systems supported multi-programming. Several jobs could run more or less concurrently.

Emphasis was on increasing utilization of CPU by overlapping one program's computation with another program's I/O transfers. Access to and responsibility for I/O and storage management moved from programs to the operating system.

General and special purpose "time sharing" systems were in use but expensive.

1980s

Microprocessor technology enabled desktop and tower computers capable of supporting several users. Manufacturers of small Unix based systems temporarily proliferated. Hardware became cheap enough that personal computers were economically possible. Concerns about response time replaced concerns about hardware utilization.

Unix based systems directly supported concurrent operation of multiple programs with OS mediated I/O and resource management. Personal computer operating systems provided little more than support for loading programs and abstracting the lowest levels of hardware differences.

Local area networks came into use. Business applications distributed over multiple small networked machines became common.

1990s

PC processor speed and memory capacity increased while costs decreased. Graphical user interfaces, multiple concurrent applications, and larger disk storage capacity became increasingly common.

The Internet and the World Wide Web became widely accessible.

Business application architecture increasingly involved multiple programs running on multiple computers in multiple tiers tied together by multiple networks.

2000s

Personal computers have become large enough that users may have several large application programs open simultaneously. Workflow causes the user's attention to shift from application to application.

Evolution of Computer Memory Addressing

Some concepts, definitions, and over-simplifications

Computer memory stores bits. Groups of bits can represent numbers, characters, or other things. Modern hardware generally provides direct support for groups of 8, 16, 32, and 64 bits. The hardware also supports certain data representations such as 8 bit characters, 16 or 32 bit integers, and 32 or 64 bit floating point numbers.

At the hardware level, machine instructions contain numbers that designate particular groups of stored bits. Such a number is a "memory address". Almost all present day computers use "byte addresses" that designate a group of 8 bits. Depending on context, a byte address may refer to a single byte, or a group of adjacent bytes.

Simple direct addressing

Memory is treated like a simple APL array. An address is simply an index into the array. There is only one such array and it is the entire memory.

Early programming required a programmer to explicitly associate data and memory addresses. One of the first uses of programming tools was to assign addresses to symbolic names.

Memory protection

Practical use of multi-programming is much simplified by hardware support to ensure one program does not inadvertently modify another program's memory. Numerous schemes have been used. All require that the hardware registers controlling the protection are accessible only to the operating system.

This, together with other considerations, leads to a distinction between system and user modes of processor operation.

Based and segmented addressing schemes

Multi-programming requires a means to assign a distinct range of memory addresses to each program. The simplest scheme is to adjust all the addresses used by a program when it is loaded. This is time consuming and requires the program itself to observe appropriate conventions.

Simple hardware support adds a special "base register" to the CPU. The content of this register is automatically added to each memory address as it is used. The program can now be written to always use a range of addresses beginning at zero. The operating system sets the base register to the actual memory address of the segment of memory allocated to the currently running program.

Memory protection can be provided via a "limit register" setting the upper limit of allowed access. Base and limit registers are changed together.

Memory swapping and time sharing

Observe that the introduction of the base and limit register permits an operating system to dynamically change the actual memory assigned to a running program:

- Suspend the program's execution;
- Move all of its memory contents to a different base address;
- Change the base and limit registers; resume the program's execution.

While a program is suspended, the contents of its memory can be written to auxiliary storage. Another program's memory contents could be swapped into the vacated memory and run.

Some time-sharing systems worked this way. Response times were determined by the relative size of the memory and speed of access to the swap storage.

Paging

Simple swapping schemes read in all of a program's memory before continuing the program's execution.

Most programs exhibit some locality of reference such that the recently used parts of its memory change slowly relative to the progress of execution. This locality arises because: instructions are executed in sequence with only occasional jumps to non-sequential addresses; a similar sequence of instructions is repeated in each traversal of a loop; related data is often stored at related addresses.

Paged memory systems divide memory into pieces of uniform size, commonly several thousand bytes. Such a piece of actual physical memory, either processor memory or auxiliary memory, is called a page-frame; the data itself is called a page. This distinction between pages and page-frames is often elided.

Hardware support for paged memory usually provides several special bits for each memory page-frame.

- One bit marks the frame's contents as invalid. Any access to an invalid page raises an exception, called a page-fault, which is trapped by the OS. When a program is to be swapped into memory, the OS assigns a set of page-frames, marks them as invalid, and resumes the program's execution. When the program accesses a marked page, the OS responds to the page-fault by suspending the program, reading the page into the page-frame, and then continuing the program's execution.
- A second bit is set, automatically, by the processor, whenever a page-frame's content changes. The OS resets this bit each time a page is read in. If the bit is still reset when the page is to be swapped out, the OS knows the page has not changed since last being paged in and, if the copy in the disk swap area has not been changed, the OS can skip writing out the page.
- A third bit is set by each reference to the page-frame. By periodically resetting these "reference" bits, an OS can determine whether a page has recently been used.

Memory mapping and address translation

Paging as described above could be used to support segmented memory systems. However, modern processors contain yet one more piece of hardware support.

A special set of tables, maintained by the operating system, is used to map each address used by a program to a particular page-frame. The only relation between the address used by the program and the physical address of the page-frame is the number in the table.

Conceptually, at least, each address used by a program is broken into two parts. The high-order part is used as an index into an OS maintained table of page-frame addresses. The low-order part is used as an offset from the beginning of that page-frame. The processor hardware automatically performs this address translation for each access.

The hardware maintains a cache of recently translated page addresses so as to bypass most actual accesses of the page-mapping table. The potentially large tables, one for each process, are themselves stored in paged memory subject to swapping.

Some Virtual Memory Terms

“Virtual memory” is the apparent simple directly addressed memory used by application programs. An address used to designate locations in virtual memory is a “virtual address”. Some writers confusingly and incorrectly use “Virtual Memory” to refer to swap space or paging file as defined below.

“Real memory” is physical hardware memory, most of which is used as page-frames for mapping virtual memory. The rest contains OS kernel programs and data.

“Swap space” is an area of disk storage used as page-frames to contain pages that are paged out. “Paging file” is synonymous. “Paging” is the I/O activity used in transferring pages between swap space and real memory.

“Virtual address space” has several closely related meanings:

- The potential range of addresses that could be used by a program running in a particular environment;
- The actual virtual addresses and data in use by a program.

A "process" is an OS defined abstraction, consisting of a "virtual address space", a current point of program execution, and assorted other resources such as open files, synchronizing locks, permissions, identities, etc.

"Thrashing" describes a state of sustained high paging with little useful work done between page faults. Thrashing arises when a process or set of processes repeatedly and rapidly touches more pages than can simultaneously be allocated page-frames. Thrashing has been called an "unproductive flutter of pages". Although an OS can sometimes detect thrashing and suppress it by adjusting the mix of active processes, an OS cannot prevent thrashing entirely caused by a single process actively using more pages than the machine has page-frames.

Virtual Memory in Windows

The Intel IA32 architecture defines a 32-bit address space. This means that the processor family's instruction and data formats have room for 32 bit addresses. Most actual computers have considerably less than 4 gigabytes of real memory but such a machine is no longer impossible.

Workstation versions of Windows NT divide the 32-bit address space into two equal parts. "System memory" is reserved for the use of NT itself and "user memory" is mostly available for the application process. Note that some small areas of user memory are also reserved, mostly for compatibility reasons, and that some process specific data is stored in user memory. In addition, data allocated by NT on behalf of the application is in user memory.

NT organizes virtual memory into 4k pages. 64K segments are also used for allocation purposes.

NT virtual memory allocation

Addresses are allocated separately from memory. Initially all addresses in the user space are free and associated with no data.

The WIN32 API contains a pair of functions, "VirtualAlloc" and "VirtualFree". In one mode, these functions reserve and release regions of address space. Such regions are specified by size in multiples of 64K. Reservation of a region does not consume significant physical resources; it is simply an entry in the system's tables for the process. Releasing a region returns its addresses to free status.

In a second mode, "VirtualAlloc" and "VirtualFree" commit and de-commit specific pages within reserved address space. Committing a page allocates a page-frame in the paging file and adds an appropriate entry to the address translation

tables. The first use of the newly committed page will cause a memory page-frame to be assigned. Subsequently the page will be paged out and in as necessary. De-committing a page discards its data and makes its memory and page-file frames available for other uses.

A process can have a large contiguous region of its address space reserved but only a few pages committed. A process can have multiple contiguous (but not overlapping) reserved regions of various sizes for various purposes.

Like other systems, NT can map one page frame into several address spaces, possibly at different page addresses. This permits several processes to share one physical copy of read-only data or executable code. Page-frames are also used to buffer and cache disk files.

APL+Win and Virtual Memory

APL and memory management

An APL interpreter is essentially an exercise in effective management of dynamically allocated memory. Several aspects of APL as a language contribute.

- Primitive APL functions produce new values. Few APL constructs directly modify existing data objects. Instead newly created arrays replace existing arrays.
- The dimensions and representation of APL data is not readily predictable until the actual execution of an expression.
- The arrays generated by an APL program may range in size over eight or more orders of magnitude.
- Temporary objects half the size of a computers memory easily and usefully arise.
- The lifetimes of arrays and other objects overlap and are unpredictable.

Memory management for large pools of small objects has been well studied by computer science and is fairly well understood. There is little literature on the management of rapidly changing pools of large objects.

Historically, APL programmers have been hampered by fixed size workspaces. APL\360 based time-sharing systems swapped active workspaces between fixed size memory and disk slots. Maximum workspace size was determined by a compromise between memory costs and response time.

Memory Management

The memory manager in an APL interpreter begins with a large, usually contiguous, area of unused memory.

The manager maintains an inventory of free pieces of memory.

Allocation requests are satisfied by selecting an entire piece or by subdividing a larger piece into free and allocated parts.

Eventually, an initial contiguous piece of free memory will have become subdivided into many smaller pieces, some free and some allocated.

Fragmentation and compaction

Depending on the statistical distribution of allocation sizes and lifetimes, the distribution of sizes in the free piece inventory may reach a stable state or the inventory may fragment into unusable small pieces. A memory manager's policies for choosing free blocks for allocation and splitting affect the rate of fragmentation but no policy can always avoid fragmentation.

Fragmentation is overcome by moving allocated objects so that the free pieces are adjacent and coalesced into larger pieces. Such compaction is potentially costly since it may require moving most of the objects in the workspace. In addition, when objects contain addresses of other objects, these pointers must be adjusted when the target object is relocated. A compacting memory manager may compact preemptively or only as needed.

The APL+Win Interpreter

The general architecture of the APL+Win interpreter originated in 1982 as an interpreter for a Unix based workstation with 1Mbyte of RAM and a 20Mbyte hard drive. That machine also had serial ports connected to terminals for a team of five developers.

The UNIX virtual memory model was quite simple.

- Each process had an address space that grew upward from zero and down from a very high address. In NT terms, memory was committed for an upwardly growing bottom part and a stack that grew down from the top of the address space.
- A new process began with its program loaded at the bottom of the address space together with a small amount of per process data. The program extended the bottom part upwardly as needed.
- Most programs used the malloc() routines from the C library to manage smallish allocations.

Our design allocated a fixed size workspace "slot" just above the program. The workspace size could be increased or decreased by freeing and reallocating at a different size. Generally there was little need to change size. Users were advised to keep workspaces smaller than the machine size reduced by system overhead. The kernels paging machinery gracefully handled switching to another process.

Over the next two decades, the interpreter was readily ported to many other systems as diverse as DOS, VMS, and Windows.

Changes

Release 4.0 and before

The memory manager policies are intended for environments where hardware and operating system keep all of the active workspace in real memory while APL is active. The memory manager's policies treat all parts of the active workspace uniformly. Its compaction strategy is intended to minimize CPU time by only compacting when necessary. There is also a periodic compaction to mitigate the effect on allocation of large numbers of scraps.

The implementation and its policies are largely unchanged from those in the early UNIX APL*PLUS products.

Release 5.0

While the memory manager has been entirely rewritten, to take advantage of modern system programming tools, and to prepare it for future changes, most of it is similar in effect to the old memory manager. When the interpreter is started, it asks the NT kernel to reserve a large contiguous region of the process's address space. As explained above, this reservation does not use significant actual memory. This region is referred to as the slot.

As the needs of the workspace change, memory is committed and de-committed at each end of the slot. The SI stack is maintained at the bottom of the slot while data and functions are placed in the upper committed area. Workspace compaction moves allocated objects upwards leaving a contiguous area of unused but committed storage.

The manager is controlled by several policies:

- One decides when compactions are worthwhile.
- Another decides when to increase or reduce the size of the upper committed area and by how much.

- A third controls the expansion and contraction of the lower committed area.

These policies will be adjusted to balance the use of committed memory against the CPU cost of too frequent compactions. Indeed, they have changed a lot during the last year.

The intent is that in most situations, a default slot size, proportional to the amount of real memory in the machine, will be appropriate. The chosen default is 75% or the machines apparent real memory.

Some Questions

Can I make the slot size larger than my machine's real memory and have a really big active WS?

Perhaps, but its very unlikely that you will be at all happy with the result. Paging transfer times are measured in milliseconds. Machine instruction times are measured in nanoseconds. That's a difference of six orders of magnitude. Your disk drive is likely to wear out before your program completes execution.

Why would I ever want to make the slot size smaller?

APL shares the address space with any in-process ActiveX objects it is using. If an object should require a very large part of the address space or needs most of your machines memory to avoid thrashing, then you may need to adjust the slot size to balance the conflicting needs. Note that in this context "very large" means 100's of megabytes or more.

Can I change the slot size with the)CLEAR command?

No. No need has been demonstrated for such a facility. The eventual release system will be arranged to default to a generally useful slot size without additional parameters.

How do I specify the slot size?

With the initial builds, use a command line parameter or set "[Config]Wssize" in your ".ini" file. In the future, the interpreter will default to a generally useful slot size and will have means to specify the slot size as a percentage of real memory.

Master and Slave: Dyalog APL Directs Excel!

by Kai Jäger

Introduction

As APLers, using an application from “Microsoft’s Office Suite” like Excel or Word as an OLE object makes sense in some cases. For example, printing under Windows can be a real nightmare!

Because APL has a great strength in dealing with tables, one often wishes to print them. Since Excel offers great features and flexibility for doing this, why should we not use it for this task? One possible way to do this is to use the Excel OLE object from within APL.

If you can’t beat them, join them!

This article will look into how to: start Excel; load a specified spreadsheet; get some or all of the data into (or out of) your workspace; and use Excel as a printing machine from APL. Also, to try to make this useful for APLers with no or little experience of using OLE objects, I will show some typical techniques to get information about a specific OLE objects.

This article is not addressed to experienced programmers, and doesn’t assume you are familiar with MSDN and Visual Studio.

A typical example of what we want to do is to call a command (say via a context menu) that exports selected rows from a grid in an APL GUI application out to Excel.

Starting Excel

The first thing to do is to start Excel as an OLEclient from APL:

```
'MyOleObj'←WC'OLEClient' 'Excel.Application'
```

If Excel was already running on your PC, that’s fine. If Excel was not already running, nothing will seem to happen, but Excel will now be running in the background. This is because the Excel GUI is not displayed by default. We can expressly ask Excel to display:

```
MyOleObj.Visible←1
```

Now let's set the Caption of the Excel main window:

```
MyOleObj.Caption+ 'APL and Excel'
```

Next we need to create a brand new workbook, since without such a workbook, nothing could happen in Excel:

```
'MyOleObj.MyWorkbook' = Wb MyOleObj.Workbooks
```

"MyWorkbook" need not necessarily be a child of "MyOleObj", since there is no hierarchical dependency — but why not?

Workbooks and Templates

Do you wish to use a specific template? You probably know templates from Word (the *.dot-files); in Excel a corresponding mechanism is also available. An Excel template is a document with the file extension "xlt". If your template is not held in Excel's default template directory, make sure you specify the full path name:

```
#.MySheets+MyOleObj.MyWorkbook.Open 'Standard.xlt'
```

If you are not interested in using a template, use the method "Add" instead:

```
#.MySheets+MyOleObj.MyWorkbook.Add 0
```

Worksheets

We can deal now with the newly-created workbook, which by default contains 3 sheets. Let's create a new object that represents one of the sheets in our workbook. Because we have just created this workbook, it is by definition the active one, so we can deal with the "ActiveSheet":

```
#.MySheet+#.MySheets.ActiveSheet
```

OK, that's fine so far. For the next step we will insert some data and column titles into the sheet. First let's create two variables, one holding the column titles, the other one is needed to represent our "table":

```
ColTitles+ 'The Very First Lastname' 'Firstname' 'Street' 'Code' 'Town'
Values+0 5p''
Values;+ 'Jäger' 'Kai' 'Thisstreet 1' 9999 'Nuremberg'
Values;+ 'Taylor' 'Stephen' 'Thatstreet 2' 123456 'London'
```

The next step will be to insert the column titles into our sheet. We will need to specify starting and ending points. The starting point is (1,1), and the end point is

(1,5) since we have 5 columns. (Excel works in origin 1). What is a bit strange is that the two points defining the range are "Objects" of type "Range".

Visual Basic and Why it is Helpful

Let's introduce now a good way to get information about OLE objects, methods and properties. Make Excel the topmost window and then press Alt+F11. This should open the Visual Basic for Applications IDE. If not, it is perhaps not installed as part of Office on your computer; but you can easily install this stuff.

Now you have access to the Visual Basic Help, a valuable source about the OLE object model of Excel! For example, enter the word "Range" into the "Search help" field. Then select "Range (List)" (or similar) from the list.

Syntax Problems: Default Methods...

In the Help window you will find valuable information on how to deal with "Range". If you look around the help information, you will find statements similar to:

```
Worksheets(1).Cells(1, 1).Value = 24
```

The problem is that this won't work in APL as it stands! This is because in Visual Basic, for example, some objects have a "default method". If a named object with a default method (for example "Worksheets") is referenced without specifying a method, the default method is called. There is no such mechanism available in APL, so we need to call the "default method" explicitly. The "default method" for Worksheets is "Item", so the statement translated into APL would be:

```
((MyOleObj.Worksheets.Item 1).Cells.Item 1 1).Value2 +24
```

Note that here instead of "Value" the name "Value2" is used – we will come back to this later!

Arranging a Range

Let's define two objects of type Range, useful as start and end points:

```
StartPoint←MySheet.Cells.Item(1 1)  
EndPoint←MySheet.Cells.Item(1,ρColTitles)
```

Accessing Data: Methods and Properties

Now we can reference "Value", but be careful: it is not a property, it is a method. This method does not use an argument (in documentation it is specified as "void"). From APL a void argument is supplied as an empty numeric vector:

```
(MySheet.Range(StartPoint EndPoint)).Value 0
24 [Null] [Null] [Null] [Null]
```

So we get back 24, together with some NULLs!

With the statement:

```
(MySheet.Range(StartPoint EndPoint)).Value2+ColTitles
```

we can assign the 5 cells in the first row in one single step.

You can check the cells by calling the appropriate method "Value", but don't forget the right argument; otherwise you get a pointer to the method instead of the contents of the cells:

```
(MySheet.Range StartPoint EndPoint).Value 0
```

As you may have noticed, here is another difference between Visual Basic and APL! In Visual Basic you use "Value" for both, referencing and specifying the contents of a certain cell. In APL, you need to call the method "Value" with a right argument to reference the cell contents, while you need to specify the "Value2" property to change the cell contents.

Specifying Columns and Data

Let's insert the real data into the Excel sheet now. Again we need a "start point" and an "end point". The start point is (2,1), the end point is 1 0 + the number of rows and cells.

```
StartPoint+MySheet.Cells.Item(2,1)
EndPoint+MySheet.Cells.Item(1 0+pValues)
```

And now we can set the real data:

```
(MySheet.Range(StartPoint EndPoint)).Value2+Values
```

Changing Column Width

Currently all the columns are of the standard width. We should now tell Excel that we want the column width to fit the data. "Columns.AutoFit" is a method of the "Range" object, and it is therefore no surprise, that only the contents of the cells within that range are taken into consideration. This means that executing the statement:

```
(MySheet.Range(StartPoint EndPoint)).Columns.AutoFit
```

ignores the width of the cells in the first row, since this row is not part of the "Range" object we are currently using. Furthermore, (with some versions of Excel) an annoying result is printed in the session manager. Fortunately, both problems could be solved relatively easily. Instead of our own "Range" object, we can use the built-in object "UsedRange". This "Range" includes at least all the cells that contain a value. "At least" because maybe there are some empty rows or columns embedded within the data, but this is unimportant here,

To suppress any annoying result, we can simply use an empty dynamic function:

```
{ }MySheet.UsedRange.Columns.AutoFit
```

That's it! Easy, isn't it?

Suppressing Useless Warnings

Sometimes, Excel displays some really annoying dialogue boxes. A typical example is the "This macro has an invalid signature..." message box. If you need to make sure that no such dialogue box interrupts you, simply set:

```
MyObj.DisplayAlerts=0
```

But when you have finished, don't forget to set this back to 1!

Excel as Printer Server

We started wanting to use Excel as a "printing machine", and in fact we are able to do so by automating really everything. For such a task, you would not of course want to set

```
MyObj.Visible=1
```

The "PageSetup" Object

Before executing the print command, maybe you want to specify a number of parameters important for print tasks. For this we need a "PageSetup" object:

```
MyPageSetup←MySheet.PageSetup
```

Specifying a Header

Now we can specify a centred header:

```
MyPageSetup.CenterHeader←'APL and Excel'
```

...or a left header:

```
MyPageSetup.LeftHeader←'Stephen Taylor'
```

...or a right header:

```
MyPageSetup.RightHeader←'Kai Jäger'
```

...or a footer which should be centred:

```
MyPageSetup.CenterFooter←'Written for Vector'
```

...or a left footer:

```
MyPageSetup.LeftFooter←'Page 1'
```

...or a right footer:

```
MyPageSetup.RightFooter←'2004-12-26'
```

Miscellaneous Print Properties

Maybe you do not want to print the whole spreadsheet but only part of it. For this, we again need a "Range" object:

```
StartPoint←MySheet.Cells.Item(1 1)  
EndPoint←MySheet.Cells.Item(1 0+pValues)  
MyRange←MySheet.Range(StartPoint EndPoint)  
MyPageSetup.PrintArea←MyRange.AddressLocal 0  
MyPageSetup.CenterHorizontally←0  
MyPageSetup.CenterVertically←1
```

Display Print Results

To display the result, we need to call the method "PrintPreview". This method also does not use an argument:

```
MySheet.PrintPreview 0
```

Got a DOMAIN ERROR?! We come back to this shortly.

Printing...

To print the sheet, merely call the method "PrintOut" (again with an empty vector as right argument).

```
MySheet.PrintOut 0
```

Coming to an End

If you want save the newly created spreadsheet, simply call the method "Save" As:

```
MySheet.SaveAs 'MyFilename.xml'
```

And finally we want quit Excel:

```
MyOleObj.Quit
```

Reading a Spreadsheet from Disk

Finally you may have a problem very close to what we discussed so far:

You (that is, your APL program) knows about an existing spreadsheet, or at least you expect there is such a spreadsheet, and now you want to read this data into your workspace. Of course you need the name and the folder where the spreadsheet can be found, but you prompt your user for that.

There are different ways to read the data from a spreadsheet into APL: you can use DDE (which is a wobbly thing); or you can use ODBC; or you can use Excel via OLE. Although the ODBC solution has some advantages (Excel is not needed, and it's fast), we will only discuss the OLE solution here.

We already know how to load a specific file into Excel:

```
file←'C:\My Documents\My Spreadsheet.xls'  
MyOleObj.MyWorkbook.Sheet←MyOleObj.MyWorkbook.Open file
```

As you can see, the Open method is able to load not only a template but also an ordinary spreadsheet.

Again: the "Range" Object

Again the UsedRange object could be used to access all data in the current worksheet. This is fine since we don't need to know anything about the data contained in that spreadsheet:

```
MyRange+MyOleObj.ActiveSheet.UsedRange  
Data+MyRange.Value 0
```

That's it!

Valuable Sources of Information

So far we can do some useful things, but for a more complex task we need a general overview over the Excel OLE object model, of course.

This information is available on the MSDN pages on the web by Microsoft. The link is quite easy to remember:

```
http://msdn.microsoft.com/library/default.asp?url=/library/  
en-us/dv\_wrcore/html/wrconExcelObjectModelOverview.asp
```

However, maybe you are interested in an alternative way: enter <http://msdn.Microsoft.com> into the browser's address field and then the words <excel object model> into the search field. Select http://msdn.microsoft.com/library/en-us/dv_wrcore/html/wrconExcelObjectModelOverview.asp from the result list.

Hints and Tips

Note that:

- there may or may not be some empty rows and columns around the real data
- some cells may contain a NULL
- Dates are returned as numeric vectors of length 7 – like 0TS!

To find out which cells are empty, use 0NULL:

```
Data=00NULL
```

If you need to read the data from the second sheet of a workbook instead of the first one:

```
(MyOleObj.Sheets.Item 2).UsedRange.Value 0
```

If there is no such sheet available, because that file consists of one sheet only, you will get a DOMAIN ERROR here.

However, this technique may be dangerous: another user may have moved the second worksheet to be the third one now. So it is maybe a better idea to use the name of the sheet you are interested in. For example, if you know that this worksheet is named "ABC", you can do this:

```
(MyOleObj.Sheets.Item 'ABC').UsedRange.Value 0
```

And of course you can use this syntax, too:

```
MySheet=MyOleObj.Sheets.Item 'ABC'  
MySheet.UsedRange.Value 0
```

A note on APL syntax:

```
MySheet.Range(StartPoint EndPoint).AddressLocal 0
```

gives a Length Error but

```
(MySheet.Range(StartPoint EndPoint)).AddressLocal 0
```

works just fine.

The F1 key in APL

If Visual Basic for Applications is installed together with Office, entering the name of an OLE item (object, property, method or event) and then pressing F1, should display a proper help file. However, on some but not all machines that does not work. Instead a message box is displayed stating that the help file could not be found:



(The exact file name and location will vary depending on the versions of Office and Windows running on your PC.)

We found that a common reason for this is that the file VBAXL10.chm is not found in *C:\Programme\Microsoft Office\Office11* but in *C:\Programme\Microsoft Office\Office11\1031\VBAXL10.CHM* instead.

This is really annoying! One possible solution is to simply copy the help file from *C:\Programme\Microsoft Office\Office11\1031* to *C:\Programme\Microsoft Office\Office11* (UK and US readers should look in Program Files).

Visual Basic Macros

A final suggestion. Sometimes you might need to do something but are unable to find out how from the documentation. In such a case it may be helpful to use a Visual Basic Macro: start "Recording Macro" within Excel, perform the needed actions, stop recording and then take a look into the Visual Basic code. Although there are some differences between Visual Basic and APL, this may still be helpful.

Interruptions and Trapping

If you need to use OLE automation within a real application, be aware that sometimes astonishing things may happen. Just because a certain operation works well 20 or 30 times does not mean that it will run at every time! Sometimes there are DOMAIN ERRORS or the like, that you are not able to reproduce or to explain, and of course this is really bad news. However, you will get a feeling for how to deal with the code over the time. Adding some explicit ":Trap" statements might be a good idea here, but you should avoid ":Trap 0".

For example: the user decides to "print" a document with the "Microsoft Document Image Writer" Office 2003 comes with. Then the user gets a "Save File" dialog, because this printer driver does not print but create a file you may display with a viewer. If for any reason the user then presses the "Cancel" button in this "Save" Dialog box, you get a DOMAIN ERROR in APL!

Therefore you should not use

```
MySheet.PrintOut 0
```

but:

```
:Trap 11 0 MySheet.PrintOut 0 0 :EndTrap
```

Conformity of APL Implementations to the ISO APL Standard Extended

by F.H.D. van Batenburg

Introduction

For many years I have been using more than one particular APL implementation. In the process of switching I got acquainted with many handy features found in one implementation and not in others. Some of those unique features I could very well do without, others I love so much that I miss them dearly in other APL implementations. Of course I tried to convince the vendor to add some of those features, but some vendors are interested in such user-requests, others less so.

Why the occasional reluctance to add a beautiful feature that is present in another implementation? I can imagine that pleas for namespaces or dynamic functions (as in Dyalog APL), APL-like structured control (as in APL2C), rank (as in SHARP APL and in APL2C), framed outer product (as in SHARP APL) or component file primitives (as in APLX) are not so trivial to implement, but implementation of assignment operator (Dyalog APL), left and right (APL2C) or indirect copy (APL2) do not seem so hard.

Pondering the question of what are reasonable additions to ask for in an APL implementation, I decided that the reasonable minimum set of features is what is defined in the ISO standard for APL.

It was for this reason that I started to investigate how well various APL implementations conform to the ISO standard.

APL ISO Standards

Currently there are two ISO standards for APL. The first one, ISO8485, was published in 1989 [7]. This standard defines "flat" APL without nested arrays. After this milestone, work started on the next standard, titled "Programming language APL, extended" to define APL with nested arrays. This resulted in a final draft thanks to the work of many, but not in the least thanks to the dogged perseverance of chairman Lee Dickey and the enthusiastic and vigorous debating desire and the hard work of Leigh Clayton. A few years ago this draft was accepted without any changes as ISO/IEC13751, so now we also have a standard

for APL nested arrays [8]. It was this second standard, with nested arrays, that I used as a yardstick for the various APL implementations.

The first thing I did was to go through the standard and jot down all the items that were not in the old APL\360 books. This resulted in a selection of items that I intended to check. After all, I had no intention of checking for the occurrence of basic items that have been in all APL implementations for decades, such as addition, subtraction, multiplication, reshape and similar functions. This did not mean that simply checking for unfamiliar symbols would be sufficient, there were also checks to be made on whether the existing symbols were implemented in the way the standard prescribes; in particular if the functionality of the standard extends that of the first standard. This browsing resulted in (a superset of) the list that you see in column one of the table.

Next I checked the six APL implementations that I had at my disposal (either the code or the manuals or Help files) for their implementation of each item in that list. As I said, I consulted the Help files or the manuals and often did a quick check by simple typing an appropriate expression. For APL2000 I consulted program and documentation of version 4 [1], for APLX program and documentation of version 1.1 [2], for APL2C program version 5.0.3, for Dyalog APL documentation and program version 10 [4], for APL2 I used the language reference and program version 2.0 [6] and finally for SHARP APL I used the Unix documentation suite [5] downloaded from internet with most manuals dated around 2000.

The results of my findings were coded by + and – marks in the table.

Implementation Scores

In general, investigating the various items in the table and assigning a proper plus/minus was not much of a problem. Most of the decisions I had to make were rather easy: a particular symbol was available or not.

If the symbol was available, the subtler question arose of whether the function worked as defined in the standard or not. Mostly this was also rather straightforward to answer with “yes”, but not always, e.g. the symbol of “Table” is defined in APLX, but its function is different from the standard. A similar case is GCD (∨) and LCM (^); these symbols are present in all implementations for the functions “and” and “or” but their extended functionality for GCD and LCM is not always implemented. Another case was ⌈⌋ that returned its result on screen in APL2C, but not as a result. I choose to consider this a simple bug that will be repaired, hopefully soon, rather than a design decision.

A few choices that had to be made were more questionable. Another investigator might have taken another position. Those choices are elaborated upon below.

First of all I decided not to take into account component files, shared variables and complex numbers. These functions are optional facilities in the standard and I felt that these should be omitted for this investigation. Maybe these features could be the topic of another article. Probably APL2 would have scored better if I had included these optional features. On the other hand, some other choices might have worked out in the reverse direction, e.g. error trapping.

Error trapping functionality was also simply taken from the standard. This ignores the fact that some APL implementations have a much more powerful way of error trapping so they have no need for the, probably in their view inferior, error trapping that is defined in the standard.

As could be expected the biggest problems rose with the scores of SHARP APL.

A non-issue was reduction. SHARP APL does this quite differently from other implementations. It manipulates frames, which roughly looks like it reduces over the first dimension (this is a superficial description, in essence the ramifications are more profound). So the initial plusses that I had for all the implementations regarding reduction suggested a universality that was not there. The standard however, defines the reduction operator in both ways, so I ended up with "plusses" for all implementations of reduction.

The real problem was nested arrays. SHARP APL has quite a different brand and uses different symbols (for sometimes comparable operations). Nested arrays in SHARP APL are similar to those in J. This type is sometimes called "boxed" arrays (because the data is boxed in and no longer available for manipulation except by specially designed operators). It is also called "grounded" arrays (because each enclosure increases the enclosure-level by one including scalars, whereas in ungrounded nested arrays you can enclose scalars as much as you want to without ever rising above level 0).

Theoretically speaking one could argue that SHARP APL has boxed arrays and can add nested arrays whenever they want to align to the standard. The reasons for this is that boxing is done by < and unboxing by >, which creates a new datatype "boxed array" rather than a special structure of familiar data. So one can have a nested array with an element of datatype boxed array. Eventually I chose to consider boxed arrays to be a rather weak type of nested arrays and awarded both minus and plus for "comparable" functions using other symbols.

In the end there appeared several items that were available in all implementations. These I removed from the table in order to show only the items that are still a problem for one or more vendors. Examples of such removed functions are "without" (the dyadic "~"), match and compress with integers (now called replicate). The results are presented in the following table.

	APL2000	APLX	APL2C	Dyalog APL	SHARP APL	IBM/APL2
Table: $\overline{\tau}$	—	— ¹⁾	—	—	+	—
Join: $\overline{\tau}$	+	+	—	+	+	—
N-wise reduction	+	+	+	+	—	+
Depth: $\equiv$	+	+	+	+	—	+
Each: $\overline{\tau}$	+	+	+	+	—	+
Enclose: ϵ	+	+	+	+	$\pm(<)$	+
Disclose: ρ	+	+	+	+	$\pm(>)$	+
Enlist: ϵ	+	+	+	+	—	+
First: $\uparrow$	+	+	+	+	—	+
Duplicate: $\overline{\tau}$	—	—	+	+	$\pm(\epsilon)$	—
Commute: $\overline{\tau}$	—	—	+	+	$\pm(\epsilon)$	—
Rank: $\overline{\tau}$	—	—	+	—	+	—
Unique: ν	—	—	—	+	$\pm(\uparrow)$	—
Left: $\leftarrow$	—	—	+	—	+	—
Right: $\rightarrow$	—	—	+	—	+	—
LCM: $\wedge$	—	—	+	—	+	—
GCD: $\vee$	—	—	+	—	+	—
EventMessage: $\square EM$	—	+	+	+	—	+
EventType: $\square ET$	—	+	—	—	—	+
EventSimulate: $\square ES$	—	+	—	—	—	+
Stop: $\square STOP$	+	+	—	+	+	+
Trace: $\square TRACE$	+	+	—	+	+	+
ExecuteAlt: $\square EA$	—	+	+	—	—	+
Transfer Form:	—	+	+	—	—	+

¹ Different implementation.

² Using $\square ML+2$

ITF						
Defined Operators	-	+	+	+	-	+
)NMS	-	+	+	-	-	+
)OPS	-	+	+	+	-	+
)SIC	+	+	+	-	+	+
)SINL	+	+	-	+	+	+
Total -	17	9	8	12	15.5 ³⁾	10

Results

The total score of ISO deviations is summarised in the bottom line of the table. For this summary I charged one penalty for a "-" and 0.5 for the "+/-" for boxed array functions.

You can see that in the end APL2C, APLX and APL2 have the best (lowest) score. At the other end of the spectrum are SHARP APL and APL2000 with the worst score. Dyalog APL lies more or less in between.

The good score of APL2C did not surprise me very much. The implementer Tilman Otto used the functionality of APL2 as a reference and in private conversations he expressed his interest in also agreeing with the ISO standard.

The third good rating, APLX was also no surprise. This APL implementation originates from APL68000 for Mac. For a long time this has been a very powerful APL for (first Atari and later) Macintosh and the MicroAPL team was (and is) always interested in listening to user comments.

Dyalog APL does not rate badly with 12, but not very well either. It has a lot of handy features that are not in the standard and I wish that other vendors would implement them, but being not ISO I will ignore them for the moment. On the other hand it misses several items of the standard. Some of those deficiencies are not too bad; for example the fact that)SIC is missing is not so much of a problem as)RESET gives the same functionality. Others such as left, right, LCM, GCD, rank and ITF are more serious deficiencies though.

The good score of APL2 was also not surprising. I already had the feeling that IBM its implementation of nested arrays has laid the groundwork for some discussions in the standards committee so it is not so surprising that the ISO

³ +/- was counted as 0.5

standard agrees not too badly with the IBM implementation (or is it the other way around?).

As expected, SHARP APL does not rate well. This is mainly due to its boxed arrays. This was to be expected, of course. Some people with more experience in boxed arrays than I have and who I trust as fairly objective assured me that in the long run one gets used to handling them. Nevertheless they also confessed to me that they found boxed arrays clumsier to handle and they would rather have nested arrays though. It would be interesting to see if many people share this opinion and if Soliton can be convinced to add nested arrays in the future. After all, they also felt the need after many years to standardise and implement the control structures in the ugly and non-APL-ish way that APL2000 has introduced.

It was a surprise to discover that the rating of APL2000 was so bad, worse even than the rating of SHARP APL. I can only hope that this might stimulate APL2000 to invest some effort in adding functionality to their implementation to atone for the ISO standard.

Conclusion

Did this survey bring me the enlightenment I was after? Partly, and partly not. I am now convinced that some of my wishes were not unreasonable; rank, left, right and commute are part of the standard and should be available in every APL. Some of my other wishes such as such as assignment operator, APL-ish control, indirect copying, namespace and dynamic functions should probably wait for the next "extra-extended" standard.

F.H.D. van Batenburg; Leiden, 11 June 2005

F.H.D. van Batenburg
Theoretical Biology of Leiden University
Kaiserstraat 63
2311GP Leiden
The Netherlands
email: Batenburg@rulsfb.LeidenUniv.nl
<http://wwwbio.LeidenUniv.nl/~Batenburg/index.html>

References

- [1] APL2000 (2001): 4.0 User manual. 4.0 System Functions manual.
Help files version 4
 - [2] APLX(2002): APL language manual version 1.1. Help files version 1.1
 - [3] APL2C (2001): User manual version 5
 - [4] Dyalog (2003): Language Reference manual version 10,
Help files version 10
 - [5] Soliton Inc (2000): SHARP APL for Unix; Documentation UW-000-0401 (9905).
 - [6] IBM (1994): Language Reference. SH21-1061-1.
APL2 language summary SX26-3851-00
 - [7] ISO (1989): Programming Language APL. ISO8485(1989).
 - [8] ISO (2001): Programming languages, their environments and system software
interfaces - Programming language Extended APL. ISO/IEC13751 (2001).
-

A Skeleton Key to Windows Performance Monitoring

by William Rutiser (a short talk at the APL2000 meeting, 2003)

These notes are appropriate to Windows 2000 Professional. Things are likely to be more or less different on other versions of Windows. Yes, the title is misleading; perhaps it should be "A Crowbar and a Roadmap to Some Starting Points....".

Begin with the "Start" Menu then select "Settings" then "Control Panel" then "Administrative Tools" then "Performance". Subsequently this will be condensed to select "Start | Settings | Control Panel | Administrative Tools | Performance". This will start the "Performance Tool". The Performance Tool is a snap-in for the *Microsoft Management Console* or MMC. MMC is a general-purpose facility; this tool is just one instance. It starts with two panes. The left pane contains a page named "Tree" that shows a tree-view starting at "Console Root". Select "System Monitor". This opens a promising looking graphical display in the right pane.

Selecting "Help | Help Topics | Index | System Monitor" will bring up the MMC Help viewer with a list of sub-topics. "Overview" and "interface" may be good starting points. Also see "settings reference", "toolbar reference", and "configurations, setting up".

What's going on here? The kernel keeps track of many different statistics of interest when analyzing performance. Current values of the statistics are available in *counters*. The System Monitor can display a graph of selected counters changing with time. There are a lot of counters to choose from and a lot of options to twiddle. There appears to be a way to save display configurations for future use.

We need to add some counters to the display.

Select "Help | Help Topics | Index | adding performance counters | System Monitor" to read how to do it. As an example try:

- Select the "New Counter Set" icon (leftmost in the System Monitor toolbar) to clear any selected counters.
- Right click the System Monitor details pane and select "Add Counters..."
- Wait patiently for the dialog.
- Select "Performance Object | Memory".
- Select "Select counters from list".
- Select "% Committed Bytes in use".
- Select "Explain". This shows the official definition of the counter.
- Select "Add" to add this counter to the displayed counter set.
- Repeat the last three steps for "Page Reads/sec" and "Page Writes/sec".
- Now select "Close".

You should now see an updating display with three coloured lines. At the bottom of the pane is a control with a row for each counter in the display set. Just above this are some statistics for the selected counter. Select different rows.

If your computer is not doing much, you should see relatively flat lines and generally low numbers. Start some applications to stir things up. Open a version 4.0 APL interpreter and clear to a large number. Add a version 5.0 interpreter. Try a large slow one liner. Watch the graphs and numbers change.

You should now be able to experiment with different configurations. Explore.

Index to Advertisers

Vector Back Numbers

4

All queries regarding advertising in VECTOR should be made to Gill Smith, at 01439-788385, Email: apl385@compuserve.com.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles (hardcopy with diskette as appropriate) to the Vector Working Group via:

Vector Administration, c/o Gill Smith
Brook House
Gilling East
YORK, YO62 4JJ
Tel: +44 (0) 1439-788385
Email: apl385@compuserve.com

Authors wishing to use Word for Windows should contact Vector Production for a copy of the APL2741 TrueType font, and a suitable Winword template. These may also be downloaded from the Vector web site at www.vector.org.uk

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members' news) should be sent to Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Please also copy us with all electronically submitted material so that we have early warning of possible problems.

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

2005/2006 Committee

Chairman	Paul Grosvenor 01293-562700 paul@optima-systems.co.uk	Optima House, Mill Court Spindle Way, CRAWLEY West Sussex, RH10 1TT
Secretary	Anthony Camacho 0117-9730036 acam@blueyonder.co.uk	11 Auburn Road Redland BRISTOL, BS6 6LS
Treasurer	Nicholas Small 01223-570850 treas.apl@bcs.org.uk	12 Cambridge Road, Waterbeach, Cambridge CB5 9NJ
Journal Editor	Stephen Taylor sjt@vector.org.uk	81 South Hill Park LONDON NW3 2SS
Education	Dr Alan Mayer 01792-295779 a.d.mayer@swansea.ac.uk	European Business Management School, Swansea University, Singleton Park, SWANSEA SA2 8PP
Activities	Ray Cannon 01252-874697 ray_cannon@compuserve.com	21 Woodbridge Rd, Blackwater Camberley, Surrey GU17 0BS
Projects	Ian Clark earthspot2000@hotmail.com	
Administration	Rowena Small 01223-570850 treas.apl@bcs.org.uk	12 Cambridge Road, Waterbeach, Cambridge CB5 9NJ

Journal Working Group

Editor:	Stephen Taylor	see above
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Support Team:	Anthony & Sylvia Camacho (0117-9730036), Ray Cannon (01252-874697), Stephen Taylor (077-1340 0852) Bob Hoekstra (01483-771028), Marc Griffiths, Ian Clark	

Typeset by APL-385 with MS Word for Windows

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for 'A Programming Language' - an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframes, workstations and personal computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

Causeway Graphical Systems Ltd
The Mallings, Castlegate
MALTON, North Yorks YO17 7DP
Tel: 01653-696760
Email: sales@causeway.co.uk
Web: www.causeway.co.uk

Dyalog Ltd
Grove House, Luyvers Close
Chineham Court, BASINGSTOKE
Hants RG24 0AC
Tel: 01256-338461
Email: sales@dyalog.com
Web: www.dyalog.com

Insight Systems ApS
Nordre Strandvej 119G
DK-3150 Hellebæk, Denmark
Tel: +45 70 26 13 28
Email: info@insight.dk
Web: www.insight.dk

MicroAPL Ltd
The Roller Mill, Mill Lane
Uckfield, E Sussex TN22 5AA
Tel: 01825-768050
Email: MicroAPL@microapldemon.co.uk
Web: www.microapl.co.uk/apl

Kx Systems
555 Bryant St. No. 375
Palo Alto CA 94301 USA
Tel: +1 866 kdb fast
Email: (general enquiries) info@kx.com
In Europe: Simon Garland: simon@kx.com

Optima Systems Ltd
Optima House, Mill Court, Spindle Way, Crawley,
West Sussex RH10 1HT
Tel: 01293 662700
Email: paul@optima-systems.co.uk

Compass Ltd
Compass House
60 Prestley Road
GUILDFORD, Surrey GU2 6YU
Tel: 01483-514500

APL2000 Inc
One Research Court
Suite 140
Rockville MD 20850, USA
Tel: +1 (800) 208 7150
Email: sales@apl2000.com
Web: www.apl2000.com

HMW Computing
Hamilton House
1 Temple Avenue
LONDON EC4Y 0HA
Tel: 0870-1010-469
Email: HMW@4xtra.com

Soliton Inc
44 Victoria Street, Suite 2100
Toronto, Ontario M5C 1Y2, Canada
Tel: +1 (416) 864 9355
Email: sales@soliton.com
Web: www.soliton.com

First Derivatives plc
First Derivatives House
Kilmorey Business Park, Kilmorey Street,
Nawry, Co. Down, N. Ireland BT34 2DH
Tel: 028 3025 2242
Email: enquiries@firstderivatives.com

The Carlisle Group
541 Jefferson Avenue, Scranton PA, 18510
USA
Tel: +1 (570) 963-2036
Web: www.carlislegroup.com